



DEPARTMENT OF ELECTRICAL
AND COMPUTER ENGINEERING

TIAGO DANIEL SANTOS GOUVEIA

BSc in Electrical and Computer Engineering

AUTOMATIC GENERATION OF SIMULATION-BASED DIGITAL TWINS FROM ASSET ADMINISTRATION SHELLS

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon
September, 2026



DEPARTMENT OF ELECTRICAL
AND COMPUTER ENGINEERING

AUTOMATIC GENERATION OF SIMULATION-BASED DIGITAL TWINS FROM ASSET ADMINISTRATION SHELLS

TIAGO DANIEL SANTOS GOUVEIA

BSc in Electrical and Computer Engineering

Supervisor: André Dionísio Bettencourt da Silva Parreira Rocha

Assistant Professor, NOVA School of Science and Technology - NOVA University of Lisbon

Co-supervisor: Nelson Nascimento de Freitas

PhD Student, NOVA School of Science and Technology - NOVA University of Lisbon

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon

September, 2026

Automatic Generation of Simulation-Based Digital Twins from Asset Administration Shells

Copyright © Tiago Daniel Santos Gouveia, NOVA School of Science and Technology, NOVA University Lisbon.

The NOVA School of Science and Technology and the NOVA University Lisbon have the right, perpetual and without geographical limitations, to archive and publish this dissertation in print or digital form, or by any other existing or future means, and to disseminate it through scientific repositories and to allow its copying and distribution for non-commercial educational or research purposes, provided that proper credit is given to the author and the publisher.

*To my parents, to Francis and to all the friends who made this
achievement possible.*

ACKNOWLEDGEMENTS

First of all, I want to thank my supervisor, Professor André Rocha, for taking me on for my internship and for placing his trust in me and my work. Thank you for all the experiences you gave me along the way, including company visits, presentations at Futurália, and university open days. Working with you made me grow a lot as a person.

To my co-supervisor, Nelson Freitas, I am really glad that I get to walk away with a lifelong friend from work. I have never met someone so similar to me and who could understand me so well. Thank you for all the patience, all the help with the thesis, and everything across the projects we worked on together. We started out battling console errors in ROS, moved on to fighting evil sorcerers with a hostage sheep, and ended up defending the homeland, at your service, Commander.

To the RICS research team, thank you all for the support and for welcoming me so well. A special thanks to Miguel, António, Marta, and Mafalda for all the laughs, help, and great times working together. I really enjoy being part of this team and look forward to everything we will keep achieving together.

To NOVA FCT, thank you for all these years. Looking back at my time here, with this great environment and the people around me, I honestly could not see myself anywhere else.

To my beloved Francis, when our eyes first met I could never have imagined what the future had in store for us. From unplanned dates to life's little coincidences, I have been so much happier ever since you came into my life. This work would simply never be done without you by my side, giving me all your love, endless support, and a good scolding whenever I needed it.

To Pinto, you have been here since the very beginning. Whenever I need an escape from reality, you are the one I turn to, and it has been amazing to watch each other grow up over all these years. A big thank you as well to the rest of the group, Cris, Delfa, and Gordo, for always being there.

To my university friends, a very special thank you to Raquel. When I hit rock bottom and felt like no one else was there, you were the one who reached out a hand and pulled me right back up. It is just a shame you are a bit smelly sometimes. To the rest of the Bilhas group, Nando, Raúl, Bia, Gui, Íuri, Lourenço, Tiago, and Gabriel, thank you for the

parties at Nando's house, the volleyball matches, the wild nights out, the "tá visto", and so many other unforgettable memories.

To my parents, thank you for your endless love and for always supporting me. Thank you for understanding when I was overwhelmed with work and respecting the space I needed, while always having my back. The education, values, and guidance you gave me made me the person I am today, and this achievement is just as much yours.

” *“The best way to make a system reliable is to make it
simple.”*

— **Dick Morley**
(Father of the Programmable Logic Controller)

ABSTRACT

The Industry 4.0 paradigm relies on Cyber Physical Systems (CPSs) and Cyber-Physical Production Systems (CPPSs) to achieve agile and resilient production. Within this landscape, Digital Twins (DTs) provide virtual representations capable of mirroring physical assets through real-time bidirectional telemetry. However, traditional development relies on proprietary simulation software, fragmented 3D Computer-Aided Design (CAD) files lacking kinematics, and tedious manual matching of Input/Output (I/O) addresses, making reconfigurations slow and error-prone.

To overcome these barriers, this dissertation presents an open, modular, model-driven framework for automated procedural generation and live synchronization of industrial DTs. Developed under the Design Science Research (DSR) methodology, the architecture spans four decoupled tiers: low-polygon asset digitization in Blender (Layer 0), no-code semantic authoring with 3D raycasting and Asset Administration Shell Package (AASX) packaging (Layer 1), containerized storage and discovery via Eclipse BaSyx v2 microservices (Layer 2), and procedural scene assembly, physics simulation, dual-mode execution, and drift correction in Unity 3D (Layer 3).

The framework was empirically validated on a modular Flexible Manufacturing System (FMS) bench driven by an Industrial Shields Programmable Logic Controller (PLC) running FreeRTOS, as well as through parametric stress testing across 800 simulation runs scaling from 5 to 1000 conveyor modules. Results demonstrate cold-boot instantiation of the physical pilot bench in under 2.8 s and steady-state rendering rates exceeding 480 Frames Per Second (FPS). Scalability testing confirms near-linear scaling up to 250 modules and stable real-time execution up to 500 modules (averaging over 90 FPS), establishing model-driven Asset Administration Shell (AAS) standards as an effective foundation for next-generation DTs.

Keywords: Industry 4.0 · Cyber-Physical Systems · Procedural Simulation · Virtual Commissioning · Digital Twin · Asset Administration Shell

RESUMO

O paradigma da Indústria 4.0 assenta na convergência de CPSs e CPPSs para alcançar processos produtivos ágeis e resilientes. Neste ecossistema, os DTs fornecem réplicas virtuais capazes de espelhar ativos físicos através de telemetria bidirecional em tempo real. No entanto, o ciclo de vida da engenharia de DTs industriais enfrenta elevadas barreiras de implementação. O desenvolvimento convencional depende de software de simulação proprietário, modelos 3D CAD fragmentados desprovidos de semântica cinemática e sensorial, e mapeamento manual exaustivo de endereços de I/O, tornando a reconfiguração frequente proibitivamente morosa e sujeita a erros.

Para superar estas limitações, esta dissertação propõe uma arquitetura aberta, modular e orientada a modelos para a instanciação procedimental e sincronização em tempo real de DTs industriais. Desenvolvida sob a metodologia DSR, a solução estrutura-se em quatro camadas desacopladas: digitalização e modelação 3D de baixo custo poligonal (Camada 0), aplicação desktop sem código para mapeamento sensorial via *raycasting* 3D e empacotamento AASX (Camada 1), registo dinâmico e repositório de pacotes baseado em microserviços Eclipse BaSyx v2 em Docker (Camada 2), e montagem procedimental de cenários, simulação física de contactos, execução em modo duplo (Virtual Commissioning (VC) e telemetria em direto) e compensação cinemática de desvio em Unity 3D (Camada 3).

A metodologia foi validada empiricamente numa bancada mecatrónica didáctica modular FMS baseada em componentes *fischertechnik*, controlada por um PLC Industrial Shields com FreeRTOS, bem como através de testes de stress paramétricos ao longo de 800 execuções entre 5 e 1000 módulos transportadores. Os resultados experimentais comprovam a instanciação completa da bancada piloto em cerca de 2,8 s, sustentando taxas médias de renderização superiores a 480 FPS. Os ensaios de escalabilidade confirmam complexidade quase linear até 250 módulos e execução estável em tempo real até 500 módulos (acima de 90 FPS), demonstrando a viabilidade da AAS como norma unificadora para o gémeo digital industrial.

Palavras-chave: Indústria 4.0 · Sistemas Ciber-Físicos · Simulação Procedimental · Comissionamento Virtual · Digital Twin · Asset Administration Shell

DISCLOSURE OF USAGE OF GENERATIVE ARTIFICIAL INTELLIGENCE

The author, Tiago Daniel Santos Gouveia, acknowledges the use of the following generative artificial intelligence tools in the development of this thesis. These technologies were utilized under strict supervision, and all AI-generated outputs were critically reviewed and verified for accuracy. The author accepts full responsibility for the validity and originality of the content, confirming that the use of these tools aligns with all institutional policies and standards of academic integrity.

Generative AI tools used: Gemini 3.6 Flash.

Activities supported by Generative Artificial Intelligence Tools:¹

Conceptualization

☐ Idea generation

☐ Defining the research objective

☐ Formulating research questions and hypotheses

☐ Feasibility assessment and risk evaluation

☐ Hypothesis viability assessment

☐ Simulated debate and argument testing

Literature Review

☐ Search and Discovery

☒ Literature summarization and synthesis

☐ Concept Mapping and Systematization

☐ Market/patent landscape analysis

☐ Gap Identification and Novelty Evaluation

☐ Cross-lingual literature comprehension

Methodology

☐ Experimental Design Optimization

☒ Development of experimental or research protocols

☐ Methodological Instrument Selection

Software Development and Automation

☒ Code Generation

☒ Debugging and Repair

☐ Algorithm design

☐ Refactoring and Optimization

☐ Process automation

☐ Code documentation and comment generation

¹This list extends the GAIDeT taxonomy.

Data Management

- | | | |
|---|--|--|
| ☐ Data collection | ☐ Data analysis | ☐ Synthetic data generation |
| ☐ Validation | ☐ Quantitative Plotting and Charting | ☐ De-identification and anonymization support |
| ☐ Data cleaning | ☐ Reproducibility and rerun checks | ☐ Audio-to-text transcription and diarization |
| ☐ Data curation and organization | ☐ Data labeling and annotation assistance | |

Visuals and Multimedia

- | | | |
|---|--|--|
| ☐ Synthetic Asset Generation | ☐ Image Enhancement and Editing | ☐ Diagrammatic and Schematic Design |
|---|--|--|

Writing and Editing

- | | | |
|--|---|--|
| ☐ Drafting Text | ☐ Formulation of conclusions | ☐ Citation formatting and bibliography management |
| ☒ Polishing and Editing | ☐ Tone Adjustment | ☐ Press releases and outreach materials |
| ☐ Abstract and Executive Summary Generation | ☐ Translation | ☐ Title Generation |

Ethics Review

- | | | |
|--|---|--|
| ☐ Bias analysis and discrimination assessment | ☐ Ethical risk analysis | ☐ Data confidentiality monitoring |
| | ☐ Monitoring compliance with ethical standards | |

Quality Assurance

- | | | |
|--|---|--|
| ☐ Simulated Peer Review | ☐ Identification of limitations | ☐ Publication support |
| ☐ Consistency checking against field trends | ☐ Publication strategy and journal selection | |

CONTENTS

Abstract	vii
Resumo	viii
Disclosure of Usage of Generative Artificial Intelligence	ix
Contents	xi
List of Figures	xv
List of Tables	xviii
Acronyms	xix
1 Introduction	1
1.1 Background and Contextualization	1
1.2 Motivation and Problem Statement	2
1.3 Research Question and Problem Breakdown	3
1.4 Research Objectives	3
1.5 Approach and Contributions	4
1.6 Research Methodology	5
1.7 Dissertation Structure	5
2 State of the Art	6
2.1 Evolution of Production Systems in Industry 4.0	6
2.1.1 Manufacturing Paradigms: From Dedicated to Reconfigurable Systems	6
2.1.2 Enabling Technologies of Smart Manufacturing	7
2.1.3 Reference Architectural Model Industrie 4.0 (Reference Architectural Model Industry 4.0 (RAMI 4.0))	8
2.2 The Digital Twin Paradigm in Manufacturing	9
2.2.1 Historical Evolution of the Paradigm	9
2.2.2 Taxonomy of Virtual Representations: Digital Model (DM), Digital Shadow (DS), and DT	10
2.2.3 The DT Lifecycle	11

2.2.4	Reference Architectures for Industrial DTs	12
2.3	Semantic Modeling and Interoperability Standards	12
2.3.1	AAS Architecture and Metamodel	13
2.3.2	Standardized Submodel Templates and Industrial Digital Twin Association (IDTA) Specifications	14
2.3.3	Portable Packaging: The AASX Container	14
2.3.4	AAS Middleware and Toolchains	15
2.4	Virtual Environments and Game Engines in Industry 4.0	16
2.4.1	Game Engines for Industrial Simulation	16
2.4.2	Bridging CAD and Real-Time Graphics	16
2.4.3	Behavioral Simulation and VC	17
2.4.4	Industrial Communication in the Game Engine Loop	17
2.5	Automated Generation of DTs	19
2.5.1	Bottlenecks in Manual Workflows	20
2.5.2	Model-Driven Approaches	20
2.5.3	Data-Driven Approaches	22
2.5.4	Generative Artificial Intelligence (AI) and Large Language Models	22
2.6	Identified Research Gaps and Opportunities	23
3	Conceptual System Architecture	25
3.1	Design Requirements and Constraints	25
3.1.1	Functional Requirements	26
3.1.2	Non-Functional Requirements	27
3.2	Architectural Overview	27
3.2.1	Upstream Stage: Layer 0 - Physical Asset Digitization	30
3.2.2	Layer 1 - Asset Modeling	31
3.2.3	Layer 2 - Storage and Registry	33
3.2.4	Layer 3 - Runtime Execution and Physical Synchronization	34
3.2.5	Cross-Layer Information and Interaction Flows	36
3.3	Asset Data Model Specification	37
3.3.1	AssetDescriptor (Core Entity)	38
3.3.2	VisualStructure Class	39
3.3.3	BehaviorMapping Class	40
3.3.4	VirtualPins Class	42
3.3.5	ControlLogic Class	44
3.3.6	NetworkEndpoint Class	44
3.4	Chapter Summary and Architectural Transition	45
4	Technological Implementation	46
4.1	System Architecture and Technology Realization	47
4.1.1	Physical Testbed Context: The Modular FMS Bench	47

4.1.2	Overall Component Architecture and Inter-Layer Interfaces . . .	48
4.2	Layer 0: Physical Asset Digitization and 3D Modeling	50
4.3	Layer 1: Asset Modeling Desktop Application	51
4.3.1	AAS Data Model & Submodel Specifications	51
4.3.2	Modeling Tool Graphical User Interface (GUI) and 3D Viewport Architecture	52
4.3.3	Interactive Modeling Workflow and Surface Raycasting	53
4.3.4	Integrated Rule Editor for Control Logic	54
4.4	Layer 2: Storage and Registry (Eclipse BaSyx v2)	56
4.4.1	Microservice Architecture and Port Allocation	58
4.4.2	Package Ingestion and Registry Resolution	58
4.5	Layer 3: Runtime Execution and Physical Synchronization	58
4.5.1	Dynamic Ingestion Flow and Procedural Scene Construction . . .	59
4.5.2	Conveyor Physics Mechanics, Dual-Mode Execution, and Drift Cor- rection	60
4.6	Physical Automation Setup and Embedded Firmware	62
4.7	Implementation Traceability and Requirement Coverage	63
4.8	Chapter Summary	65
5	Results and Evaluation	66
5.1	Demonstrative Case Study: End-to-End Asset Onboarding	66
5.1.1	CAD Design and Kinematic Rigging in Blender	67
5.1.2	3D Model Ingestion and Hierarchy Verification	67
5.1.3	Interactive Pin Mapping and Control Logic Definition	67
5.1.4	Control Logic Rule Definition	68
5.1.5	AASX Compilation and Repository Publication	69
5.1.6	Zero-Touch Procedural Instantiation and Live Telemetry	70
5.2	Empirical Validation: Modular FMS Pilot Bench	71
5.3	High-Density Scalability Stress Testing	73
5.3.1	Benchmark Methodology and Experimental Test Rig	73
5.3.2	Startup Initialization and Procedural Instantiation Complexity . .	74
5.3.3	Rendering Fluidity and Frame Rate Degradation	75
5.3.4	Unit Computational Processing Cost	76
5.3.5	Dynamic Temporal Stability and Statistical Correlations	77
5.4	Discussion and Limitations	78
5.4.1	Systematic Verification of Research Objectives	79
6	Conclusion and Future Work	81
6.1	Conclusions and Summary of Contributions	81
6.1.1	Operational Scope and Process Boundaries	82
6.2	Future Work	82

6.2.1	Physics Acceleration via Unity Data-Oriented Technology Stack (DOTS) and Massively Parallel Compute Shaders	83
6.2.2	Robotic Manipulation Integration via Robot Operating System (ROS) and ROS 2	83
6.2.3	Expansion to Continuous and Hybrid Non-Discrete Manufacturing Systems	83
6.2.4	Extensible Control Logic Interpreters: C, Python, Scratch, and Node-RED	84
6.2.5	Multi-Agent System (MAS) Coordination and Autonomous Negotiation	84
6.2.6	Immersive Interfaces: Virtual Reality and Augmented Reality (Virtual Reality (VR)/Augmented Reality (AR))	85
	Bibliography	86
	Appendix A: Physical FMS Controller Specifications and I/O Pin Mapping	91
A.1	M-DUINO 57R+ Controller Technical Specifications	91
A.2	Digital Sensor Inputs (I-Registers)	92
A.3	Relay Actuator Outputs (R-Registers)	92

LIST OF FIGURES

2.1	Evolution from Industry 1.0 to Industry 4.0 [17].	7
2.2	The three-dimensional architecture of RAMI 4.0 [22].	8
2.3	Timeline of DT evolution across industrial eras [1].	10
2.4	Taxonomy of virtual representations classified by automated data flow: (a) Digital Model, (b) Digital Shadow, and (c) Digital Twin [5].	11
2.5	Industry 4.0 Component and AAS structure [22].	13
2.6	Digital description of a modular conveyor within the official AASX Package Explorer [22].	15
2.7	Technical configurations of VC: Model-in-the-Loop (MiL), Software-in-the-Loop (SiL), and Hardware-in-the-Loop (HiL) [11].	17
2.8	Architecture and data flow in an educational DT using Unity 3D, AAS, and Open Platform Communications Unified Architecture (OPC UA) [6].	19
2.9	Data fragmentation across engineering tools in machine development [8].	20
2.10	Architecture of the model-based platform for automated DT generation [10].	21
2.11	DT code generation process for NVIDIA Omniverse using Large Language Models (LLMs) [44].	22
2.12	Framework for automated AAS submodel generation from technical documentation using LLM agents [45].	23
3.1	Three-layer conceptual system architecture of the proposed DT framework.	29
3.2	Unified Modeling Language (UML) Component Diagram of Layer 0 (Physical Asset Digitization).	31
3.3	UML Activity Diagram of Layer 1 (Asset Modeling).	32
3.4	UML Activity Diagram of Layer 2 (Storage and Registry).	34
3.5	UML Activity Diagram of Layer 3 (Runtime Execution and Physical Synchronization).	36
3.6	UML Class Diagram of the conceptual asset metamodel and constituent submodel classes.	38
3.7	UML Class Diagram of the core <i>AssetDescriptor</i> entity.	38

3.8	UML Class Diagram of the <i>VisualStructure</i> hierarchy (<i>VisualStructure</i> , <i>SceneObject</i> , and <i>Transform</i>).	39
3.9	UML Class Diagram of the <i>BehaviorMapping</i> hierarchy (<i>BehaviorMapping</i> and <i>BehaviorBinding</i>).	41
3.10	UML Class Diagram of the <i>VirtualPins</i> hierarchy (<i>VirtualPins</i> , <i>PinGroup</i> , and <i>Pin</i>).	43
3.11	UML Class Diagram of the <i>ControlLogic</i> class.	44
3.12	UML Class Diagram of the <i>NetworkEndpoint</i> class.	45
4.1	Physical modular FMS conveyor testbed built with fischertechnik mechatronic modules, showing the conveyor tracks, magnetic sensors, punch mechanism, and Industrial Shields M-DUINO PLC.	47
4.2	UML Component Diagram of the framework implementation architecture	49
4.3	UML Component Diagram of Layer 0 (Physical Asset Digitization).	50
4.4	Comparison between the physical modular FMS conveyor module and its reconstructed 3D simulation model in Blender.	51
4.5	UML Component Diagram of Layer 1 (Asset Modeling).	51
4.6	GUI of the Asset Modeling application showing the interactive 3D viewport, configuration panels, and model structure hierarchy.	52
4.7	UML Sequence Diagram of the Layer 1 interactive asset modeling workflow.	54
4.8	User Interface (UI) of the no-code <i>ControlLogic</i> rule editor.	55
4.9	UML Sequence Diagram of the <i>ControlLogic</i> rule configuration and deployment workflow.	56
4.10	UML Component Diagram of Layer 2 (Storage and Registry).	57
4.11	UML Deployment Diagram of the Eclipse BaSyx v2 Docker deployment and container topology.	57
4.12	UML Component Diagram of Layer 3 (Runtime Execution and Physical Synchronization).	59
4.13	UML Sequence Diagram of the Unity 3D runtime execution environment.	61
4.14	3D visualization and DT runtime running in Unity 3D.	61
4.15	UML Sequence Diagram of the embedded FreeRTOS firmware execution.	62
5.1	Interactive virtual pin mapping in the desktop onboarding wizard, showing: (a) sensor placement using surface raycasting, and (b) kinematic actuator binding.	68
5.2	Standardized AASX container for Conveyor G uploaded and registered in the Eclipse BaSyx v2 repository, displaying its submodels in the AAS Viewer.	69
5.3	Procedural scene assembly and operational DT execution in Unity 3D, showing Conveyor G integrated into the reconstituted simulation scene.	70
5.4	Distribution of startup initialization time and steady-state rendering frame rate across 100 experimental runs of the modular FMS pilot bench.	72

5.5	Dynamic execution timeline of the modular FMS bench, contrasting frame rate with frame latency over simulation time.	73
5.6	Procedural benchmark simulation layout showcasing the modular arrangement of conveyors and integrated punch stations (1 punch mechanism per 5 conveyors).	74
5.7	Procedural scaling complexity: total startup initialization time versus unit instantiation cost per conveyor module.	75
5.8	Rendering fluidity and frame rate degradation across conveyor module counts, showing mean steady-state FPS, standard deviation, and 1% low FPS.	76
5.9	Unit computational processing cost per conveyor module in microseconds (μs /conveyor).	76
5.10	Dynamic simulation fluidity over the benchmark execution window across all evaluated module counts.	77
5.11	Pearson linear correlation matrix computed across procedural scalability metrics.	78

LIST OF TABLES

2.1	Comparative analysis of state-of-the-art automated DT frameworks.	24
4.1	Traceability matrix mapping architectural layers and technical components to design requirements.	64
5.1	Experimental results for the modular FMS pilot bench across 100 test runs. .	71
5.2	Consolidated stress test results across 800 benchmark runs (Mean $\pm$ Standard Deviation).	74
A.1	Hardware specifications of the Industrial Shields M-DUINO 57R+ PLC controller.	91
A.2	Digital sensor input pin mapping on the Industrial Shields M-DUINO 57R+ controller.	92
A.3	Relay actuator output pin mapping on the Industrial Shields M-DUINO 57R+ controller.	93

ACRONYMS

AAS	Asset Administration Shell (pp. vii , viii , xii , xiii , xv , 2–6, 9, 10, 13–15, 17–19, 21–24, 46, 48, 51, 53, 56, 58, 59, 63, 64, 66, 67, 69, 78, 79, 81–85)
AASX	Asset Administration Shell Package (pp. vii , viii , xii , xiii , xv , xvi , 3, 4, 6, 14, 15, 24, 46, 48, 51, 55, 56, 58, 63–65, 69, 71, 73, 75, 79)
AGV	Automated Guided Vehicle (p. 33)
AI	Artificial Intelligence (pp. xii , 6, 20, 22, 23)
AML	AutomationML (pp. 12, 13, 21, 24)
AMR	Autonomous Mobile Robot (p. 21)
API	Application Programming Interface (pp. 13, 16, 22, 27, 28, 33, 37, 63)
AR	Augmented Reality (pp. xiv , 5, 85)
AST	Abstract Syntax Tree (p. 55)
BPMN	Business Process Model and Notation (p. 22)
B-Rep	Boundary Representation (p. 16)
CAD	Computer-Aided Design (pp. vii , viii , xii , xiii , 2, 6, 10–12, 16, 20–22, 24, 28, 30, 31, 50, 67, 81)
CNC	Computer Numerical Control (p. 7)
CPPS	Cyber-Physical Production System (pp. vii , viii , 1, 2, 6, 7, 9, 23, 25, 47)
CPS	Cyber Physical System (pp. vii , viii , 2, 7)
CPU	Central Processing Unit (pp. 76, 79, 82, 83)
DIN	Deutsches Institut für Normung (p. 8)
DM	Digital Model (pp. xi , 9–11, 28)
DMS	Dedicated Manufacturing System (p. 6)
DOTS	Data-Oriented Technology Stack (pp. xiv , 79, 83)
DS	Digital Shadow (pp. xi , 2, 10, 11)
DSR	Design Science Research (pp. vii , viii , 1, 5, 81)

DT	Digital Twin (pp. vii , viii , xi , xii , xv , xvi , xviii , 1–6 , 8–12 , 14 , 16 , 17 , 19–27 , 29 , 31 , 33 , 34 , 45–47 , 51 , 58 , 60 , 61 , 65–67 , 70–72 , 75 , 76 , 78 , 80–85)
ECAD	Electronic Computer-Aided Design (p. 20)
ECLASS	Standardized Product Classification and Description (pp. 12 , 13)
ECS	Entity Component System (p. 83)
EN	European Standard (p. 8)
FIFO	First-In, First-Out (pp. 62 , 63)
FMI	Functional Mock-up Interface (pp. 17 , 21 , 82 , 83)
FMS	Flexible Manufacturing System (pp. vii , viii , xii–xiv , xvi–xviii , 7 , 46 , 47 , 50 , 51 , 66 , 71–73 , 78 , 81 , 91 , 92)
FMU	Functional Mock-up Unit (pp. 17 , 21 , 24 , 83)
FPS	Frames Per Second (pp. vii , viii , xvii , 16 , 71 , 72 , 74–79 , 81 , 82)
GLB	Binary glTF (pp. 14 , 16 , 46 , 50 , 52 , 55 , 59 , 64 , 65 , 67 , 69 , 70 , 73)
glTF	GL Transmission Format (pp. 14 , 16 , 48 , 50 , 64)
GPIO	General-Purpose Input/Output (p. 62)
GPU	Graphics Processing Unit (pp. 66 , 83)
GUI	Graphical User Interface (pp. xiii , xvi , 48 , 51–53 , 56)
HiL	Hardware-in-the-Loop (pp. xv , 17)
HTTP	Hypertext Transfer Protocol (pp. 4 , 15 , 33 , 44 , 46 , 48 , 55 , 58–60 , 62 , 63 , 69)
I/O	Input/Output (pp. vii , viii , xiv , 3 , 4 , 24 , 25 , 28 , 31 , 37 , 42 , 43 , 48 , 79 , 81 , 91 , 92)
IDTA	Industrial Digital Twin Association (pp. xii , 2 , 6 , 13–15 , 51)
IEC	International Electrotechnical Commission (pp. 2 , 6 , 8 , 9 , 12–14 , 24 , 51 , 55)
IIoT	Industrial Internet of Things (p. 7)
IoT	Internet of Things (p. 24)
IP	Internet Protocol (pp. 33 , 44 , 52 , 53 , 64)
ISO	International Organization for Standardization (p. 12)
JSON	JavaScript Object Notation (pp. 14 , 52 , 53 , 55 , 63 , 84)
LLM	Large Language Model (pp. xv , 22 , 23)
MaaS	Manufacturing as a Service (p. 8)
MAS	Multi-Agent System (pp. xiv , 84)
MES	Manufacturing Execution System (p. 20)
MiL	Model-in-the-Loop (pp. xv , 17)
MQTT	Message Queuing Telemetry Transport (pp. 9 , 15 , 16 , 18 , 21 , 44 , 58 , 84 , 85)

OPC	Open Packaging Convention (pp. 14 , 69)
OPC UA	Open Platform Communications Unified Architecture (pp. xv , 9 , 10 , 12 , 13 , 15 , 16 , 18 , 19 , 24)
PDF	Portable Document Format (p. 22)
PLC	Programmable Logic Controller (pp. vii , viii , xvi , xviii , 2 , 4 , 5 , 11 , 17 , 18 , 20 , 28 , 31 , 35 , 43 , 44 , 46–49 , 52 , 55 , 58 , 60 , 62–64 , 68 , 70 , 78 , 81 , 85 , 91)
PLM	Product Lifecycle Management (pp. 9 , 20)
PubSub	Publish-Subscribe (p. 12)
RAG	Retrieval-Augmented Generation (p. 23)
RAM	Random Access Memory (p. 66)
RAMI 4.0	Reference Architectural Model Industry 4.0 (pp. xi , xv , 2 , 6 , 8 , 9 , 12)
REST	Representational State Transfer (pp. 4 , 9 , 13 , 15 , 16 , 18 , 33 , 44 , 48 , 51 , 55 , 58 , 63 , 64 , 69 , 70)
RMS	Reconfigurable Manufacturing System (pp. 7 , 25)
ROS	Robot Operating System (pp. xiv , 83)
SDK	Software Development Kit (pp. 46 , 48 , 51 , 69)
SiL	Software-in-the-Loop (pp. xv , 17 , 24)
SME	Small and Medium-sized Enterprise (pp. 2 , 19)
SysML	Systems Modeling Language (p. 21)
TCP	Transmission Control Protocol (pp. 4 , 44 , 46 , 48 , 58 , 60 , 62–64 , 70 , 79 , 82 , 85)
UI	User Interface (pp. xvi , 38 , 55)
UML	Unified Modeling Language (pp. xv , xvi , 31–41 , 43–45 , 48–51 , 53–59 , 61 , 62)
URDF	Unified Robot Description Format (p. 83)
URI	Uniform Resource Identifier (p. 13)
USD	Universal Scene Description (p. 16)
UUID	Universally Unique Identifier (p. 13)
VC	Virtual Commissioning (pp. viii , xii , xv , 2 , 17 , 24 , 46 , 48 , 52 , 54 , 58 , 60 , 65 , 79 , 82 , 83)
VR	Virtual Reality (pp. xiv , 85)
VRAM	Video Random Access Memory (p. 66)
VTK	Visualization Toolkit (p. 52)
XML	Extensible Markup Language (pp. 12 , 14)
XR	Extended Reality (p. 16)

INTRODUCTION

This chapter introduces the research context, motivation, and core objectives of this dissertation. As industrial automation transitions toward flexible Cyber-Physical Production Systems (CPPSs), automated creation and synchronization of interactive Digital Twins (DTs) becomes essential for rapid factory reconfiguration.

The chapter is organized as follows:

- [Section 1.1](#) outlines the industrial background, smart manufacturing paradigms, and standard reference architectures.
- [Section 1.2](#) presents the engineering bottlenecks in traditional simulation workflows and formulates the problem statement.
- [Section 1.3](#) details the main research question and decomposes the problem into three investigative sub-problems.
- [Section 1.4](#) outlines the primary research objective and the specific goals guiding this work.
- [Section 1.5](#) describes the proposed four-layer framework architecture and summarizes the main technical contributions.
- [Section 1.6](#) presents the Design Science Research (DSR) methodology followed in this work.
- [Section 1.7](#) describes the overall structure of the remaining chapters in this dissertation.

1.1 Background and Contextualization

Modern factories need to adapt quickly to changing market demands. In traditional manufacturing, production lines were built to make large volumes of a single product with dedicated machinery. However, today's industrial systems must be modular and reconfigurable, allowing machines to be added, moved, or swapped to produce smaller

batches of customized items [1]. This flexibility is made possible by Cyber Physical Systems (CPSs) and CPPSs, which connect physical shop-floor equipment with computers, sensors, and real-time networks [2, 3].

In this context, the DT has become an important tool for Virtual Commissioning (VC), operational monitoring, and testing control logic without interrupting real production [4]. While static 3D models or Digital Shadows (DSs) only offer one-way data display, a true DT maintains automated, two-way communication with the physical machine [5]. Telemetry updates the virtual model in real time, while simulation decisions can send commands back to physical controllers [6].

To help equipment from different vendors share data, the Plattform Industrie 4.0 community established the Reference Architectural Model Industry 4.0 (RAMI 4.0) reference architecture. Within RAMI 4.0, the Asset Administration Shell (AAS) (International Electrotechnical Commission (IEC) 63278), maintained by the Industrial Digital Twin Association (IDTA), serves as the standard digital representation for physical assets [7]. An AAS organizes mechanical properties, operational variables, and communication settings into standardized submodels, providing a common format to access machine data.

1.2 Motivation and Problem Statement

Even though DTs offer clear benefits, building and maintaining them in practice is still difficult, especially for Small and Medium-sized Enterprises (SMEs) [8]. Traditional development pipelines face three main bottlenecks:

First, 3D geometry is fragmented across engineering tools. Most factory machines are designed using detailed Computer-Aided Design (CAD) files intended for mechanical manufacturing. These files contain millions of polygons and internal components, such as bolts and brackets, which slow down real-time game engines. At the same time, standard CAD exports lack kinematic joint definitions, movement limits, and trigger colliders needed to detect workpieces in physics simulations [9].

Second, the toolchains used to build simulation scenes are mostly manual and vendor-locked. Automation engineers have to import 3D meshes into proprietary simulation software, manually set up coordinate systems, and script motion behaviors by hand. When a physical machine is rearranged or rewired on the factory floor, the simulation scene must be manually rebuilt in the editor, which creates maintenance delays and causes the virtual model to become outdated [10].

Third, simulation tools are often decoupled from control hardware. Standard models rarely include electrical pin mappings or control rules. Moving between offline VC and an operational DT connected to physical Programmable Logic Controllers (PLCs) requires remapping signal tags across different software tools by hand, which is tedious and prone to configuration mistakes [11].

These limitations explain why commercial simulation platforms struggle in agile, reconfigurable production lines [3]. Because proprietary tools depend on pre-compiled

scenes, static component catalogs, and manual address mapping, even minor adjustments to physical equipment require time-consuming manual intervention. To overcome these barriers, this research explores how open, standardized information models can replace manual workflows and drive automated simulation pipelines [12].

1.3 Research Question and Problem Breakdown

This dissertation investigates how standardized AASs can be used to generate and synchronize interactive DTs automatically. The core research question guiding this work is:

Research Question: *How can standardized AASs be used to procedurally assemble and synchronize interactive, simulation-based DTs at runtime?*

To address this central question, the overall research problem is decomposed into three focused sub-problems:

- **Semantic Asset Integration:** Structuring 3D geometry, kinematic motion bounds, electrical Input/Output (I/O) pins, and declarative control logic rules within a unified, vendor-neutral AAS submodel schema.
- **Procedural Scene Assembly:** Ingesting containerized AAS packages at runtime, dynamically constructing 3D visual hierarchies, setting up physical colliders, and configuring network endpoints in memory without pre-compiled scenes.
- **Real-Time Performance and Scalability:** Decoupling asynchronous network telemetry streams from the fixed physics loop and compensating for kinematic spatial drift to maintain bounded real-time performance as layout module counts scale.

1.4 Research Objectives

The primary objective of this dissertation is to design, build, and test an open framework that procedurally generates and synchronizes industrial DTs using standardized AAS packages.

To achieve this goal, four specific objectives were defined:

1. **Definition of a Semantic Asset Metamodel:** Establish an open data model using AAS submodels to capture 3D visual structures, kinematic motion limits, virtual I/O pins, declarative control rules, and network addresses inside portable Asset Administration Shell Package (AASX) files.
2. **Development of a Visual Asset Modeling Pipeline:** Build a desktop authoring application that allows engineers to configure 3D assets, place virtual sensors on

model surfaces interactively, and export valid AASX packages without writing software code.

3. **Implementation of a Procedural Simulation Runtime:** Create a real-time runtime engine in Unity 3D that queries the semantic repository, instantiates 3D meshes and colliders in memory, and operates in both virtual commissioning and operational DT modes.
4. **Empirical Validation and Scalability Evaluation:** Validate the framework on a physical modular conveyor testbed, followed by automated stress tests scaling from 5 to 1000 modules to measure startup times, frame rates, and latency limits.

1.5 Approach and Contributions

This work uses a four-layer architecture to connect physical factory equipment with real-time virtual simulation:

In Layer 0 (Geometric Preparation), physical machine dimensions are measured and turned into lightweight 3D models in Blender. Coordinate origins and movement axes are aligned with physical pivots, and unnecessary interior geometry is removed.

In Layer 1 (Semantic Authoring), a Python desktop application lets users import the 3D model, configure machine properties, and place sensors on surfaces using raycasting. The tool then builds and exports an AASX package containing the 3D mesh and all required submodels.

In Layer 2 (Middleware and Storage), Eclipse BaSyx v2 microservices running in Docker containers store the uploaded AASX packages and host Representational State Transfer (REST) registries so client applications can discover available machines on the network.

In Layer 3 (Runtime Simulation), a Unity 3D engine downloads the models from BaSyx at startup, builds the 3D scene in memory, and connects to an Industrial Shields PLC using Transmission Control Protocol (TCP) and Hypertext Transfer Protocol (HTTP) to synchronize movements and sensor states.

The main contributions of this dissertation are:

1. **An Open Model-Driven Framework and Procedural Pipeline:** A four-layer architecture that replaces static, pre-compiled simulation files by procedurally generating interactive virtual environments directly from AAS packages at runtime.
2. **A Unified AAS Submodel Schema:** A portable metamodel specification linking 3D meshes, kinematic joint limits, virtual I/O pins, and IF-THEN control logic rules into standardized submodel structures.
3. **An Interactive No-Code Authoring Tool:** A desktop application incorporating 3D surface raycasting for visual sensor placement and automated AASX bundle generation without manual programming.

4. **Empirical Evaluation and Performance Benchmarks:** Quantitative experimental validation on physical mechatronic hardware and parametric stress tests up to 1000 conveyor modules, establishing realistic latency, startup time, and frame rate bounds for procedurally assembled DTs.

1.6 Research Methodology

To achieve the research objectives, this dissertation follows the DSR methodology [13], an established framework for solving engineering problems by designing, developing, and evaluating practical software artifacts. The work was divided into four main phases:

In Phase 1 (Problem Analysis and Literature Review), existing work on Industry 4.0, DT taxonomies, and semantic standards was reviewed to identify the limitations of current simulation tools (Chapter 2).

In Phase 2 (Architecture and Metamodel Design), the overall four-layer architecture, requirements, and AAS submodel schemas were specified (Chapter 3).

In Phase 3 (Implementation), the complete toolchain was implemented, including 3D asset preparation in Blender, the PyQt6 and PyVista authoring tool, Eclipse BaSyx microservices, the Unity 3D procedural runtime, and FreeRTOS firmware on an Industrial Shields PLC (Chapter 4).

In Phase 4 (Experimental Evaluation), the framework was tested on the physical conveyor testbed and evaluated through stress benchmarks across multiple layout scales to assess startup times, frame stability, and synchronization behavior (Chapter 5).

1.7 Dissertation Structure

The remainder of this dissertation is organized into five chapters:

Chapter 2 (State of the Art) reviews related work on smart manufacturing, DTs, the AAS standard, game engine simulation, and existing automated model generation methods.

Chapter 3 (Conceptual System Architecture) presents the four-layer system architecture, functional requirements, and detailed specifications for each AAS submodel.

Chapter 4 (Technological Implementation) describes the software and hardware implementation of each layer, covering the authoring tool, containerized BaSyx services, the Unity runtime loader, and PLC firmware.

Chapter 5 (Results and Evaluation) presents the experimental findings from the physical testbed and the scalability stress tests, analyzing startup duration, rendering frame rates, and latency limits.

Chapter 6 (Conclusion and Future Work) summarizes the findings, discusses current limitations, and outlines future research directions, including multi-threaded physics, robotic arm integration, multi-agent coordination, and Augmented Reality (AR) interfaces.

STATE OF THE ART

This chapter presents the background, standards, and related work for this dissertation. It reviews the main technologies behind smart manufacturing, DTs, AASs, and simulation environments, identifying the key gaps this work addresses.

The chapter is organized into six sections:

- [Section 2.1](#) describes the evolution of manufacturing systems, the main enabling technologies, and the RAMI 4.0 reference architecture.
- [Section 2.2](#) explains the DT concept, its history, data flow classifications, lifecycle stages, and general architectures.
- [Section 2.3](#) focuses on semantic modeling using the AAS (IEC 63278), submodel templates from the IDTA, the AASX package format, and open-source tools.
- [Section 2.4](#) discusses the use of game engines in industrial automation, CAD conversion challenges, kinematic simulation, and network communication.
- [Section 2.5](#) reviews existing methods for generating DTs automatically, grouping them into model-driven, data-driven, and Artificial Intelligence (AI)-based methods.
- [Section 2.6](#) discusses current limitations in the literature and outlines the research opportunities addressed by this dissertation.

2.1 Evolution of Production Systems in Industry 4.0

Manufacturing systems have evolved through several stages, from early mechanization to modern automated and networked factories [14, 15]. The fourth industrial revolution, or Industry 4.0 [15], connects physical shop-floor machinery with digital systems, forming what are known as CPPSs.

2.1.1 Manufacturing Paradigms: From Dedicated to Reconfigurable Systems

In the 20th century, factories mostly relied on Dedicated Manufacturing Systems (DMSs). These systems used fixed production lines to produce large volumes of identical products

at low cost. However, they were difficult and expensive to change when product designs evolved [1, 3].

To handle wider product varieties, Flexible Manufacturing Systems (FMSs) were developed, using automated handling and Computer Numerical Controls (CNCs) to produce pre-defined part families. Later, Reconfigurable Manufacturing Systems (RMSs) were introduced to allow faster and cheaper changes. An RMS is designed to be modular, integrable, customizable, scalable, convertible, and diagnosable [1]. In an RMS, machines, conveyors, and workstations can be added, removed, or rearranged as needed. To support this level of flexibility without long downtimes, software systems must be able to discover, configure, and connect new equipment automatically, without manual reprogramming of simulation scenes or controller software [3, 16].

Figure 2.1 shows the progression across the four industrial revolutions, moving from steam-powered machines to modern networked systems.

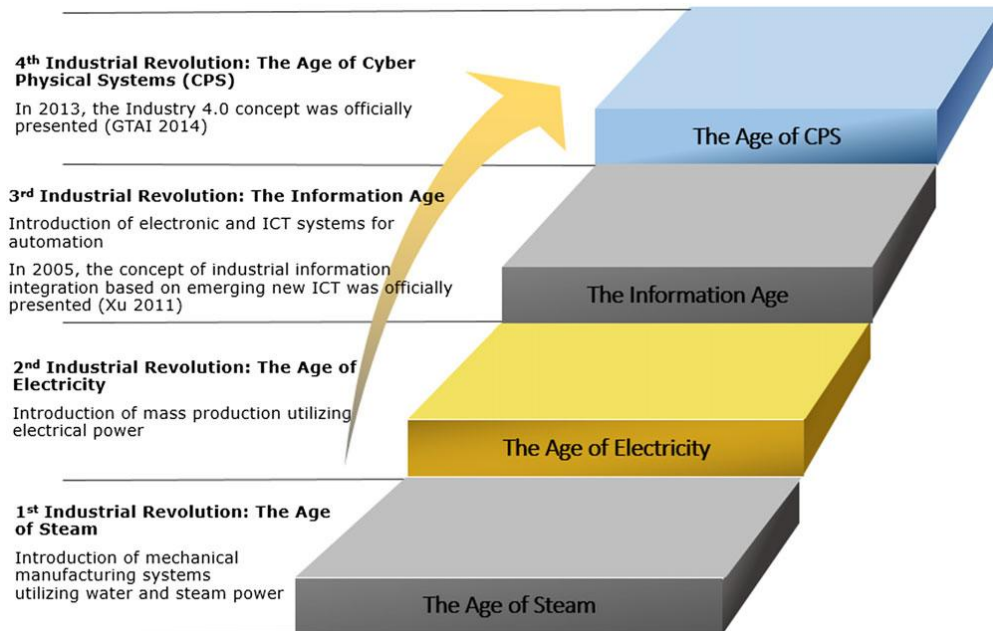


Figure 2.1: Evolution from Industry 1.0 to Industry 4.0 [17].

2.1.2 Enabling Technologies of Smart Manufacturing

Several core technologies work together to make CPPSs possible:

- **CPSs and CPPSs:** Connect physical machinery with sensors, microcontrollers, and communication networks [2], allowing devices to coordinate actions and collect real-time data [18].
- **Industrial Internet of Things (IIoT) and Edge Computing:** Provides reliable networking between shop-floor devices and controllers. Processing data near the machines reduces network load and communication delays [19, 20].

- **Cloud Computing and Manufacturing as a Service (MaaS):** Enables storage and heavy computing tasks in the cloud, helping companies share production resources and run plant-wide optimization algorithms [17, 19].
- **Semantic Interoperability and Information Modeling:** Uses standardized data models so that different software tools can share mechanical, electrical, and operational machine data without translation errors [7, 21].
- **The DT:** Serves as a digital representation of a physical asset, combining real-time sensor data, 3D visualization, and simulation models [4, 20].

2.1.3 Reference Architectural Model Industrie 4.0 (RAMI 4.0)

Because Industry 4.0 involves many different standards and vendors, Plattform Industrie 4.0 defined RAMI 4.0 (Deutsches Institut für Normung (DIN) European Standard (EN) IEC 63088). RAMI 4.0 provides a common three-dimensional model to organize industrial assets across their entire lifecycle, factory hierarchy, and software layers [22], as shown in Figure 2.2.

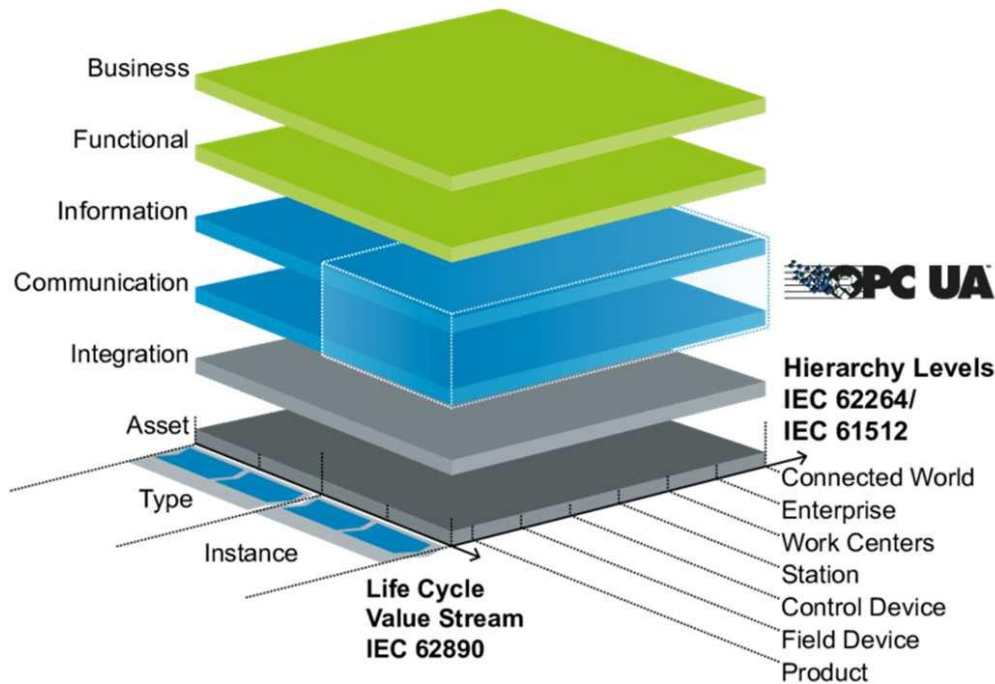


Figure 2.2: The three-dimensional architecture of RAMI 4.0 [22].

The three axes of RAMI 4.0 are defined as follows:

1. **Layers (Vertical Axis):** Divides industrial systems into six viewpoints:
 - *Asset Layer:* The physical equipment, including machines, tools, cables, and operators.

- *Integration Layer*: The boundary that connects physical hardware to computers using sensors, barcodes, and interfaces.
 - *Communication Layer*: Communication protocols and network connections, such as Open Platform Communications Unified Architecture (OPC UA), Message Queuing Telemetry Transport (MQTT), and REST.
 - *Information Layer*: Data models, parameters, and variable definitions.
 - *Functional Layer*: Control logic, application code, and simulation models.
 - *Business Layer*: Enterprise systems, production orders, and overall process rules.
2. **Life Cycle and Value Stream (Horizontal Axis)**: Based on IEC 62890, this axis separates the design phase (“Type”) from the operational phase (“Instance”), helping track an asset from initial engineering to decommissioning.
 3. **Hierarchy Levels (Third Axis)**: Extends the standard automation levels (IEC 62264 and IEC 61512) to include: *Product, Field Device, Control Device, Station, Work Centers, Enterprise, and Connected World*.

In RAMI 4.0, any asset that has a digital identity and connects to the network is called an *Industry 4.0 Component*. It consists of the physical asset combined with its digital representation, called the AAS [2, 22]. The AAS links the Asset and Integration layers, providing a standard way to access machine data and functions.

2.2 The Digital Twin Paradigm in Manufacturing

In modern manufacturing, DTs have emerged as a core pillar of CPPSs. While CPPSs define the operational infrastructure integrating sensors, actuators, and distributed control on the shop floor, the DT provides the high-fidelity virtual representation and computational intelligence that mirrors, evaluates, and optimizes physical operations [4, 23]. However, despite their clear advantages for monitoring, testing, and optimization, the term is frequently applied loosely across literature to describe any 3D animation or simple dashboard [5]. A clear taxonomy and historical foundation are therefore essential to distinguish simple virtual representations from fully synchronized DTs.

2.2.1 Historical Evolution of the Paradigm

The idea of using Digital Models (DMs) to mirror real systems dates back to NASA’s Apollo program, where ground simulators replicated spacecraft conditions during missions [1]. The term “Digital Twin” was introduced in 2002 by Michael Grieves for Product Lifecycle Management (PLM) [20, 24].

In 2012, NASA and the U.S. Air Force defined the DT as an integrated multi-physics simulation of a vehicle or system that uses sensor data and operational history to mirror the

life of its physical twin [25]. As illustrated in the timeline from Khan et al. [1] (Figure 2.3), this evolution progressed through key developmental milestones:

- **Conceptual Origins and Theorization (1970 to 2010):** Early ideas emerged from aerospace mirroring in the 1970s, followed by the formal introduction and theorization of the “Digital Twin” concept by Michael Grieves in 2002 and 2003 for product lifecycle management.
- **Formal Definition and Consolidation (2010 to 2016):** NASA established the first formal engineering definition in 2012, initiating a period of consolidation where the paradigm expanded into broader manufacturing and cyber-physical automation.
- **Industrial Scaling, Open Standards, and Smart Applications (2021 to Present):** Recent years mark the transition toward large-scale industrial deployment, driven by interoperable open standards, such as AAS and OPC UA, sustainability objectives, and domain-wide adoption ranging from reconfigurable factories to smart cities [1, 26].

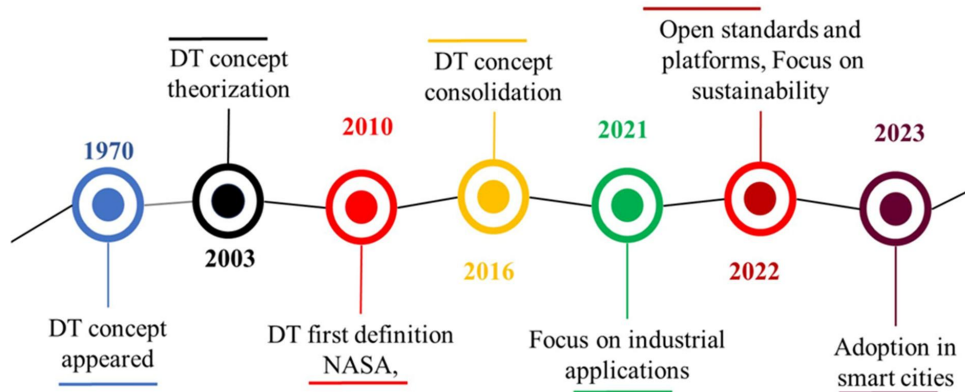


Figure 2.3: Timeline of DT evolution across industrial eras [1].

2.2.2 Taxonomy of Virtual Representations: DM, DS, and DT

To clarify what constitutes a DT, this dissertation follows the classification from Kritzinger et al. [5], which separates virtual representations by the level of automated data flow between the physical and digital systems.

2.2.2.1 Digital Model (DM)

A DM has manual data exchange in both directions (Figure 2.4a). Static CAD drawings, offline kinematic studies, and spreadsheets are typical examples. If changes are made to the physical machine, an engineer must manually update the software model. Likewise, changes made in the model must be implemented by hand on the shop floor.

2.2.2.2 Digital Shadow (DS)

A DS has automated, one-way data flow from the physical asset to the digital object. (Figure 2.4b). Sensors and controller tags send live data to the virtual model, so the digital representation updates automatically when the machine moves or changes state. However, the data flow is one-way: the DS does not send control commands back to the machine.

2.2.2.3 Digital Twin (DT)

A DT has automated, two-way data flow between the physical asset and the digital representation (Figure 2.4c). Sensor readings and controller states update the virtual model automatically, while simulations or decisions made in the digital environment can send commands back to the physical controller to change its operation.

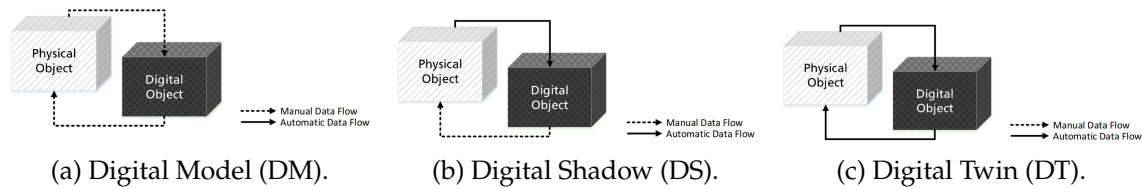


Figure 2.4: Taxonomy of virtual representations classified by automated data flow: (a) Digital Model, (b) Digital Shadow, and (c) Digital Twin [5].

Under this definition, many industrial systems marketed as DTs are strictly DSs, because they only display live telemetry without providing bidirectional control or automated feedback to the physical equipment. Establishing a true DT requires closing this loop, enabling both offline virtual testing and real-time bidirectional synchronization.

2.2.3 The DT Lifecycle

A DT evolves with the machine over its lifecycle [1, 20], passing through five main phases:

1. **Design and Modeling:** Starts as a DM during early engineering, where CAD shapes and expected kinematics are tested before building the machine.
2. **Virtual Commissioning:** Tests controller programs against the virtual model before physical installation, using software emulation or physical PLCs to catch logic errors early [11].
3. **Operational Connectivity:** The machine is installed and connected to the DT, establishing live data links to mirror actual movements and states.
4. **Optimization and Control:** Operating data is used to improve performance, detect faults, and adjust setpoints on the physical machine [4].
5. **Reconfiguration and Decommissioning:** When lines are reorganized or parts replaced, the DT updates its structure to match the new physical setup.

2.2.4 Reference Architectures for Industrial DTs

Several reference architectures have been proposed for DTs [23]. Tao et al. [4] describe a five-part model: physical entity, virtual entity, services, data, and connection links. Talkhestani et al. [18] present an intelligent architecture that adds software agents for plug-and-produce operations.

In line with these concepts, this dissertation organizes the digital twin workflow into four distinct functional stages:

- **Physical Layer:** Machinery, sensors, conveyors, and controllers operating on the factory shop floor.
- **Authoring Layer:** Tools that ingest raw geometric models and enrich them with kinematic limits, sensor triggers, and operational specifications.
- **Storage and Registry Layer:** Central services that store standardized asset descriptors, maintain device metadata, and provide on-demand endpoint discovery over standard network interfaces.
- **Simulation and Runtime Layer:** The virtual physics and rendering environment that dynamically instantiates assets, executes internal behavior logic, and maintains live two-way synchronization with physical controllers.

2.3 Semantic Modeling and Interoperability Standards

To operationalize the RAMI 4.0 reference architecture (Section 2.1.3) and enable automated DT generation, industrial assets must be transformed into *Industry 4.0 Components* equipped with standardized digital representations and machine-readable interfaces. In heterogeneous manufacturing environments, achieving seamless semantic interoperability requires a structured stack of open standards [16, 27]:

- **AutomationML (AML) (IEC 62714):** An Extensible Markup Language (XML)-based neutral format focused on modeling plant topology, equipment hierarchies, and CAD references during the offline engineering phase [21, 27].
- **OPC UA (IEC 62541):** A service-oriented communication architecture providing real-time data transport, Publish-Subscribe (PubSub) messaging, and information model browsing across shop-floor devices [6, 27].
- **Standardized Product Classification and Description (ECLASS) (International Organization for Standardization (ISO) 13584):** An international classification system and dictionary providing globally unique identifiers for industrial product properties and technical specifications.

While formats like AML handle offline engineering data and OPC UA manages network transport, neither provides a complete digital lifecycle wrapper. To unify these domain-specific standards into a single accessible model, Plattform Industrie 4.0 introduced the AAS (IEC 63278) [2, 22]. The AAS acts as the standardized digital wrapper for Industry 4.0 components, encapsulating properties, 3D assets, communication endpoints, and behavioral logic into a unified container accessible via standard REST Application Programming Interfaces (APIs) across the entire asset lifecycle.

2.3.1 AAS Architecture and Metamodel

The AAS serves as the digital representation of an industrial asset, containing its identification, technical data, and software interfaces [2, 22], as shown in Figure 2.5.

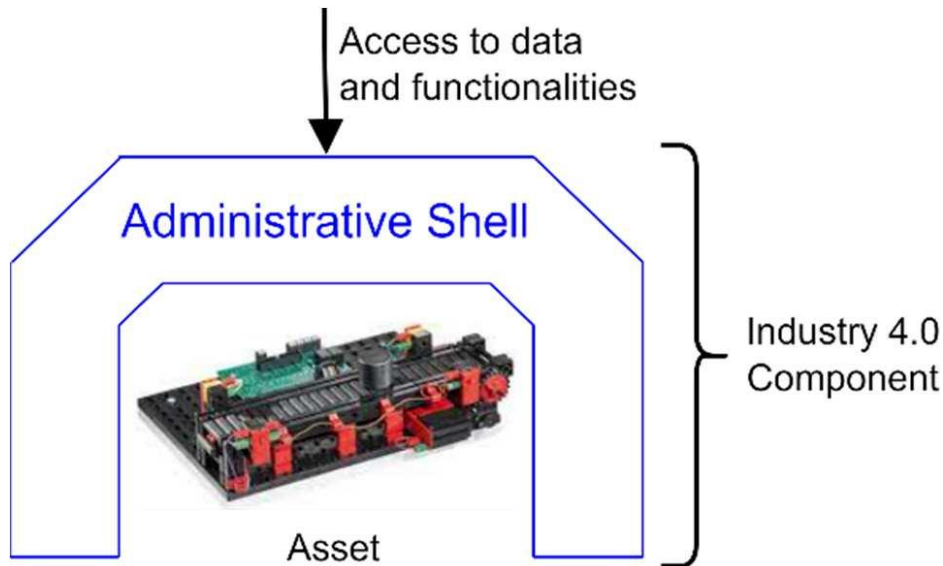


Figure 2.5: Industry 4.0 Component and AAS structure [22].

The AAS metamodel, standardized under IEC 63278 and specified by the IDTA, defines a structured hierarchy for representing asset properties [22, 28]. It includes the following key parts:

- **AAS Container:** The top-level container, identified by a unique ID (Uniform Resource Identifier (URI) or Universally Unique Identifier (UUID)).
- **Asset Information:** States whether the asset is a generic product type or a specific physical unit on the shop floor.
- **Submodel:** A group of related properties for a specific domain, such as identification, technical specs, or kinematics. Each submodel has an identifier and a reference to a standardized semantic definition (such as ECLASS or IDTA templates).
- **Submodel Elements:** The actual data fields within a submodel [27, 28]:

- *Property*: Stores a single value, such as a number, text, or boolean.
- *MultiLanguageProperty*: Stores text in multiple languages.
- *Range*: Sets minimum and maximum limits for a numerical value.
- *File* and *Blob*: References or embeds files, such as 3D meshes, user manuals, or drawings.
- *SubmodelElementCollection* and *SubmodelElementList*: Groups multiple submodel elements together.
- *Operation*: Defines invokable functions with input and output parameters.

2.3.2 Standardized Submodel Templates and IDTA Specifications

The main benefit of the AAS is that different systems can read and interpret data without custom integration code when adhering to standardized *Submodel Templates* released by the IDTA [2, 16]. Well-established templates frequently analyzed in literature include:

- **Digital Nameplate (IEC 61406)**: Replaces physical nameplates with machine-readable serial numbers, manufacturer details, and conformity marks.
- **Technical Data**: Provides a standardized frame for operating limits, dimensions, voltages, and ratings for industrial equipment.
- **Contact Information**: Standardizes manufacturer and service contact details for maintenance and lifecycle support.

While these templates cover static administrative metadata, official submodel specifications for real-time physics dynamics, kinematic joint constraints, and executable control logic remain absent or fragmented in current standards [22, 29]. Consequently, integrating behavioral, spatial, and geometric aspects into standardized semantic containers represents a significant open research gap in Industry 4.0 literature.

2.3.3 Portable Packaging: The AASX Container

To make it easy to share DT assets across different computers and tools, the standard defines the AASX package (IEC 63278-1). The AASX format uses the Open Packaging Convention (OPC), the same container format used by common office documents.

An AASX package bundles:

1. The AAS and its submodels saved as JavaScript Object Notation (JSON) or XML.
2. Embedded binary files, such as 3D meshes (Binary glTF (GLB)/GL Transmission Format (glTF)), wiring diagrams, and icons.
3. Relationship files that link the data together.

Using AASX allows an asset to be distributed as a single file. An application can download the package and immediately load its 3D model, joints, pin mappings, and connection addresses without needing extra files.

Figure 2.6 illustrates how a physical conveyor module is traditionally modeled inside the standard IDTA toolchain using the official AASX Package Explorer [22]. In conventional workflows, the automation engineer must manually populate extensive hierarchical tables, defining properties, ranges, and submodel element collections without visual feedback or spatial awareness of the physical asset.

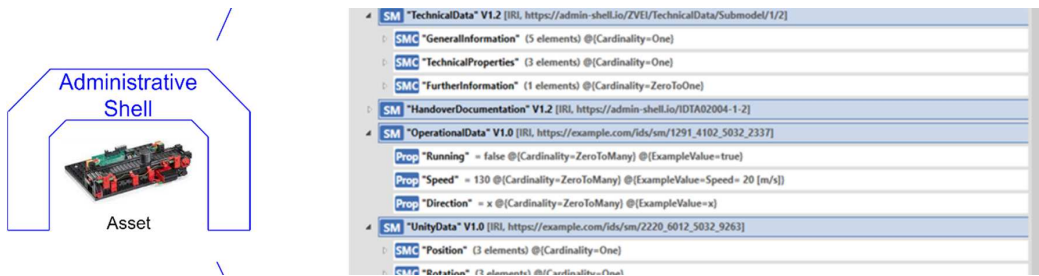


Figure 2.6: Digital description of a modular conveyor within the official AASX Package Explorer [22].

2.3.4 AAS Middleware and Toolchains

To use AASs during operation, server software must host them and provide network access through REST, HTTP, and MQTT:

- **Eclipse BaSyx:** An open-source implementation of Plattform Industrie 4.0. It provides modular services including:
 - *AAS Server:* Hosts AAS data and provides standard REST endpoints.
 - *AAS Registry:* Keeps track of where each asset is hosted so applications can find them.
 - *DataBridge:* Connects AAS properties directly to field protocols like OPC UA and MQTT [29, 30].
- **AASX Package Explorer:** A graphical tool used to create, inspect, and validate AASX files [22].
- **Development Libraries:** Open-source libraries such as `aas-core` and `basysx-python-sdk` allow software applications to read and write AAS data across Python, C#, and TypeScript.

2.4 Virtual Environments and Game Engines in Industry 4.0

While older factory monitoring used flat 2D screens, modern systems increasingly rely on 3D environments that accurately display robot motions, part transfers, and layout configurations [31, 32].

2.4.1 Game Engines for Industrial Simulation

Real-time game engines, especially Unity and Unreal Engine, have become popular choices for industrial DTs, offering several practical benefits:

1. **Rendering Speed:** Game engines are optimized for graphics hardware, rendering complex 3D scenes smoothly at 60 Frames Per Second (FPS) or higher.
2. **Flexible Scripting:** Unlike closed industrial simulation software, engines like Unity provide open programming environments (C#) that make it straightforward to add network libraries for MQTT, OPC UA, or REST APIs.
3. **Lower Licensing Barriers:** Standard game engine licenses are generally more accessible than specialized industrial simulation software (such as Siemens Tecnomatix [31]).
4. **Cross-Platform Support:** Projects can run on desktop computers, web browsers, or Extended Reality (XR) headsets [33].

Platforms like NVIDIA Omniverse (based on the Universal Scene Description (USD) format) are also growing, but Unity remains widely used in research due to its lightweight runtime, physics engine, and extensive documentation [6, 10, 33].

2.4.2 Bridging CAD and Real-Time Graphics

A common problem identified in the literature is that mechanical CAD models are not designed for real-time simulation engines [9, 11, 32].

CAD files use Boundary Representations (B-Reps) to store exact production details, like screw threads and internal components. In contrast, game engines need lightweight triangle meshes to keep high frame rates. Preparing CAD models for real-time simulation typically involves three main steps [9, 11]:

- **Tessellation and Simplification:** Converting curved surfaces into triangles and reducing polygon counts while keeping the visual shape.
- **Removing Internal Parts:** Deleting hidden parts, like screws and bolts, that are not needed for motion or collisions.
- **Standard Export Formats:** Saving meshes into open formats like glTF/GLB 2.0, which load quickly and keep the model hierarchy organized.

2.4.3 Behavioral Simulation and VC

Beyond visual display, a DT must replicate machine motion and sensor behavior [11]. This is essential for VC, where controller programs are tested against a virtual model before the physical machine is assembled [34].

Common VC setups include:

- **Model-in-the-Loop (MiL):** Controller logic and mechanical dynamics are simulated together within early conceptual design tools.
- **Software-in-the-Loop (SiL):** Both the machine kinematics and the PLC runtime code are simulated in software on a computer.
- **Hardware-in-the-Loop (HiL):** A physical PLC is wired or networked to the simulation computer. The game engine feeds virtual sensor signals to the PLC inputs and reads the PLC actuation outputs.

As classified by Striffler and Voigt [11], Figure 2.7 outlines the spectrum of virtual commissioning topologies, contrasting purely software-based simulation loops (such as MiL and SiL) against real-time physical HiL coupling, where physical controllers exchange sensor and actuator signals with the virtual model.

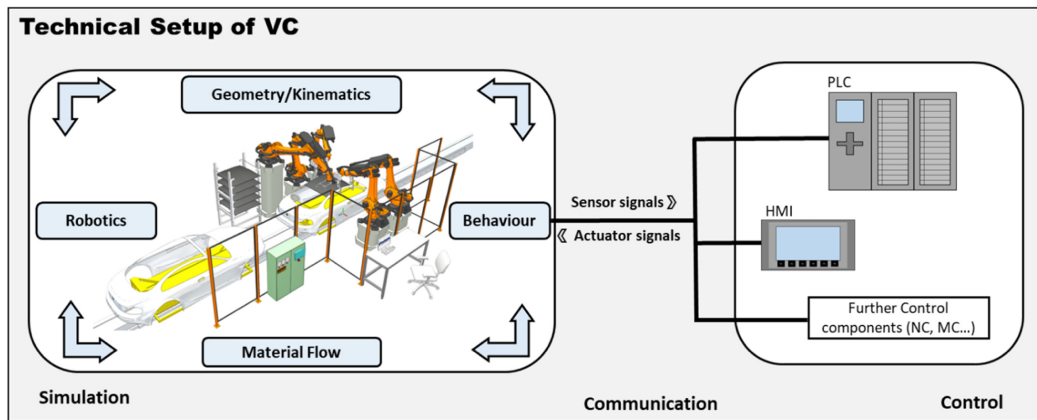


Figure 2.7: Technical configurations of VC: MiL, SiL, and HiL [11].

When complex multi-body physics equations are needed, game engines can use the Functional Mock-up Interface (FMI) standard [9]. External simulation models compiled as Functional Mock-up Units (FMUs) compute the physics, while the game engine handles visualization. Juhlin et al. [35] showed how combining AASs with FMUs allows cloud-based simulation of motor drives.

2.4.4 Industrial Communication in the Game Engine Loop

Connecting a game engine to industrial hardware requires care with thread timing. Game engines run on a continuous loop that updates graphics every frame (often 60 to 120 times per second) and updates physics at fixed intervals (typically 50 Hz, or every 20 ms).

If network queries (OPC UA, MQTT, or REST) run directly on the main graphics thread, network delays or packet retransmissions can block the engine loop and degrade rendering performance [31].

To ensure smooth runtime performance, industrial game engine architectures decouple communication onto background worker threads [6, 31, 33]:

1. **Background Threads:** Network listeners run on separate threads or coroutines, receiving incoming sensor and controller frames asynchronously.
2. **Thread-Safe Data Exchange:** Incoming states are placed into thread-safe queues or buffers to safely transfer data to Unity's main thread.
3. **Decoupled Physics Execution:** The engine's fixed physics loop reads the buffered states at regular intervals, preventing communication latency from stalling visual and mechanical simulation.

Figure 2.8 shows an example architecture from Pajpach et al. [6], where Unity 3D retrieves 3D models from an AAS repository and exchanges runtime signals with a physical PLC through an OPC UA server and Node-RED middleware. Similarly, Garcia et al. [33] used Unity 3D with BaSyx AAS data spaces for remote line monitoring.

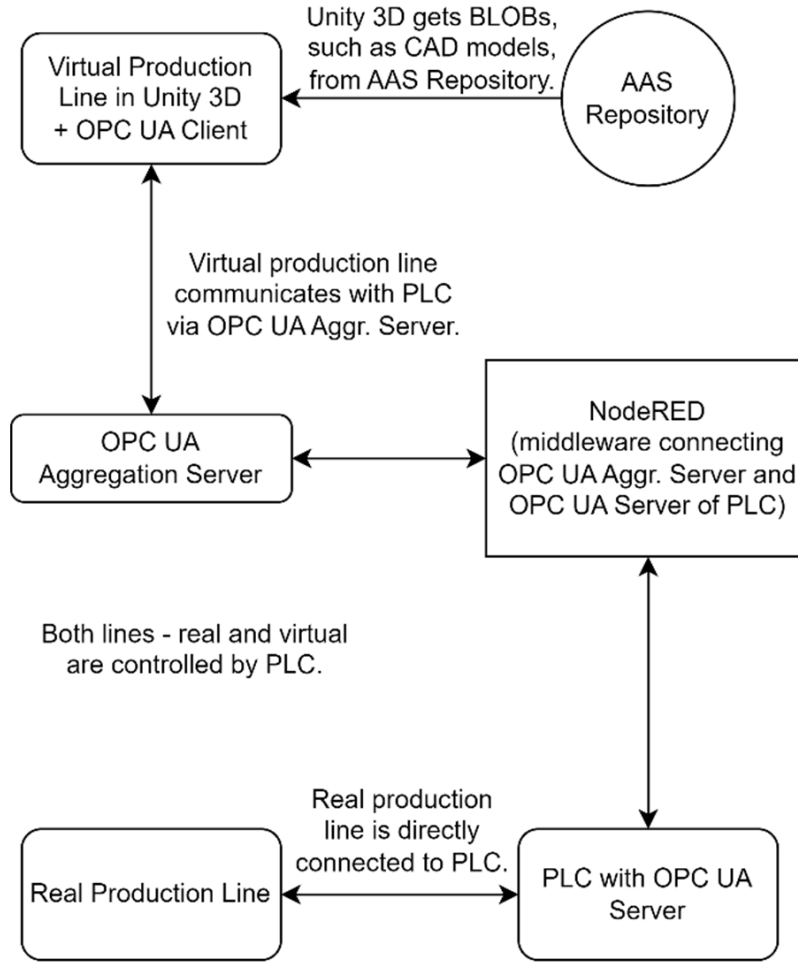


Figure 2.8: Architecture and data flow in an educational DT using Unity 3D, AAS, and OPC UA [6].

2.5 Automated Generation of DTs

Although DTs offer significant advantages for smart manufacturing [4, 20], the high engineering overhead required to build, configure, and maintain them remains a primary barrier to widespread industrial adoption [1, 5, 36]. In traditional development lifecycles, constructing a functional DT requires manual aggregation of multi-disciplinary assets, including 3D geometric meshes, kinematic joint definitions, electrical sensor mappings, and communication driver configurations [8, 10]. For SMEs and agile production lines with frequent product revisions and layout changes, this manual modeling process creates a major operational bottleneck [1, 23].

To transition DTs from static, offline prototypes into dynamic, self-configuring virtual assets, recent research emphasizes automated DT generation. The goal of automated generation is to eliminate manual engineering steps by programmatically transforming raw design artifacts, semantic standards, or operational logs into fully functional, synchronized digital twins [12, 16, 37]. Achieving this vision requires overcoming toolchain fragmentation, establishing unified metamodels, and automating both static scene synthesis and

dynamic execution loops.

2.5.1 Bottlenecks in Manual Workflows

Traditional DT development is mostly manual. As shown in Figure 2.9, engineering data is split across different tools [8]: mechanical engineers work in CAD, electrical engineers design schematics in Electronic Computer-Aided Design (ECAD), software developers write PLC code in vendor tools, and asset data is stored in PLM or Manufacturing Execution System (MES) databases.

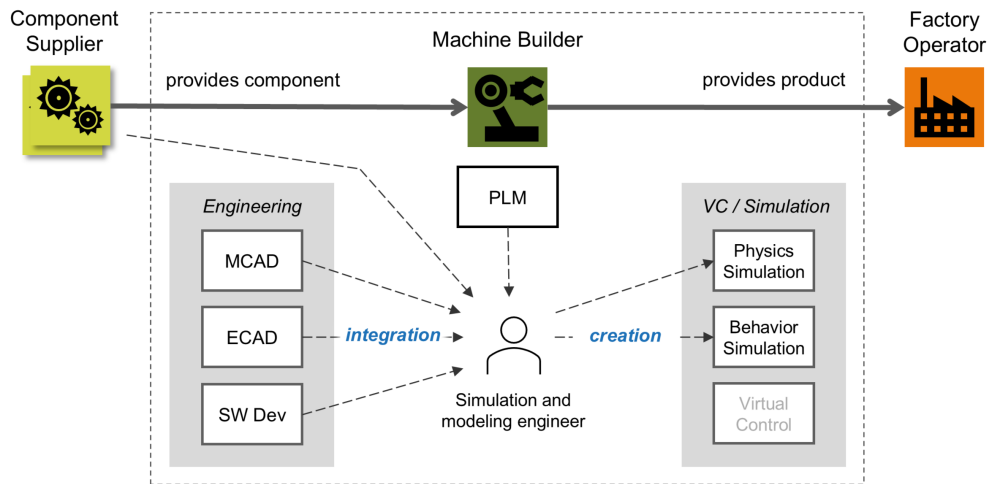


Figure 2.9: Data fragmentation across engineering tools in machine development [8].

To build a simulation model, an engineer has to collect all these files, import 3D meshes into the engine, set up parent-child relationships, configure joint limits, and manually connect virtual sensors to PLC addresses [10, 38].

This manual approach has three main drawbacks:

1. **High Time and Cost:** Building a model for a single workstation can take several weeks of engineering work [37].
2. **Risk of Configuration Errors:** Connecting hundreds of signal addresses by hand easily leads to mistakes that can cause simulation bugs.
3. **Maintenance Delays:** In reconfigurable factories, machines are frequently moved or swapped. If the simulation model is not updated immediately, it quickly becomes outdated and unreliable [16, 39].

To address these issues, recent research focuses on automating DT creation. Existing methods generally follow model-driven, data-driven, or AI-based approaches.

2.5.2 Model-Driven Approaches

Model-driven approaches use existing engineering files as the source to build the DT components, 3D scenes, and signal connections automatically [12, 16].

Examples include:

- **AutomationML-Based Methods:** Alexopoulos et al. [10] parse AML files containing plant topology and device types to instantiate 3D machines and set up communication drivers in Unity, as shown in Figure 2.10. Chowdhury, Stockman, and Kooning [9] extract part relationships and mass properties from CAD files to assemble motion models in Unreal Engine using FMI.
- **AAS-Based Configuration:** Richard et al. [40] use AAS descriptions to configure Autonomous Mobile Robots (AMRs) automatically. Ahvanouee et al. [16] use AAS submodels to exchange data between different engineering tools across supply chain partners.
- **SysML Models:** Heermann et al. [41] use textual Systems Modeling Language (SysML) v2 models to generate DT software modules and MQTT topic structures.
- **ECAD and Multi-Physics Synthesis:** Schaper et al. [8] parse electrical and fluid schematics together with AAS component data to connect FMU simulation blocks automatically. Koch et al. [38] similarly generate simulation models using structured component descriptions.
- **Automatic AAS Assembly:** Gollner et al. [42] present a method that queries equipment databases to generate populated AAS files automatically. Vaz et al. [29] generate Eclipse BaSyx DataBridge configs directly from interface submodels.

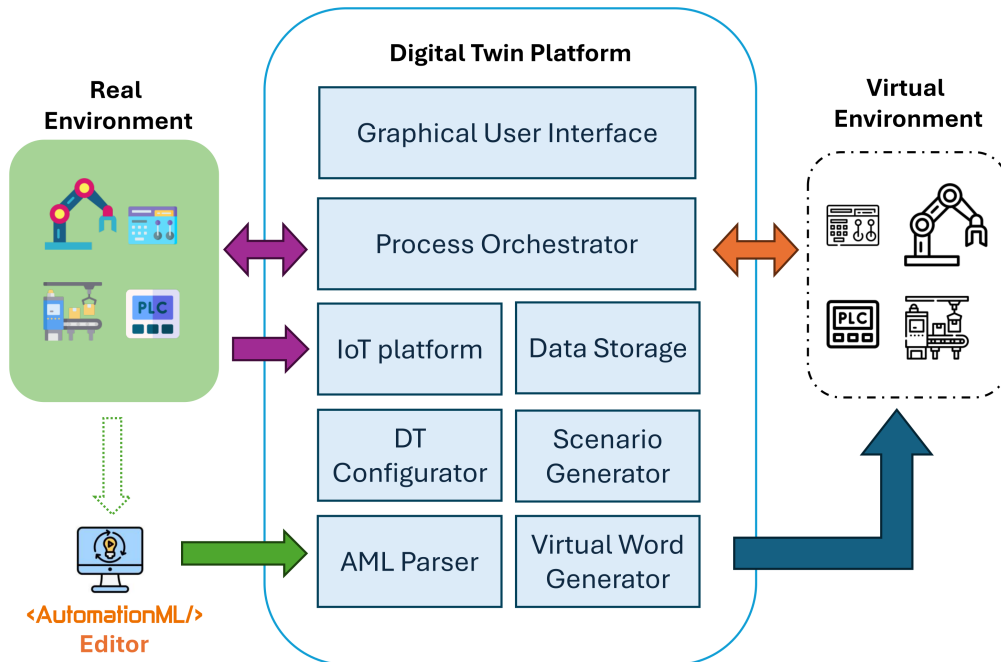


Figure 2.10: Architecture of the model-based platform for automated DT generation [10].

Model-driven methods provide reliable, deterministic results because they follow validated engineering designs. However, they require clean, up-to-date source files.

2.5.3 Data-Driven Approaches

In older factories where machines lack up-to-date CAD or design files, data-driven methods discover the DT structure from historical and live sensor data [37, 43]:

- **Process Mining from Event Logs:** Lugaresi and Matta [43] apply process mining to timestamped machine logs to determine routing, part flows, and buffer queues, building discrete event simulation models without design drawings.
- **Machine Learning Surrogates:** Garcia et al. [37] train machine learning models on sensor readings to estimate energy use, creating behavior models directly from measurement data.

Data-driven methods work well when engineering files are missing, but they produce approximations rather than exact 3D geometric models and depend heavily on having clean sensor data.

2.5.4 Generative AI and Large Language Models

Recent research explores using Large Language Models (LLMs) and generative AI to automate DT creation, code generation, and semantic asset modeling [10, 28, 44, 45].

As illustrated in Figure 2.11, Dong and Ren [44] present a framework using LLMs to translate natural language prompts into Python script APIs that procedurally assemble 3D scenes within NVIDIA Omniverse. Alexopoulos et al. [10] leverage generative AI to produce simulation test scenarios represented as Business Process Model and Notation (BPMN) process flows.

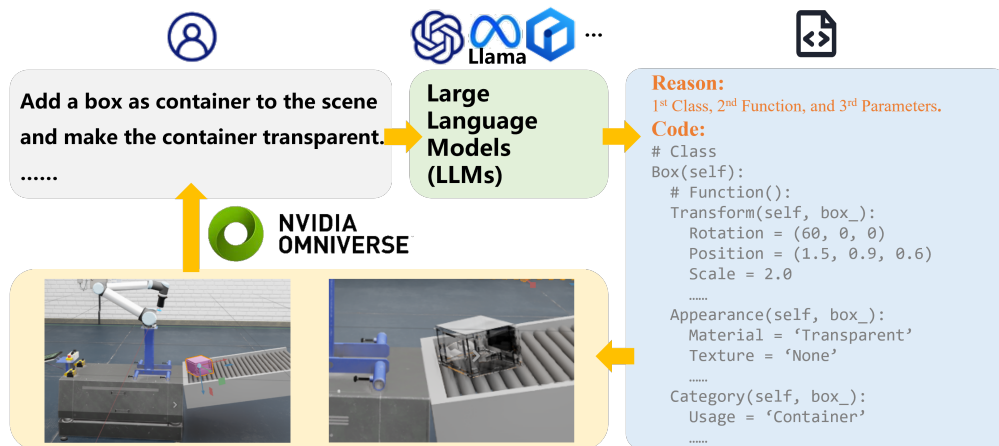


Figure 2.11: DT code generation process for NVIDIA Omniverse using LLMs [44].

Beyond 3D scene drafting and scenario generation, recent contributions focus on bridging unstructured text, machine learning, and standardized semantic models. As shown in Figure 2.12, Xia et al. [45] introduce an autonomous multi-agent LLM framework that ingests unstructured technical documentation (such as component Portable Document Format (PDF) datasheets) and extracts metadata to generate standardized AAS submodels

automatically, using Retrieval-Augmented Generation (RAG) to enforce schema compliance. Concurrently, Reichardt et al. [28] conceptualize the "AI Asset Administration Shell", embedding machine learning models and AI pipeline metadata directly into standardized AAS submodel templates to enable AI-driven operational monitoring within CPPSs.

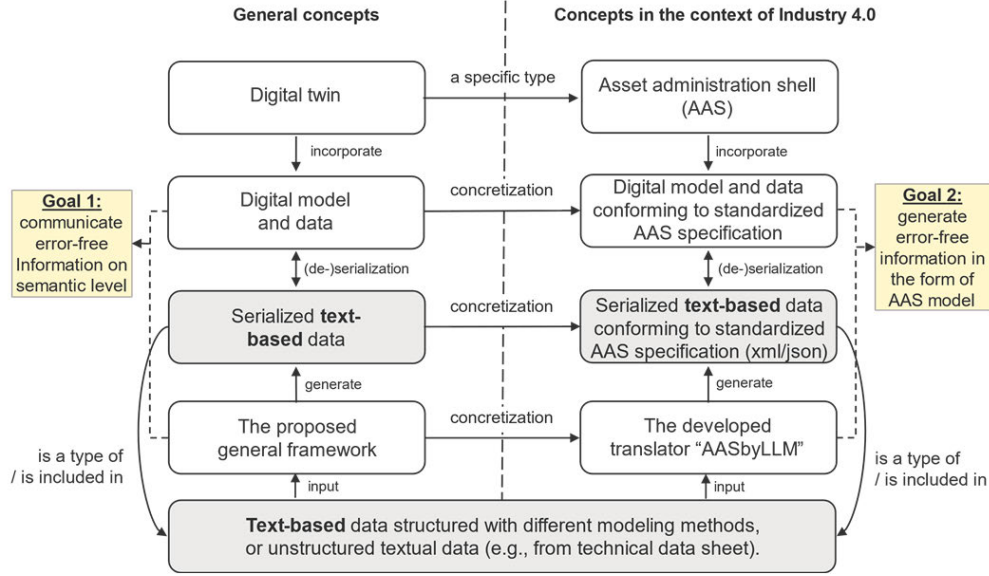


Figure 2.12: Framework for automated AAS submodel generation from technical documentation using LLM agents [45].

Despite their high flexibility and rapid prototyping capabilities, LLMs inherently present stochastic behavior and hallucination risks [44, 45]. In industrial automation, deterministic behavior and strict schema validation are essential for machine safety, meaning generative AI tools currently serve best as conversational query interfaces or initial drafting assistants rather than deterministic generators for physical control logic [10, 44, 45].

2.6 Identified Research Gaps and Opportunities

Comparing the state-of-the-art approaches across model-driven, data-driven, and generative AI paradigms highlights clear trade-offs. Model-driven methods [8, 10] provide reliable structures, but usually result in pre-built scenes that cannot adapt dynamically at runtime. Data-driven methods [37, 43] are useful for legacy equipment, but lack detailed 3D geometry and kinematics. Generative AI methods [28, 44, 45] make initial scene drafting and datasheet parsing fast, but lack the deterministic consistency needed for machine control systems. Most importantly, nearly all existing solutions still require an engineer to manually assemble 3D assets in the simulation editor before compilation, making rapid reconfiguration on the factory floor difficult.

Reviewing this literature shows that while individual pieces of the DT have progressed, integrating them into an automated procedural system remains an open challenge. Four

main gaps stand out:

1. **Gap 1: Fragmented Aspects vs. Unified Semantic Metamodel.** While 3D extraction tools [9, 44] handle meshes without electrical I/O or control logic, semantic tools [29, 42] manage properties without integrated 3D kinematics. An open submodel schema is needed that combines 3D meshes, kinematic limits, virtual I/O pins, and control logic rules in a single portable package.
2. **Gap 2: Proprietary Formats vs. Standardized AAS Source of Truth.** Several automated systems generate custom scripts or prefabs tied to a specific simulation tool [8, 10, 38]. Keeping the standardized AAS as the runtime source of truth ensures the digital asset remains interoperable across different factory systems.
3. **Gap 3: Pre-Compiled Scenes vs. Runtime Procedural Instantiation.** In most game engine setups [10, 31, 33], scenes must be manually compiled ahead of time, requiring software rebuilds whenever factory equipment changes. A runtime loader that ingests self-contained AASX packages directly into memory eliminates this dependency.
4. **Gap 4: Offline Simulation vs. Live Closed-Loop Synchronization.** Virtual commissioning tools [8, 11] support offline logic testing but lack online hardware synchronization, whereas Internet of Things (IoT) dashboards stream telemetry without physics simulation. A unified framework should support both modes using the same asset description.

To synthesize these gaps, Table 2.1 compares the proposed framework against leading approaches in the literature across key architectural capabilities.

Table 2.1: Comparative analysis of state-of-the-art automated DT frameworks.

Approach	Open Standard (AAS / AML)	Unified Schema (3D + I/O + Logic)	Procedural Runtime	Physics / Colliders	Live Hardware Synchronization
Alexopoulos et al. [10]	AML	Partial (CAD + AML)	No (Pre-compiled)	Yes	No (Simulation only)
Schaper et al. [8]	AAS / AML	No (FMU focused)	No (Offline VC)	Yes	No (Offline SiL)
Chowdhury, Stockman, and Kooning [9]	Proprietary	No (Kinematics only)	Partial (Editor)	Yes	No (Offline)
Vaz et al. [29]	AAS	No (Signals only)	No	No	Yes (DataBridge)
Pajpach et al. [6]	AAS / OPC UA	No (Separate models)	No (Pre-compiled)	Yes	Yes (OPC UA polling)
This Dissertation	AAS (IEC 63278)	Yes (AASX package)	Yes (100% in-memory)	Yes	Yes (Dual-mode)

These gaps motivate the design and research goals of this dissertation. Chapter 3 presents the conceptual architecture designed to address these challenges.

CONCEPTUAL SYSTEM ARCHITECTURE

This chapter presents the conceptual architecture, semantic data model, and interaction workflows of the proposed DT framework. It defines the functional layers, interaction boundaries, and communication mechanisms connecting the physical shop floor to the virtual simulation. By decoupling simulation mechanics from operational asset data, the architecture allows industrial systems to evolve modularly while maintaining digital-physical alignment.

The remainder of this chapter is organized as follows:

- [Section 3.1](#) defines the functional and non-functional requirements that guide the architectural design, along with their normative taxonomy and literature rationale.
- [Section 3.2](#) introduces the conceptual architecture, outlining the core design principles, the upstream digitization stage (Layer 0), the operational activity workflows (Layers 1 to 3), and cross-layer interaction flows.
- [Section 3.3](#) details the semantic metamodel, decomposing industrial assets into standardized submodels for 3D geometry, kinematics, I/O pin mappings, control logic, and network endpoints.
- [Section 3.4](#) summarizes the conceptual architecture and provides the transition to the technological implementation.

3.1 Design Requirements and Constraints

The design of a modular and automated DT framework must address several key limitations identified in the State of the Art ([Chapter 2](#)). Current industrial DTs are often held back by tight coupling between 3D assets and proprietary simulation tools, slow manual synchronization pipelines, and a lack of vendor-neutral semantic interoperability [8, 10]. At the same time, the transition toward RMSs and CPPSs requires plug-and-produce capabilities, allowing machines and stations to be added, swapped, or rearranged without costly downtime or manual re-engineering [3].

Following the specification methodology of Alemão et al. [46], requirements are organized into two main categories:

- **Functional Requirements (FR):** Define the concrete actions, functions, and behaviors the system must perform, as well as its interactions with users and external services. They represent what the software architecture must execute.
- **Non-Functional Requirements (NFR):** Define the quality attributes, constraints, and operating boundaries of the system. Rather than describing specific features, they specify how well the system must perform in areas such as latency, scalability, fault isolation, and usability.

To clearly define the necessity of each requirement, this work adopts the modal taxonomy used by Alemão et al. [46]:

- **“Shall”:** Mandatory and strictly required for system compliance.
- **“Should”:** Recommended feature or architectural intention that is desirable, but whose absence does not compromise basic compliance.
- **“May”:** Optional capability that provides additional flexibility without affecting core execution.

Based on these principles and the gaps identified in the literature, the functional and non-functional requirements are defined below:

3.1.1 Functional Requirements

- FR1 Plug-and-Play Asset Onboarding:** The framework shall support the registration and deployment of new industrial assets into the active ecosystem without requiring recompilation or manual changes to simulation software. Each asset shall be independently described, validated, and onboarded without disrupting running operations [3, 18].
- FR2 Self-Describing Asset Packages:** All information needed to configure and execute a DT shall be bundled into a single, standardized, and portable package per asset. This archive shall encapsulate 3D visual geometry, kinematic degrees of freedom, virtual sensor/actuator pin mappings, network endpoints, and behavioral control rules [7, 42].
- FR3 Dynamic Scene Assembly:** The simulation engine shall construct the virtual environment procedurally at startup by querying a central semantic registry, removing the need for pre-built or hard-coded scene files. The virtual environment shall adapt its spatial layout dynamically to reflect all registered assets [10].

FR4 Physical-Virtual Synchronization and Drift Mitigation: During runtime, the virtual model shall maintain continuous, bidirectional state synchronization with physical equipment. The synchronization mechanism shall detect discrepancies between periodic telemetry frames and automatically apply drift correction to preserve physical-digital consistency [4, 5].

3.1.2 Non-Functional Requirements

NFR1 Low-Latency Telemetry: Sensor state changes and actuation events from physical hardware should reach the simulation engine with minimal latency to ensure responsive and timely state replication.

NFR2 Scalability and Modularity: The central registry and storage services should support concurrent asset registrations and multiple simulation clients without performance degradation or architectural changes.

NFR3 Layer Independence and Fault Isolation: Architectural layers shall remain functionally decoupled through well-defined API contracts. A disruption, restart, or failure in the modeling or repository layer shall not stop active real-time simulation or shop-floor execution.

NFR4 Configuration Usability and Accessibility: Creating and packaging a DT asset descriptor should not require low-level programming skills. The framework should provide an intuitive, guided interface suitable for automation and shop-floor engineers.

These requirements define the criteria that guide the three-layer conceptual architecture presented next.

3.2 Architectural Overview

Current industrial DT implementations face several practical bottlenecks that limit their flexibility and scalability on the shop floor:

1. **Monolithic Coupling and Engineering Overhead:** Most commercial simulation platforms bundle 3D geometry, kinematic definitions, control scripts, and communication drivers into a single proprietary project file [11, 38]. When a shop-floor station is modified, such as adjusting the stroke of a pneumatic cylinder, changing a clamp, or moving a proximity sensor, the virtual model cannot adapt on its own. An engineer must reopen the simulation tool, manually edit the mechanics and signal links, and redeploy the scene. This manual maintenance creates significant downtime and makes rapid reconfiguration difficult.

2. **Fragmented Engineering Data:** Machine specifications are usually split across different tools and file types. Mechanics are stored in CAD files, electrical connections in circuit diagrams or spreadsheets, and control logic in PLC programs [12, 16]. For instance, adding a new conveyor module requires manually matching PLC I/O addresses with simulation tags and positioning the 3D meshes by hand. Without a standardized, self-contained asset format, these disconnected sources lead to configuration mistakes and prolonged setup times [8, 42].
3. **Network Latency and Channel Congestion:** Several experimental architectures send both heavy visual assets and real-time operational data across the same communication channel or API endpoint [6, 26]. Downloading large 3D models while simultaneously streaming high-frequency sensor updates creates network delays and jitter. In fast assembly lines, delayed sensor readings cause the simulation to lag behind the physical equipment, creating positional drift and breaking synchronization [5].
4. **Lack of Simulation-Ready Models for Existing Machinery:** Many factories rely on brownfield equipment that either lacks 3D DMs entirely or only has detailed manufacturing CAD drawings [11, 34]. These manufacturing files often contain thousands of internal details, such as screw threads or wiring harnesses, while lacking kinematic joints and local pivot axes. Loading such models directly into a real-time physics engine degrades rendering performance and prevents moving components from functioning correctly.

To address these challenges, the framework separates the asset lifecycle into three conceptual layers, supported by an initial preparation stage (Layer 0), as shown in [Figure 3.1](#):

- **Layer 1 (Asset Modeling):** Prepares and semantically annotates modular asset packages.
- **Layer 2 (Storage and Registry):** Stores asset files and manages a central directory of network endpoints.
- **Layer 3 (Runtime Execution and Synchronization):** Procedurally builds the virtual scene, runs the physics engine, and synchronizes state with the physical machines.

The following subsections describe the role of each layer, the specific problems it solves, and how its components operate.

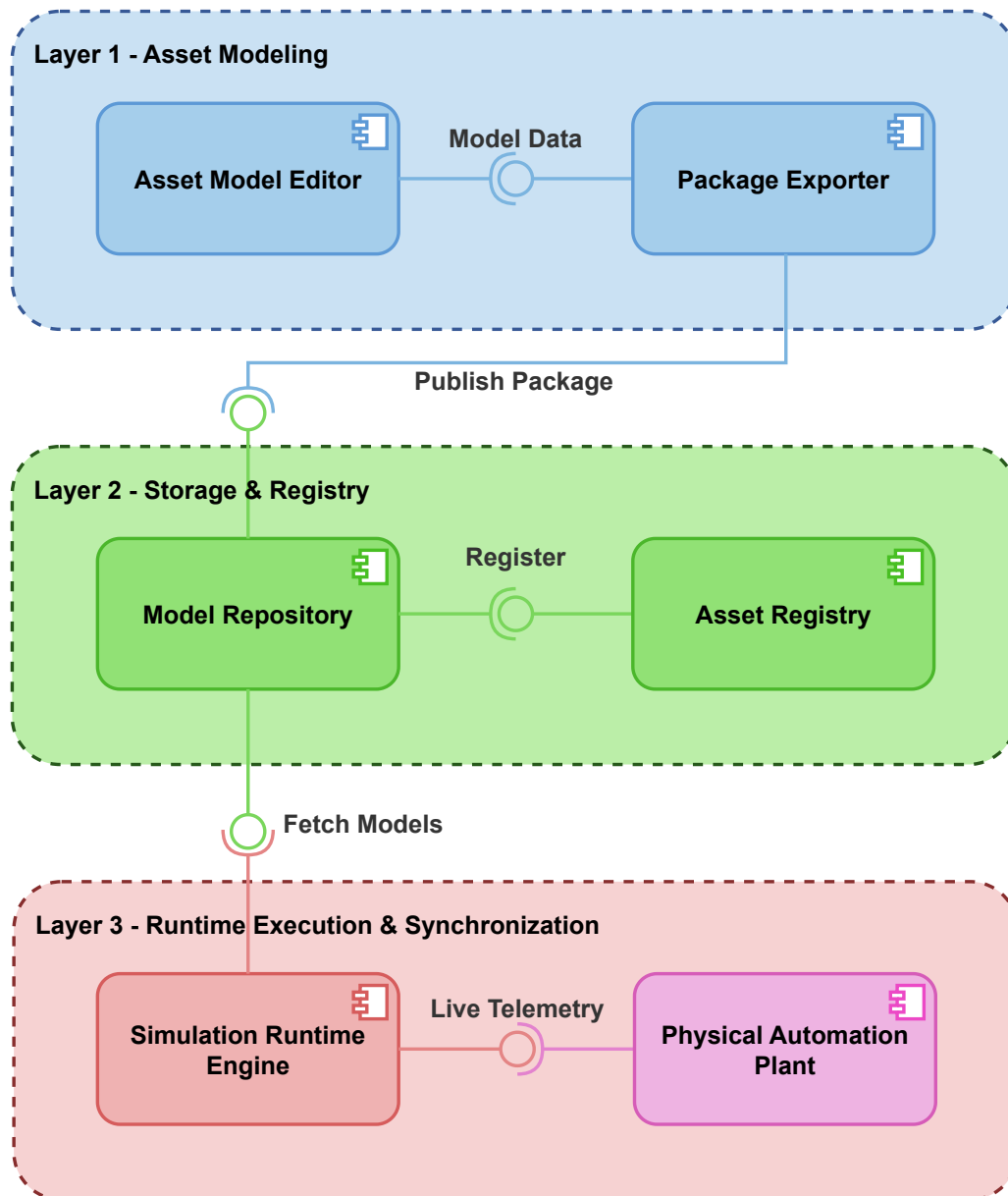


Figure 3.1: Three-layer conceptual system architecture of the proposed DT framework.

3.2.1 Upstream Stage: Layer 0 - Physical Asset Digitization

Before an industrial machine can be mirrored in software, the framework requires a clean geometric foundation. In an ideal greenfield project, machine builders supply lightweight, simulation-ready 3D models with pre-configured joints. In real factory environments, however, engineers typically face one of two negative extremes: either no digital 3D model exists at all, or the available CAD files come directly from mechanical manufacturing.

Manufacturing CAD assemblies pose a major obstacle for real-time simulation. For example, a raw CAD assembly of a pneumatic stamping station often contains internal screw threads, bearing balls, wiring, and dense solid geometry amounting to hundreds of thousands of polygons. Attempting to drop this raw file into a real-time graphics or physics engine causes severe frame drops. More critically, CAD exports typically flatten all parts into a static hierarchy, stripping away the pivot points, motion axes, and stroke limits needed to simulate motion. Without properly configured local coordinate frames, a physics engine cannot determine whether a stamping rod is supposed to extend linearly or rotate around an arbitrary origin.

Layer 0 fills this gap by acting as the geometric preparation stage ([Figure 3.2](#)). Its objective is to produce a low-overhead, rigged 3D asset that accurately matches the physical dimensions of the equipment while discarding unnecessary internal complexity.

The workflow relies on two main components:

1. **3D Mesh Modeler:** Processes the raw mechanical CAD model to produce a simulation-ready asset. Unnecessary internal details are removed, exterior surfaces are decimated to an efficient polygon budget, and basic collision boundaries are defined. Movable elements, such as pneumatic pushers, diversion gates, or rotating tables, are separated into independent sub-meshes, with their local coordinate frames and pivot points aligned directly with their functional axes of motion. Volumetric trigger zones (such as invisible box or cylindrical trigger colliders) are positioned at physical sensor locations, allowing the simulation engine to detect product passage or presence when parts enter these sensing volumes.
2. **Model Exporter:** Exports the structured visual hierarchy, kinematics rigging, trigger zones, and materials into an open, vendor-neutral 3D binary format. This produces a lightweight, self-contained 3D geometry file ready for semantic authoring in Layer 1.

Once the geometric boundaries, pivot points, and sensor trigger volumes are established in Layer 0, the resulting 3D asset provides the spatial foundation needed for operational configuration. However, this geometry contains no automation data. It must now enter Layer 1, where it is enriched with kinematics, electrical signals, and control logic.

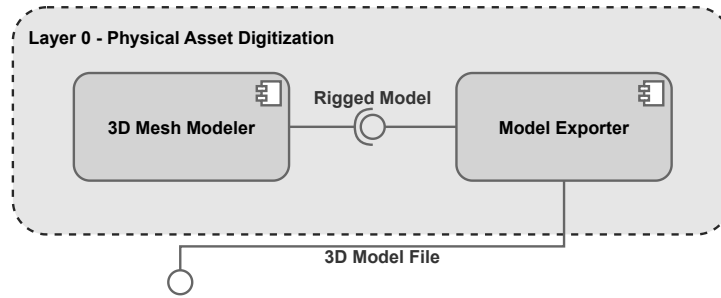


Figure 3.2: Unified Modeling Language (UML) Component Diagram of Layer 0 (Physical Asset Digitization).

3.2.2 Layer 1 - Asset Modeling

Layer 1 provides the authoring tools needed to transform static geometry into an operational DT asset. A 3D model on its own is only a visual representation without any understanding of sensors, actuators, or control behavior. In conventional workflows, configuring these dynamics requires importing the model into a proprietary simulation suite and writing custom scripts inside that specific software.

This approach creates significant lock-in. When a factory changes its simulation software or attempts to connect multiple tools at once, all previous modeling and scripting work is lost. In addition, if a limit switch on the physical machine is rewired to another PLC input terminal, an engineer has to search for the corresponding script in the simulation project and update it by hand. Operational data ends up scattered between electrical drawings, control logic, and simulation files.

Layer 1 resolves this fragmentation by keeping semantic modeling independent of any runtime engine. Its purpose is to combine visual geometry, kinematic limits, electrical pin mappings, and control rules into a standardized and portable asset package.

The procedural workflow is illustrated in the UML Activity Diagram in [Figure 3.3](#), where the *Asset Model Editor* and the *Package Exporter* coordinate the process:

1. **CAD Import and Metadata Configuration:** The engineer imports the clean 3D file produced in Layer 0 into the *Asset Model Editor* and assigns administrative metadata, including a unique global identifier, asset classification, and descriptive tags.
2. **Kinematic Parsing and Joint Limits:** The *Package Exporter* inspects the 3D transform hierarchy, identifies movable nodes, and allows the engineer to define their kinematic properties, including motion types, motion axes, and mechanical travel limits.
3. **Virtual Pin Binding and Control Logic:** In the editor, the user defines the electrical and logical interface by creating virtual pins corresponding to physical controller I/O points. Sensor pins are anchored to detection zones to publish state changes upon product detection, while actuator pins drive the kinematic mechanisms when

control commands are received. Local interlocking logic and communication topic bindings are assigned directly to these pins.

4. **Validation and Package Compilation:** Before saving, the *Package Exporter* validates the complete configuration against formal metamodel schemas, verifying that all declared pins and kinematic constraints are resolved. Once verified, it compiles the 3D geometry and descriptor files into a single, immutable asset package and publishes it to Layer 2.

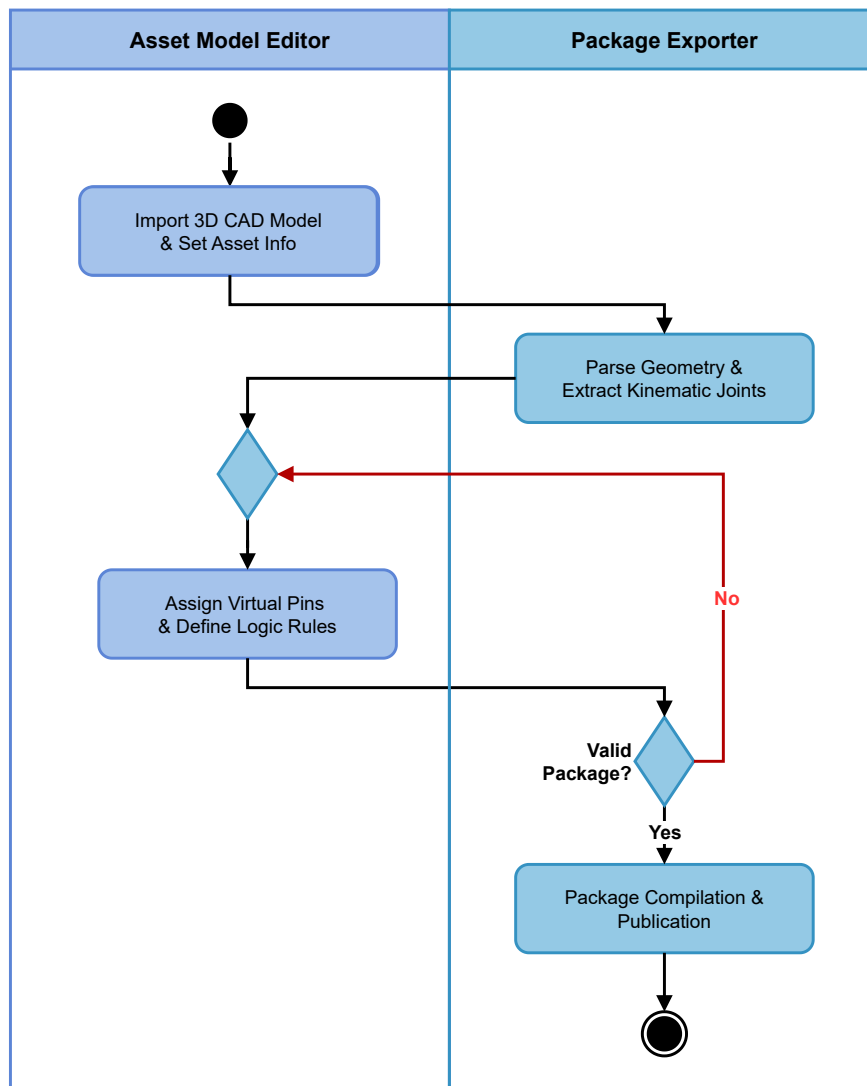


Figure 3.3: UML Activity Diagram of Layer 1 (Asset Modeling).

Compiling an asset package in Layer 1 completes the authoring pipeline for an individual machine. To make this self-contained package discoverable and accessible

across the factory network, it is transferred to Layer 2, which hosts the repository and service directory.

3.2.3 Layer 2 - Storage and Registry

In a flexible or reconfigurable factory, machines and transport units are frequently rearranged, upgraded, or swapped out. In conventional simulation architectures, loading a modified layout requires an operator to manually copy new 3D model files onto every simulation computer, update local file paths, and re-enter static Internet Protocol (IP) addresses to connect to each machine's controller.

This static approach breaks down quickly in agile production. For example, if an Automated Guided Vehicle (AGV) enters a production cell, or a faulty drilling station is replaced by a backup unit with different network settings, any simulation system relying on hard-coded addresses or local files stops working until someone manually reconfigures the environment. Mixing large file downloads with operational network queries often causes slow discovery responses and unnecessary network load.

Layer 2 acts as the central service directory and artifact distribution point for the entire DT ecosystem. Its objective is to provide on-demand discovery and retrieval, allowing any authorized client on the factory network to locate active equipment, download corresponding models, and obtain the correct network topics automatically.

To keep discovery fast and scalable, Layer 2 separates heavy static files from dynamic directory metadata (illustrated in the UML Activity Diagram in [Figure 3.4](#)) using two distinct services:

1. **Model Repository:** Acts as a dedicated file store. It receives published asset packages from Layer 1, extracts the heavy 3D geometry meshes and descriptor schemas, and hosts them through standard HTTP endpoints.
2. **Asset Registry:** Functions as a lightweight service catalog. It records high-level metadata: the unique asset identifier, device classification, operational status, and active communication endpoints.
3. **Dynamic Discovery and Provisioning:** When a simulation client starts up it queries the lightweight *Asset Registry* via a REST API to find out which assets are available. The registry responds with the metadata and download URLs. Using these links, the client fetches the 3D geometry and submodels from the *Model Repository*, loading them into memory without needing pre-installed local files.

With the asset packages cataloged and network endpoints registered in Layer 2, the DT models are ready to be consumed. Layer 3 interacts directly with Layer 2 to discover these assets and instantiate the runtime simulation.

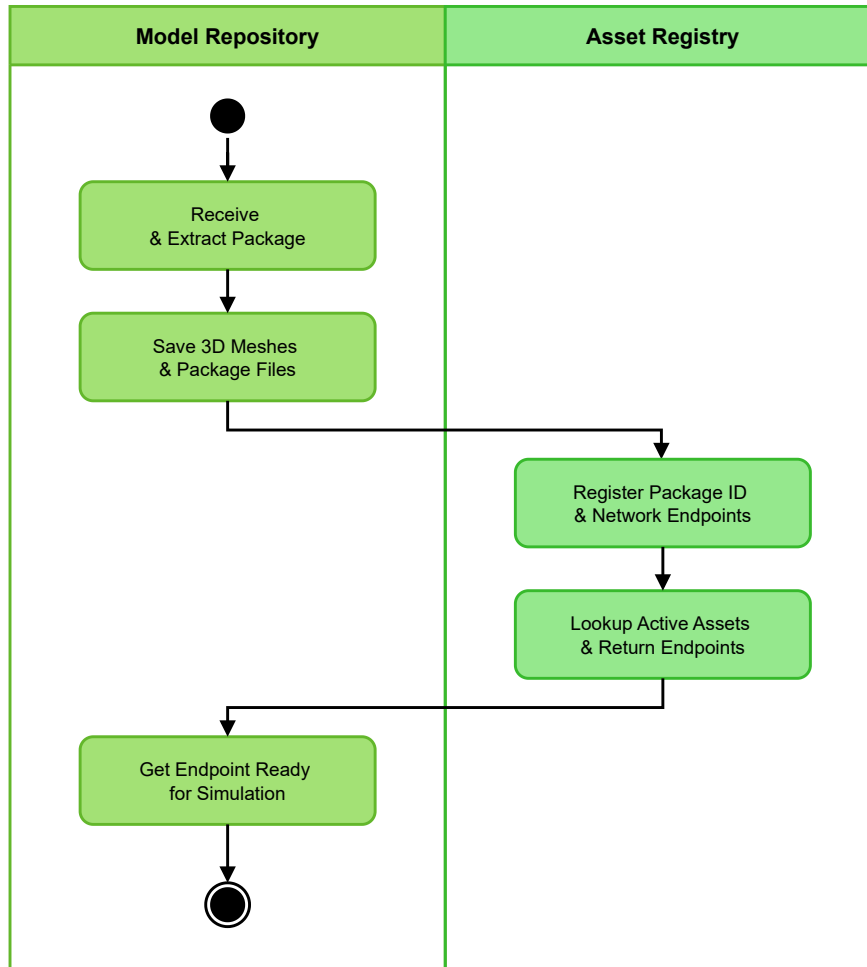


Figure 3.4: UML Activity Diagram of Layer 2 (Storage and Registry).

3.2.4 Layer 3 - Runtime Execution and Physical Synchronization

The ultimate value of a DT lies in running a faithful, real-time virtual counterpart that mirrors physical production without human intervention. In standard industrial setups, assembling a virtual cell requires opening a simulation editor, dragging and dropping machine models into place, manually positioning them, and setting up real-time drivers.

Even after a virtual cell is configured, maintaining accurate physical-digital synchronization presents a major technical hurdle. Physics engines compute movements through discrete time steps using numerical approximations. If a virtual belt conveyor runs solely on simulated friction, slight timing variances and rounding errors accumulate over continuous operation. A workpiece on the virtual conveyor will slowly drift away from its true physical position, causing virtual sensors to trigger at incorrect times and resulting in false alarms or desynchronized simulation states. Conversely, attempting to stream high-detail geometric meshes across the same communication channel that carries cyclic sensor telemetry creates queueing bottlenecks, delaying critical sensor packets.

Layer 3 solves these problems by combining automated procedural scene generation with real-time state synchronization and drift correction. Its objective is to build the virtual environment dynamically at startup and maintain tight digital-physical alignment throughout production.

As shown in the UML Activity Diagram (Figure 3.5), Layer 3 coordinates the runtime engine with the physical automation plant through parallel execution loops:

1. **Procedural Scene Assembly:** When launched, the simulation runtime queries the Layer 2 registry, downloads the necessary asset packages, and reconstructs the virtual factory floor directly in memory. It automatically creates 3D visual nodes, binds physics colliders, assigns kinematic joint limits, and maps virtual pins based on the downloaded descriptors, requiring no manual scene editing.
2. **Continuous Physics Simulation:** The simulation engine runs an internal physics loop at a steady frequency, calculating rigid-body collisions, gravity, and joint movements so that material handling and mechanical operations behave realistically.
3. **Isolated Low-Latency Telemetry:** In parallel, the physical machines on the shop floor run their cyclic PLC scan routines. When physical sensors or actuator states change, the controller sends compact telemetry packets over a dedicated, low-overhead messaging protocol straight to the simulation runtime, bypassing storage databases entirely to minimize delay.
4. **State Updates and Drift Correction Snapping:** When the simulation receives an incoming telemetry packet, it updates the virtual sensor indicators and commands the virtual actuators. Because friction and contact physics accumulate small numerical errors over time, a moving workpiece will naturally drift away from its real position. To eliminate this divergence, each physical sensor detection acts as an absolute ground-truth reference point: the moment the hardware sensor trips, the simulation engine instantly snaps the virtual workpiece to the exact physical coordinates of that sensor before the next physics tick. This resets accumulated drift to zero and guarantees continuous physical-digital alignment even during extended production runs.

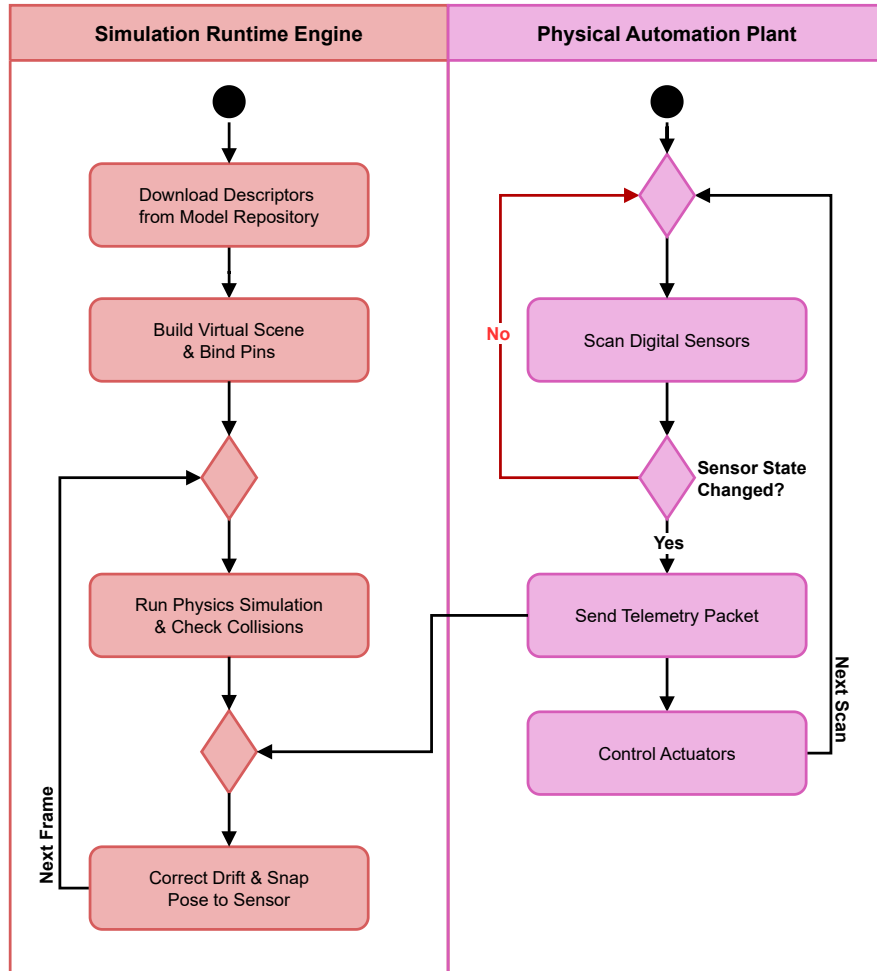


Figure 3.5: UML Activity Diagram of Layer 3 (Runtime Execution and Physical Synchronization).

3.2.5 Cross-Layer Information and Interaction Flows

While each architectural layer operates as an independent subsystem with its own lifecycle, the strength of the framework lies in how data moves across layer boundaries. In traditional monolithic simulation tools, engineering models, network configurations, and real-time signals are tightly bundled together, creating dependencies where changes in one domain require manual updates across the entire system.

To overcome this coupling, the proposed architecture coordinates three distinct communication flows, as shown in the conceptual overview in [Figure 3.1 \(page 29\)](#). These flows are separated by lifecycle phase, data volume, and timing constraints:

1. **Asset Publication Flow (Design-Time: Layer 1 → Layer 2):** This flow takes place whenever an automation engineer authors or updates an asset. Because this exchange happens during engineering rather than production, latency is not critical. However,

payload volume is relatively high, as it contains complete 3D meshes, collision data, and semantic submodel descriptors. The modeling tool packages these assets into an immutable archive, uploads the binary files to the *Model Repository*, and registers the machine metadata and network endpoints in the *Asset Registry*. Once published, the asset becomes immediately discoverable across the factory network.

2. **Dynamic Ingestion Flow (Deployment-Time: Layer 2 → Layer 3):** This flow occurs on demand when a simulation instance starts up, or when a reconfigurable workstation enters the manufacturing cell. The runtime engine first queries the *Asset Registry* using lightweight API calls to discover which physical assets are currently present and operational. Guided by the registry responses, the engine requests the corresponding 3D geometry files and submodels from the *Model Repository*. By constructing the virtual scene dynamically in memory, the system avoids pre-compiled scene files and ensures that any workstation newly added to Layer 2 is instantiated in simulation without restarting or re-engineering the application.
3. **Low-Latency Telemetry Flow (Runtime: Physical Plant → Layer 3):** Unlike the previous two flows, which handle heavy descriptive data on an infrequent basis, the telemetry flow operates continuously during live production. When hardware sensors detect part movements or PLCs execute actuation cycles, state updates must reach the simulation engine with minimal delay. To achieve this, telemetry completely bypasses the storage and registry services of Layer 2. Instead, physical controllers publish compact, event-driven sensor packets directly to the simulation engine over a dedicated, low-overhead messaging channel. This separation protects real-time synchronization from network jitter, allowing the physics engine to apply drift correction and update virtual states deterministically.

Segregating these three flows ensures that heavy model transfers never interfere with time-critical automation data. While this architectural division defines the functional boundaries and interaction patterns of the framework, executing these workflows requires a common, structured language to describe each machine. The following section defines the semantic data model that enables this cross-layer interoperability.

3.3 Asset Data Model Specification

To avoid relying on proprietary simulation tools, the framework defines a technology-neutral data model. The model standardizes how shop-floor machinery is described across all layers, separating visual 3D geometry, kinematics, I/O signals, local control logic, and network endpoints.

Figure 3.6 shows the complete UML Class Diagram of the conceptual asset metamodel. The root entity, *AssetDescriptor*, brings together five submodel classes: *VisualStructure*, *BehaviorMapping*, *VirtualPins*, *NetworkEndpoint*, and *ControlLogic*. All attributes use standard

primitive types (*String*, *Float*, *Integer*, *Boolean*, and *Vector3*, representing a 3D float coordinate or vector (x, y, z)).

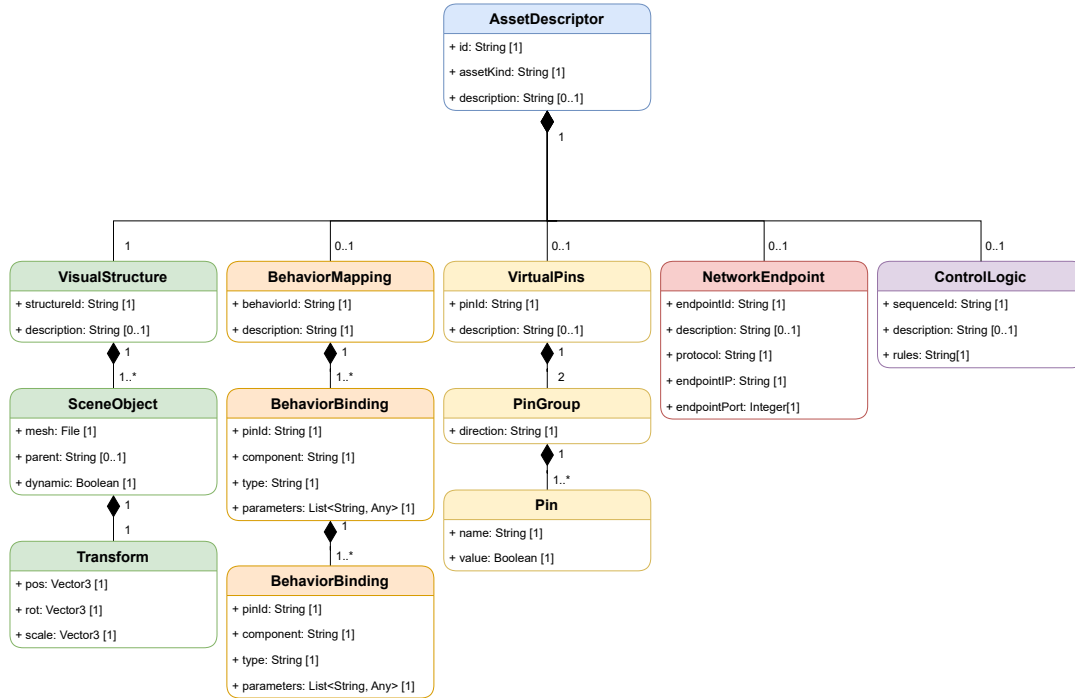


Figure 3.6: UML Class Diagram of the conceptual asset metamodel and constituent submodel classes.

Each subsection below describes one class in detail, covering its purpose, multiplicity, and attributes.

3.3.1 AssetDescriptor (Core Entity)

The *AssetDescriptor* is the main container and identity root for any machine or workstation. It holds identity and classification data, acting as the entry point for registries and simulation parsers, as depicted in the UML Class Diagram in [Figure 3.7](#).

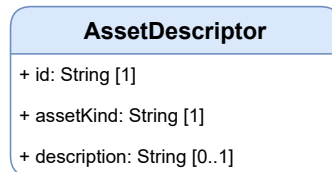


Figure 3.7: UML Class Diagram of the core *AssetDescriptor* entity.

The attributes of *AssetDescriptor* are:

- **id** (*String*, [1]): A unique identifier used for naming, User Interfaces (UIs), and registry lookups.

- *assetKind* (*String*, [1]): The functional type or category of the machine.
- *description* (*String*, [0..1]): An optional text description of the machine.

3.3.2 VisualStructure Class

The *VisualStructure* class defines the 3D visual and spatial layout of an asset, independent of any particular graphics format. An *AssetDescriptor* contains a single *VisualStructure* submodel (1). Within this submodel, the asset defines a collection of one or more *SceneObject* nodes (1..*), where each node carries exactly one *Transform* (1). The structural organization of this hierarchy is illustrated in the UML Class Diagram in [Figure 3.8](#).

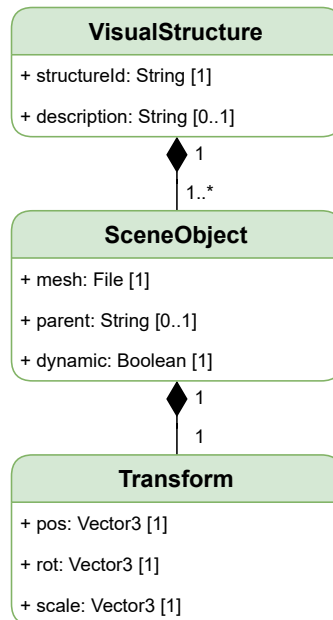


Figure 3.8: UML Class Diagram of the *VisualStructure* hierarchy (*VisualStructure*, *SceneObject*, and *Transform*).

The attributes in the *VisualStructure* hierarchy are:

VisualStructure:

- *structureId* (*String*, [1]): The identifier of this visual group within the asset.
- *description* (*String*, [0..1]): An optional text description of the assembly.

SceneObject:

- *mesh* (*File*, [1]): The 3D surface mesh file for this node.

- **parent** (*String*, [0..1]): The identifier of the parent node, defining transform inheritance for multi-part assemblies.
- **dynamic** (*Boolean*, [1]): A flag indicating whether this component moves during operation.

Transform:

- **pos** (*Vector3*, [1]): The 3D position offset (x, y, z) in meters relative to the parent frame.
- **rot** (*Vector3*, [1]): Euler angles (α, β, γ) in degrees (pitch, yaw, roll).
- **scale** (*Vector3*, [1]): Scale factors (s_x, s_y, s_z), set to (1.0, 1.0, 1.0) by default.

3.3.3 BehaviorMapping Class

While 3D meshes define visual geometry, they do not inherently convey how a physical component operates or responds to automation signals. The *BehaviorMapping* class bridges this gap by defining the operational behavior and kinematic properties of sensors and actuators (0..1). It encapsulates the domain configuration and physics parameters required by the simulation engine, grouping one or more *BehaviorBinding* entries (1..*).

The *BehaviorMapping* class does not store the raw boolean signal values. Instead, it specifies how each component behaves during operation:

- **For an Actuator:** It specifies the kinematic motion profile, vector of action, operating speed, stroke limits, and mechanical interaction with workpieces.
- **For a Sensor:** It defines the spatial detection boundary, target filtering criteria, electrical signal inversion logic, and synchronization snapping parameters.

Before inspecting the structure, the UML Class Diagram in [Figure 3.9](#) illustrates the class hierarchy for *BehaviorMapping* and *BehaviorBinding*.

The attributes across the *BehaviorMapping* hierarchy ([Figure 3.9](#)) are:

BehaviorMapping:

- **behaviorId** (*String*, [1]): Unique identifier for this behavioral profile within the asset descriptor.
- **description** (*String*, [1]): Functional description documenting the scope of the behavioral configuration.

BehaviorBinding:

- **pinId** (*String*, [1]): The identifier of the corresponding logical channel defined in *VirtualPins* that drives or observes this component.

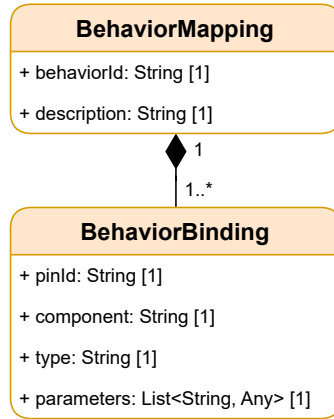


Figure 3.9: UML Class Diagram of the *BehaviorMapping* hierarchy (*BehaviorMapping* and *BehaviorBinding*).

- **component** (*String*, [1]): The targeted *SceneObject* node in *VisualStructure* whose physical behavior is modulated by this binding.
- **type** (*String*, [1]): The functional mechanism category governing execution logic in the physics runtime, such as continuous linear or rotational conveyors (*TranslateContinuous*, *RotateContinuous*), mechanical linkages and joints (*RotateHinge*, *LinearCylinder*), or spatial detection volumes (*TriggerZone*).
- **parameters** (*List<String, Any>*, [1]): Dynamic dictionary of operational configuration settings. Rather than enforcing rigid sub-classing for every physical mechanism, this attribute is modeled as an extensible key-value collection. This preserves backward compatibility while accommodating both actuators and sensors:

1. Actuator Parameters:

- **AxisX, AxisY, AxisZ** (*Double*, [1]): Normalized direction vector defining the linear line of action or rotational axis, expressed in the local coordinate frame of the target node.
- **Speed** (*Double*, [1]): Velocity magnitude driving the continuous linear surface displacement or angular articulation rate.
- **RealPortID** (*String*, [1]): Physical terminal designation or register address on the field controller, linking the abstract virtual channel to physical hardware.
- **RealActuatorEnabled** (*Boolean*, [1]): Operational routing flag determining whether actuation commands dispatch to physical hardware or execute exclusively within the virtual simulation.

- **Visible** (*Boolean*, [1]): Flag specifying whether actuation directional gizmos or bounding outlines are rendered in the viewport.

2. Sensor Parameters:

- **TargetFilter** (*String*, [1]): Classification tag defining which moving colliders are eligible to trip the sensor volume, preventing false positives from static structures or irrelevant fixtures. For instance, an inductive or magnetic proximity sensor can be configured to detect only workpieces tagged as metallic or magnetic, ignoring non-magnetic plastic carriers.
- **ActiveLow** (*Boolean*, [1]): Electrical polarity flag specifying whether the logical state represents normally open or normally closed physical sensor contact behavior.
- **RealPortID** (*String*, [1]): Physical terminal designation or register address on the field controller from which telemetry originates.
- **RealSensorEnabled** (*Boolean*, [1]): Operational routing flag indicating whether runtime execution receives live field telemetry or runs under local virtual simulation.
- **Visible** (*Boolean*, [1]): Flag controlling whether the volumetric sensing boundary is visually rendered as a debug wireframe during execution.
- **SensingRange** (*Double*, [0..1]): Geometric radius or volumetric dimension defining the extent of the detection zone.
- **MoveTarget** (*String*, [0..1]): Identifier of the target scene component, movable fixture, or detected object whose spatial coordinates are subject to realignment upon sensor state change.
- **MoveOnState** (*Boolean*, [0..1]): Signal transition edge (rising versus falling transition) that triggers kinematic pose snapping.
- **MoveMode** (*String*, [0..1]): Correction scope applied to the target component upon triggering, selecting whether the runtime aligns spatial coordinates (*Position Only*), angular orientation (*Rotation Only*), or both degrees of freedom simultaneously (*Position and Rotation*).
- **MovePosX**, **MovePosY**, **MovePosZ** (*Double*, [0..1]): Target local 3D coordinates (x, y, z) to which the component is instantly repositioned, establishing an exact positional reference at the triggering event.
- **MoveRotX**, **MoveRotY**, **MoveRotZ** (*Double*, [0..1]): Target Euler orientation angles (α, β, γ) applied to adjust component alignment, eliminating accumulated angular deviation.

3.3.4 VirtualPins Class

In contrast to *BehaviorMapping* (which defines behavioral characteristics and kinematic kinematics), the *VirtualPins* class models the pure electrical I/O interface of the asset (0..1).

It acts as the digital image of physical controller terminals or fieldbus communication tags, abstracting raw hardware addresses into standardized discrete channels.

Each *VirtualPins* instance contains two dedicated *PinGroup* collections categorized by data direction (*INPUT* and *OUTPUT*). Each group contains the individual *Pin* instances representing discrete boolean states (1..*).

To illustrate the relationship between terminals, signal direction, and individual channels, the UML Class Diagram in Figure 3.10 details the *VirtualPins* hierarchy.

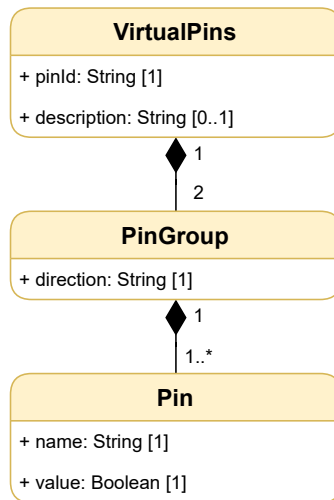


Figure 3.10: UML Class Diagram of the *VirtualPins* hierarchy (*VirtualPins*, *PinGroup*, and *Pin*).

The attributes across the *VirtualPins* hierarchy (Figure 3.10) are:

***VirtualPins*:**

- ***pinId*** (*String*, [1]): Unique identifier for this I/O terminal interface block.
- ***description*** (*String*, [0..1]): Optional documentation describing the hardware interface.

***PinGroup*:**

- ***direction*** (*String*, [1]): Signal transmission direction: *INPUT* for digital sensors sending state data from the plant, or *OUTPUT* for actuation commands received by the asset.

***Pin*:**

- ***name*** (*String*, [1]): The logical tag name or physical PLC/I/O terminal address.

- **value** (*Boolean*, [1]): The instantaneous discrete boolean state of the electrical signal (*true* for HIGH, *false* for LOW).

Through this decoupling, *VirtualPins* strictly reflects the live electrical state, while *BehaviorMapping* interprets that state to orchestrate physical kinematics inside the simulation engine.

3.3.5 ControlLogic Class

The *ControlLogic* class stores local control rules and interlocks inside the asset (0..1), as shown in the UML Class Diagram in [Figure 3.11](#). With this class, machines can run local automated sequences without needing an external PLC.

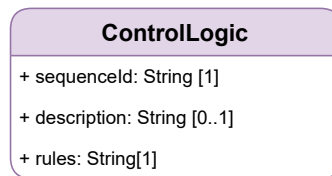


Figure 3.11: UML Class Diagram of the *ControlLogic* class.

The attributes of *ControlLogic* are:

- **sequenceId** (*String*, [1]): Unique identifier for the control sequence.
- **description** (*String*, [0..1]): Optional description of the sequence.
- **rules** (*String*, [1]): Text containing the conditional IF-THEN rules run by the simulation engine.

3.3.6 NetworkEndpoint Class

The *NetworkEndpoint* class specifies the network protocol and address used to communicate with the physical equipment or data services (0..1). The structure of this interface is depicted in the UML Class Diagram in [Figure 3.12](#).

The attributes of *NetworkEndpoint* are:

- **endpointId** (*String*, [1]): Unique identifier for this network interface.
- **description** (*String*, [0..1]): Optional description of the endpoint.
- **protocol** (*String*, [1]): Communication protocol (e.g., TCP socket, HTTP REST, MQTT).
- **endpointIP** (*String*, [1]): IP address or hostname.
- **endpointPort** (*Integer*, [1]): Target port number.

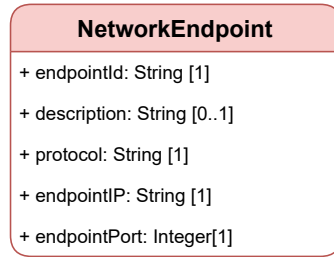


Figure 3.12: UML Class Diagram of the *NetworkEndpoint* class.

3.4 Chapter Summary and Architectural Transition

This chapter established the conceptual foundation of the DT framework. By formalizing four functional requirements (**FR1–FR4**) and four non-functional requirements (**NFR1–NFR4**) under the normative taxonomy of Alemão et al. [46], the framework directly resolves the challenges of vendor lock-in, manual reconstruction, and synchronization drift. The architecture decouples asset authoring from runtime execution through three functional layers, backed by a vendor-neutral semantic metamodel that integrates 3D visual representations, kinematic definitions, virtual pin mappings, behavioral rule logic, and network endpoints into self-describing packages.

While this chapter focused on technology-agnostic specifications, the practical deployment of this architecture requires concrete software engineering choices, standards serializations, and physical hardware interfaces. [Chapter 4](#) details the technical implementation of this architecture, describing the realization of the modeling wizard in Python and PyQt6, the semantic repository using Eclipse BaSyx v2, the runtime simulation engine in Unity, and the real-time embedded firmware on industrial Arduino controllers.

TECHNOLOGICAL IMPLEMENTATION

This chapter describes how the DT framework from [Chapter 3](#) was implemented in practice. Moving from conceptual designs to a working system requires connecting standardized semantic models, such as the AAS, with game-engine physics, interactive software tools, and industrial shop-floor hardware.

To evaluate the pipeline under realistic manufacturing conditions, each layer was implemented and tested on a physical modular FMS bench.

The chapter is organized as follows:

- [Section 4.1](#) introduces the physical FMS bench and gives an overview of the global component architecture and its interfaces.
- [Section 4.2](#) presents Layer 0 (Physical Asset Digitization), covering physical measurements, low-polygon 3D modeling, pivot rigging, and GLB export in Blender.
- [Section 4.3](#) describes Layer 1 (Asset Modeling), detailing the PyQt6 and PyVista desktop application, surface raycasting for placing sensors, the no-code control logic editor, and AASX package generation with the BaSyx Python Software Development Kit (SDK).
- [Section 4.4](#) covers Layer 2 (Storage and Registry), explaining the containerized Eclipse BaSyx v2 microservices, port mappings, and MongoDB database.
- [Section 4.5](#) details Layer 3 (Runtime Execution and Physical Synchronization), focusing on procedural scene assembly in Unity 3D, PhysX contact mechanics, dual-mode execution (VC and operational DT), and kinematic drift correction.
- [Section 4.6](#) describes the physical automation setup and embedded firmware running FreeRTOS on an Industrial Shields M-DUINO PLC, including sensor scanning, low-latency TCP telemetry, and HTTP actuation.
- [Section 4.7](#) maps each technical component to its functional and non-functional requirements in a traceability matrix.

4.1 System Architecture and Technology Realization

Turning the conceptual architecture from [Chapter 3](#) into a working implementation requires matching abstract layer roles to concrete software tools, network protocols, and hardware platforms. This section describes the modular FMS bench used for testing and outlines the overall component architecture.

4.1.1 Physical Testbed Context: The Modular FMS Bench

To validate the framework, an existing FMS testbed was used as the pilot case study ([Figure 4.1](#)), based on the modular CPPS architecture proposed by Rocha et al. [3]. The physical apparatus is an educational and industrial training model built using *fischertechnik* modular mechatronic components, organized as a closed-loop conveyor bench to transport pallets across processing and inspection stations. While compact in physical footprint, this testbed replicates the electromechanical topology and real-time control constraints of real-world production cells, making it well-suited to evaluate whether the DT can reconstruct a factory floor modularly, asset by asset, and coordinate distributed mechanisms in real time.

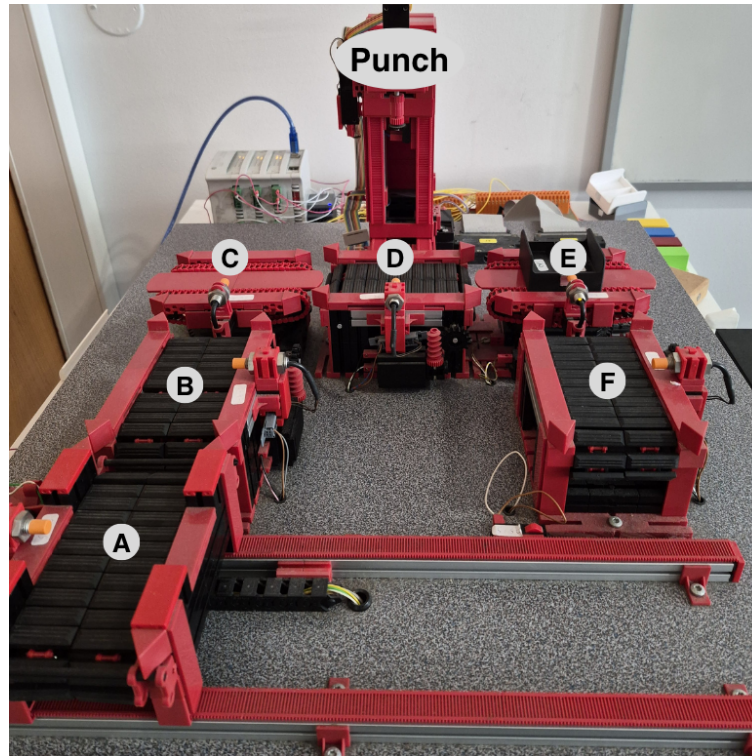


Figure 4.1: Physical modular FMS conveyor testbed built with *fischertechnik* mechatronic modules, showing the conveyor tracks, magnetic sensors, punch mechanism, and Industrial Shields M-DUINO PLC.

Mechanically, the bench includes three stationary conveyor belts (Conveyors B, D, and F), two rotating sections (Conveyors C and E), one translating conveyor (Conveyor A),

and a punch mechanism that simulates drilling, stamping, or screwing operations. Field instrumentation consists of 24 V DC magnetic track sensors to detect workpieces, contact limit switches to monitor conveyor and punch positions, and 24 V DC motors driving each section. Control is managed by an Industrial Shields M-DUINO 57R+ PLC (based on the Arduino Mega 2560), equipped with an Ethernet port and optocoupled I/O boards. Digital input pins (I0_0 to I2_3) read sensor states, while relay outputs (R0_2 to R2_8) command conveyor motors and punch actions (the full pin assignment is listed in [Chapter A](#)).

4.1.2 Overall Component Architecture and Inter-Layer Interfaces

With this physical bench as the target, the complete framework was implemented across four operational layers. The UML Component Diagram in [Figure 4.2](#) illustrates the overall system implementation architecture across the four operational layers.

The information flow spans four operational stages. In Layer 0 (Physical Asset Digitization), physical hardware components are rebuilt as 3D models in Blender using manual measurements. These models are rigged with pivot axes and sensor colliders, then exported as binary glTF files (.glb). Next, Layer 1 (Asset Modeling) runs as a desktop application that loads the 3D model into an interactive viewport. Here, users configure machine properties and assign I/O pins before the BaSyx Python SDK packages everything into an AASX bundle and uploads it to Layer 2 via REST endpoints.

In Layer 2 (Storage and Registry), Eclipse BaSyx v2 services run inside Docker containers. An AAS Environment Repository stores package files, while the AAS Registry and Submodel Registry track active assets and submodel URLs backed by MongoDB. Operators can inspect these assets through the BaSyx Web Graphical User Interface (GUI). Finally, Layer 3 (Runtime Execution and Physical Synchronization) runs the 3D simulation in Unity, connecting directly to the physical testbed controlled by the Industrial Shields M-DUINO PLC. At startup, Unity queries BaSyx to retrieve models and assemble the scene automatically. During operation, the runtime either runs in VC mode using local rules or synchronizes live with physical hardware by exchanging low-latency TCP telemetry and HTTP commands with the PLC.

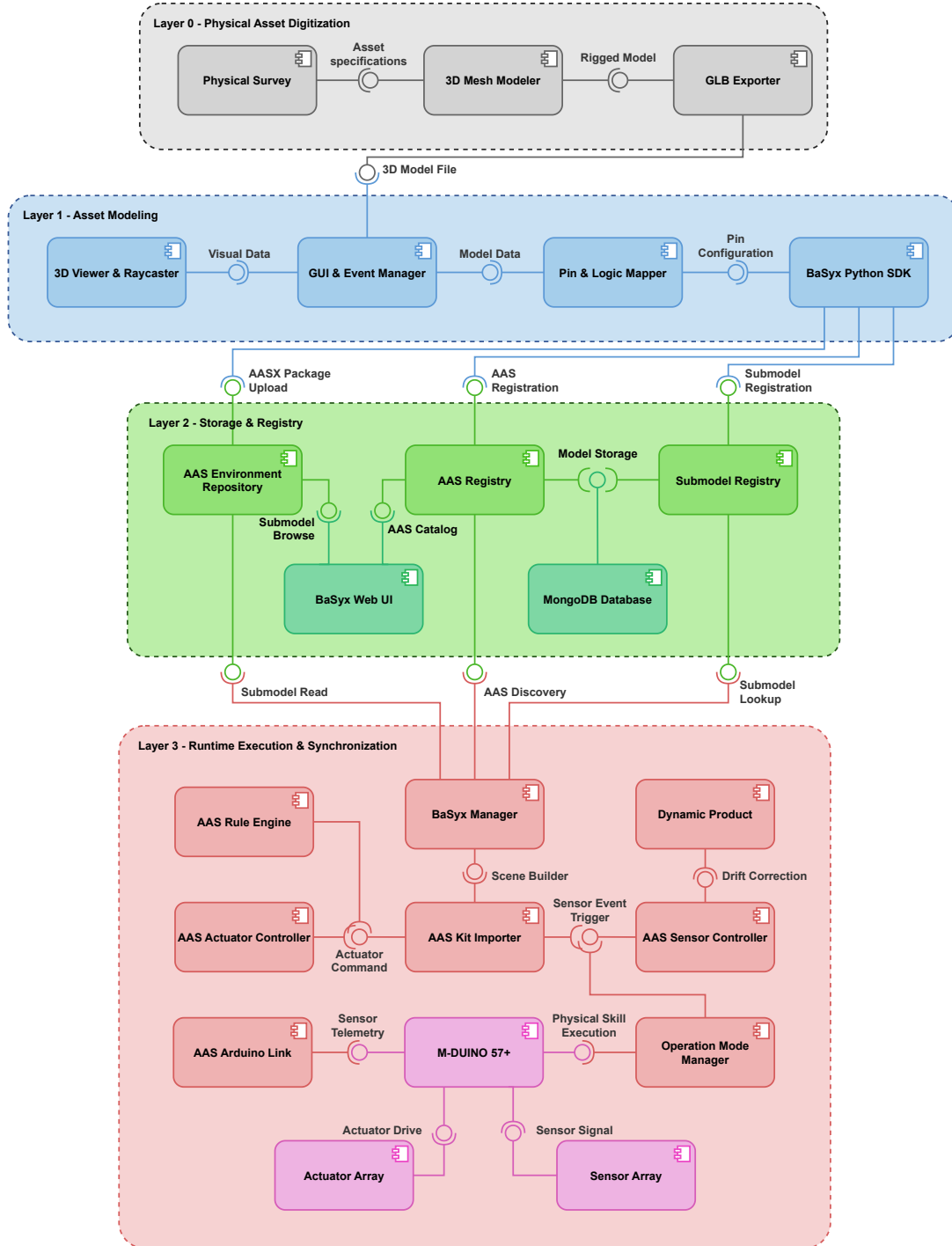


Figure 4.2: UML Component Diagram of the framework implementation architecture: (0) Physical Asset Digitization in Blender, (1) Asset Modeling in the Python desktop application, (2) Storage and Registry in Eclipse BaSyx v2 on Docker, and (3) Runtime Execution and Physical Synchronization in Unity 3D with an Industrial Shields M-DUINO PLC.

4.2 Layer 0: Physical Asset Digitization and 3D Modeling

As established in [Section 3.2.1 \(page 30\)](#), physical machinery on existing factory floors often lacks simulation-ready 3D models. When manufacturing CAD files do exist, their high polygon count slows down real-time physics engines, while missing functional motion joints, pivot points, and interaction colliders. Because the modular FMS bench had no existing vendor models, Layer 0 was carried out as an upstream preparatory stage in Blender 5.2.0 LTS to create lightweight, rigged GLB assets.

The UML Component Diagram in [Figure 4.3](#) illustrates the component architecture of Layer 0, structured into three sequential tasks:

1. **Physical Measurements:** Measuring the dimensions of the physical FMS modules using a measuring tape.
2. **3D Mesh Modeling and Kinematic Rigging:** Reconstructing modules at a 1:1 scale matching Unity’s coordinate frame.
3. **GLB Packaging:** Exporting self-contained binary glTF files (.glb) containing geometry, materials, hierarchies, and trigger colliders for direct ingestion into Layer 1.

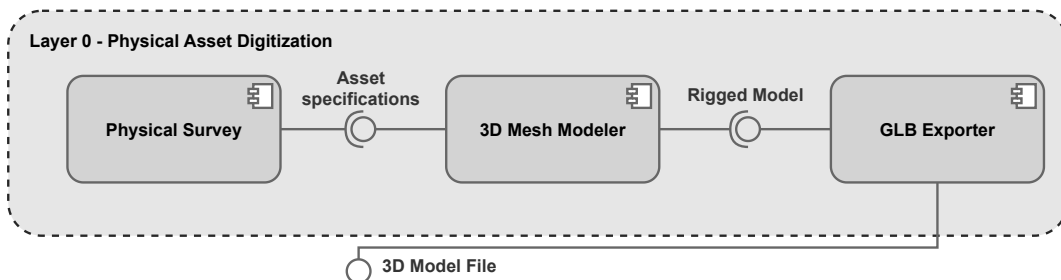
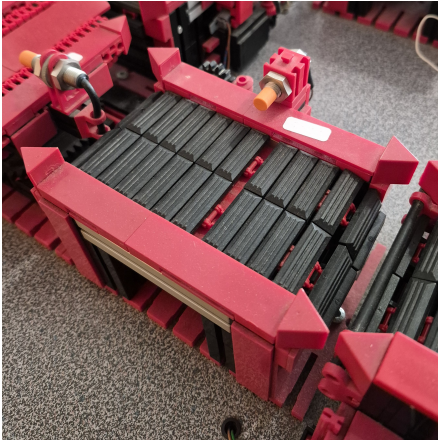


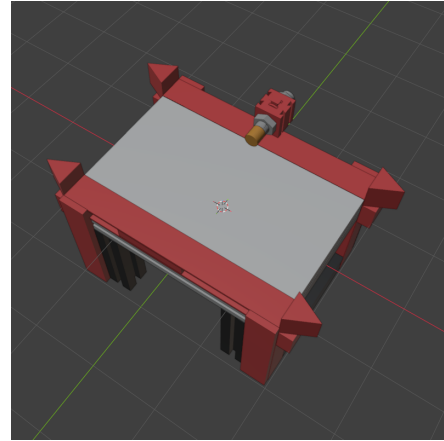
Figure 4.3: UML Component Diagram of Layer 0 (Physical Asset Digitization).

To meet the requirements of the *VisualStructure* metamodel ([Section 3.3.2, page 39](#)), each asset was structured into a clean parent-child hierarchy separating static frames from moving mechanisms. Static chassis parts were merged into a single root node with its coordinate origin set at the base reference point (0, 0, 0). Dynamic elements, such as conveyor belts, rotating conveyors, and the punch station, were isolated into child objects.

The pivot point of each moving component was aligned with its physical axis of rotation or linear motion vector. In physics engines, an offset pivot causes parts to rotate or translate along unintended paths when animated by *BehaviorBinding* scripts ([Section 3.3.3, page 40](#)). In addition, invisible geometric trigger colliders were placed at physical sensor locations to detect passing workpieces during simulation. Once modeled and validated, all modules were exported as compact binary GLB files ready for the modeling application.



(a) Physical conveyor module.



(b) Reconstructed 3D asset.

Figure 4.4: Comparison between the physical modular FMS conveyor module and its reconstructed 3D simulation model in Blender.

4.3 Layer 1: Asset Modeling Desktop Application

Layer 1 was implemented as a standalone desktop application. It combines the Asset Model Editor and Package Exporter roles from [Section 3.2.2 \(page 31\)](#) into a single tool. In line with requirement **NFR4** (Configuration Accessibility), the goal was to allow engineers to configure a DT asset without writing any code.

The UML Component Diagram in [Figure 4.5](#) shows the component structure of Layer 1 (Asset Modeling), divided into four modules: (1) the 3D Viewer & Raycaster renders the model and detects user clicks on the mesh surface, (2) the GUI & Event Manager provides the PyQt6 interface with step-by-step tabs, (3) the Pin & Logic Mapper links visual sensor placements and kinematic rules to controller pin registers, and (4) the BaSyx Python SDK builds the in-memory AAS submodels, packages the AASX archive, and uploads it to Layer 2 via REST endpoints.

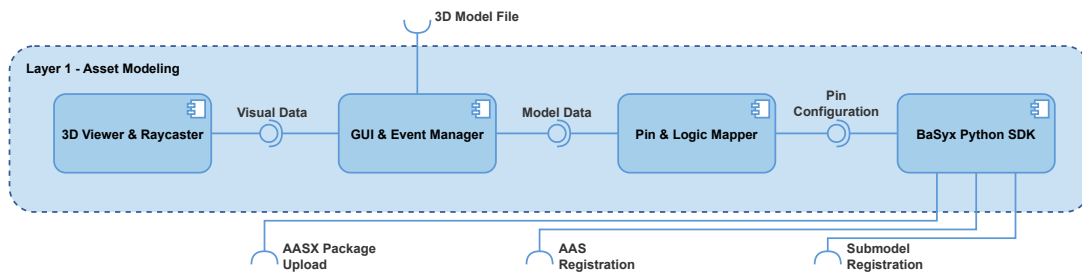


Figure 4.5: UML Component Diagram of Layer 1 (Asset Modeling).

4.3.1 AAS Data Model & Submodel Specifications

Rather than following a rigid sequence, the modeling tool acts as an editor that populates standardized IEC 63278 AAS submodels ([Figure 3.6, page 38](#)). Although the IDTA provides

official templates for static equipment data (such as *Digital Nameplate*), it lacks standard definitions for 3D physics, spatial sensor placement, and live controller synchronization.

To fill this gap, the tool implements the domain-specific submodels defined in [Section 3.3](#) (page 37):

- **VisualStructure**: Stores the 3D GLB file and defines the transform hierarchy (position, rotation, scale) for rendering.
- **VirtualPins**: Maps sensors and actuators to physical PLC addresses (such as `I0_0` or `R0_2`) and stores their boolean values.
- **BehaviorMapping**: Binds components to motion parameters, defining translation or rotation axes, continuous speeds, and sensor trigger envelopes.
- **AssetInterfacesDescription** (or **NetworkEndpoint**): Configures network communication settings, including controller IP addresses, ports, and protocol bindings.
- **ControlLogic**: Formats IF-THEN rules into structured conditions for local edge execution during VC.

4.3.2 Modeling Tool GUI and 3D Viewport Architecture

The desktop application is built with PyQt6, providing a graphical interface that avoids writing raw JSON files by hand. The central 3D viewport uses PyVista and Visualization Toolkit (VTK), with Trimesh handling geometry operations. Within the viewport, users can inspect bounding dimensions and navigate the mesh hierarchy ([Figure 4.6](#)).

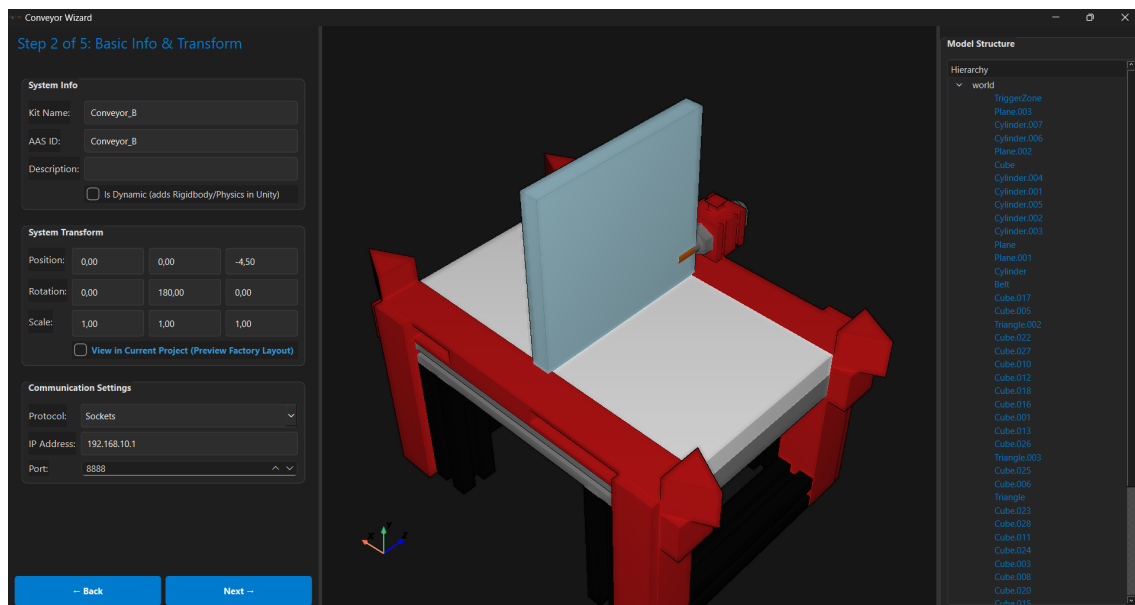


Figure 4.6: GUI of the Asset Modeling application showing the interactive 3D viewport, configuration panels, and model structure hierarchy.

As shown in [Figure 4.6](#), the interface is divided into three main sections:

- **Interactive 3D Viewport (Center):** Displays the 3D model with camera controls (orbit and pan) and visual markers for placed pins and coordinate axes.
- **Configuration Wizard (Left Panel):** Guides the engineer through five sequential setup tabs:
 1. *Model Selection (Step 1):* Selects the asset type and loads the 3D model into the viewport.
 2. *Basic Info & Transform (Step 2):* Configures metadata (AAS ID, asset name, description), 3D transform settings (position, rotation, scale), and communication endpoints (IP and port).
 3. *Configure Sensors (Step 3):* Configures virtual sensor pins directly on the 3D surface using raycasting and sets their parameters.
 4. *Configure Actuators (Step 4):* Defines virtual actuator pins, motion types, direction vectors, and speeds mapped to specific mesh nodes.
 5. *Review & Deploy (Step 5):* Displays the generated submodel tree, previews the JSON manifest, and exports or uploads the .aasx bundle.
- **Model Structure Hierarchy (Right Panel):** Displays the node tree of the asset, allowing users to inspect and select specific sub-components.

4.3.3 Interactive Modeling Workflow and Surface Raycasting

To satisfy requirement **FR2** (Self-Describing Asset Packages) and reduce setup mistakes, sensors and actuators are positioned by clicking directly on the 3D model surface instead of typing coordinates manually.

When the user clicks in the viewport, a ray is projected from the camera through the clicked pixel. The raycasting engine finds the exact 3D contact point on the surface and calculates the surface normal vector. This normal vector automatically sets the orientation of the pin, making sure sensors placed on angled or vertical surfaces point outward in the correct direction.

The complete interactive modeling sequence between the engineer, the GUI, the 3D viewport, and the AAS Data Model is illustrated in the UML Sequence Diagram in [Figure 4.7](#).

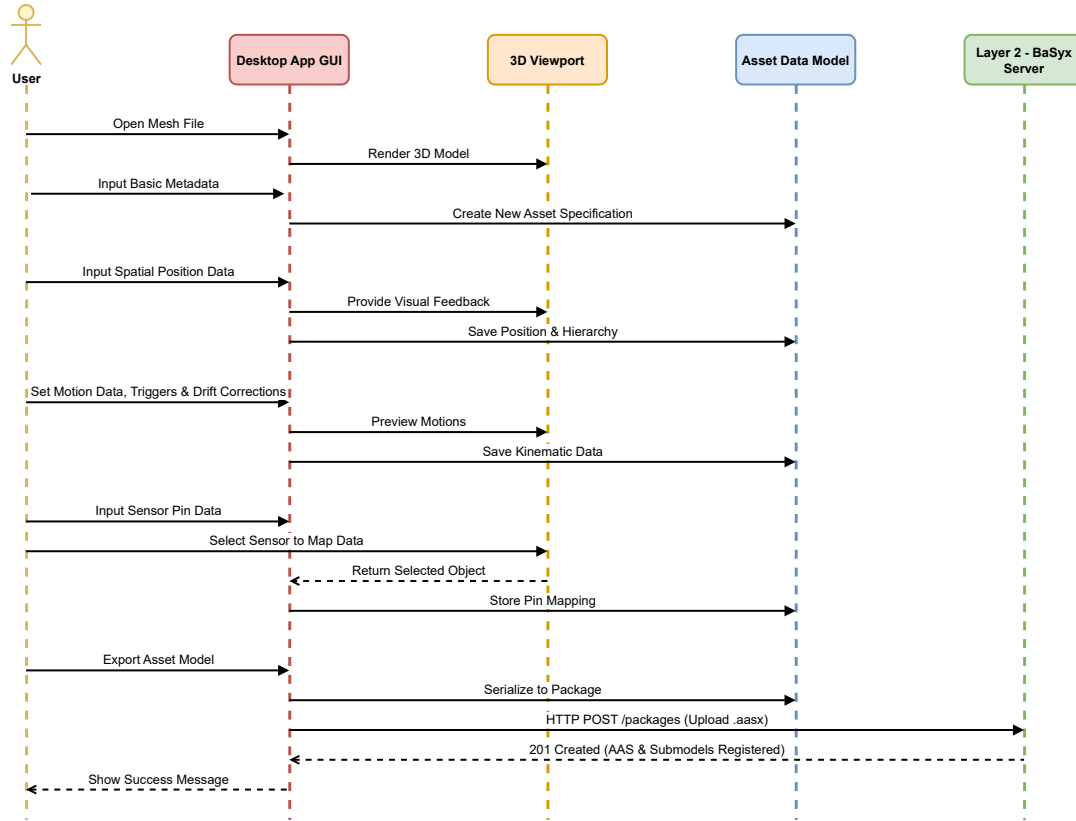


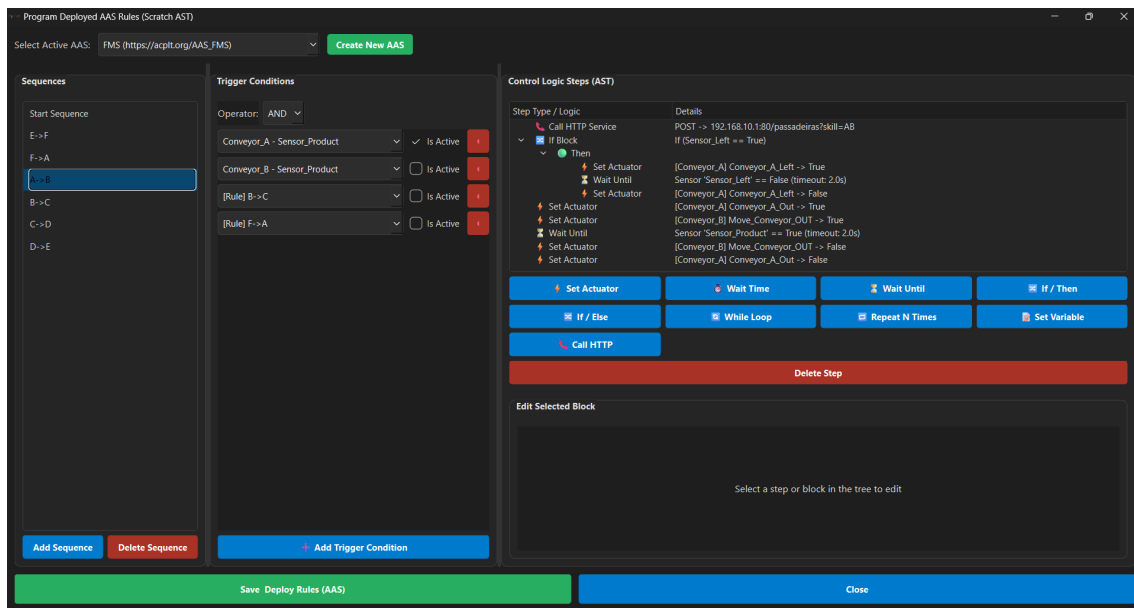
Figure 4.7: UML Sequence Diagram of the Layer 1 interactive asset modeling workflow.

4.3.4 Integrated Rule Editor for Control Logic

To support standalone validation during VC, the tool includes a no-code *ControlLogic* rule editor (**NFR4**). This interface lets domain engineers build sequence logic without writing code, using condition-action pairs tied to sensor states and actuator commands (Figure 4.8).

The editor provides nine modular block types to build the logic tree:

- **Set Actuator:** Changes the state of an actuator pin or motor relay.
- **Wait Time:** Pauses execution for a set number of seconds before continuing.
- **Wait Until:** Holds execution until a sensor condition is met or a timeout expires.
- **If / Then:** Evaluates a condition and runs nested steps when true.
- **If / Else:** Provides two execution branches based on a boolean condition.
- **While Loop:** Repeats a group of steps while a sensor condition holds true.
- **Repeat N Times:** Executes a sequence for a fixed number of cycles (N).
- **Set Variable:** Updates an internal variable used across conditional checks.

Figure 4.8: UI of the no-code *ControlLogic* rule editor.

- **Call HTTP:** Sends an outbound HTTP REST request to trigger external actions or services.

These rules are saved as structured Abstract Syntax Trees (ASTs) in the *ControlLogic* submodel, enabling Unity's local rule engine to simulate machine sequences without an external PLC. The UML Sequence Diagram in Figure 4.9 illustrates the rule configuration and deployment workflow.

Once asset properties, kinematic mappings, virtual pins, and control logic are fully defined, the application initiates the export pipeline. Using the basyx-python-sdk library, the tool builds the official IEC 63278 AASX archive in memory. The 3D GLB mesh is stored as an embedded binary *Blob*, while the five submodels are serialized into standard-compliant JSON manifests within the package. Finally, the application validates package integrity and automatically uploads the generated AASX bundle to the Layer 2 BaSyx repository via REST endpoints.

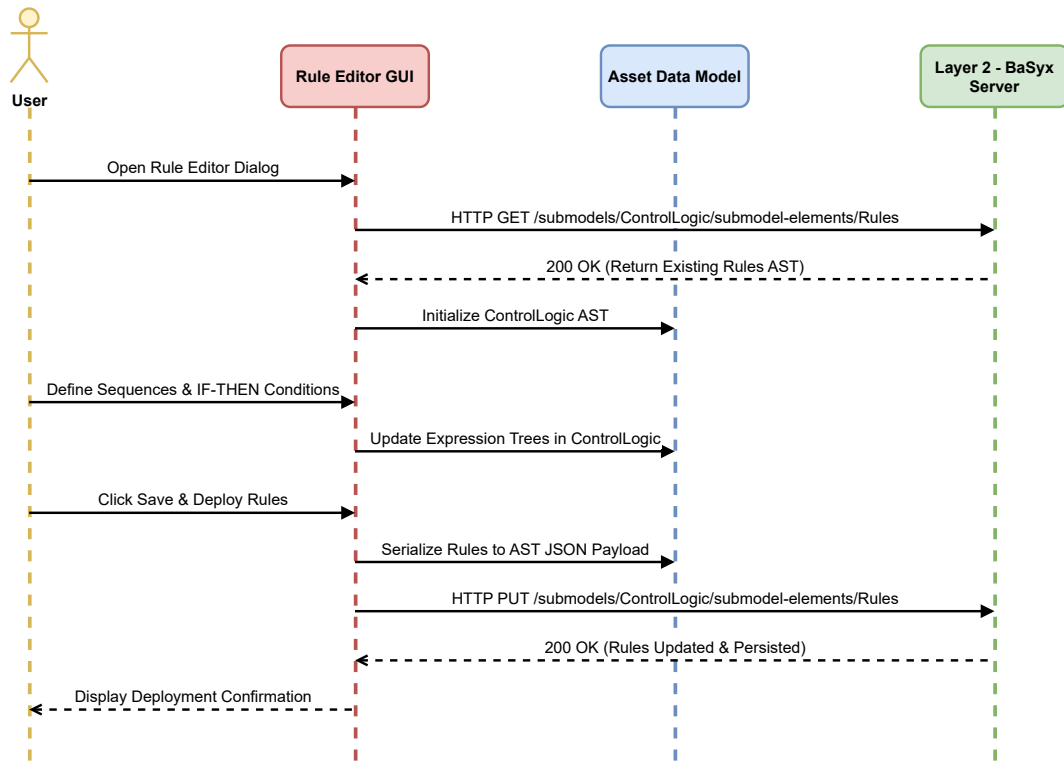


Figure 4.9: UML Sequence Diagram of the *ControlLogic* rule configuration and deployment workflow.

4.4 Layer 2: Storage and Registry (Eclipse BaSyx v2)

Layer 2 is implemented using Eclipse BaSyx v2 deployed with Docker Compose. Docker makes the entire server stack reproducible: running Docker Compose starts all services in the right order without manual configuration. This setup allows new asset submodels and services to start automatically, fulfilling requirement **FR1** (Plug-and-Play Asset Onboarding).

The UML Component Diagram in Figure 4.10 illustrates the component structure of Layer 2 (Storage and Registry), which consists of five main elements. The AAS Environment Repository stores uploaded AASX packages, handles package ingestion, and serves models to Unity clients. The AAS Registry and Submodel Registry keep track of registered shells and submodel URLs, offering endpoints for registration and discovery. A shared MongoDB database stores metadata and descriptors persistently. Finally, operators can inspect active assets and submodels using the BaSyx Web GUI.

The UML Deployment Diagram in Figure 4.11 illustrates the container networking, execution environments, and service deployment topology of Eclipse BaSyx v2.

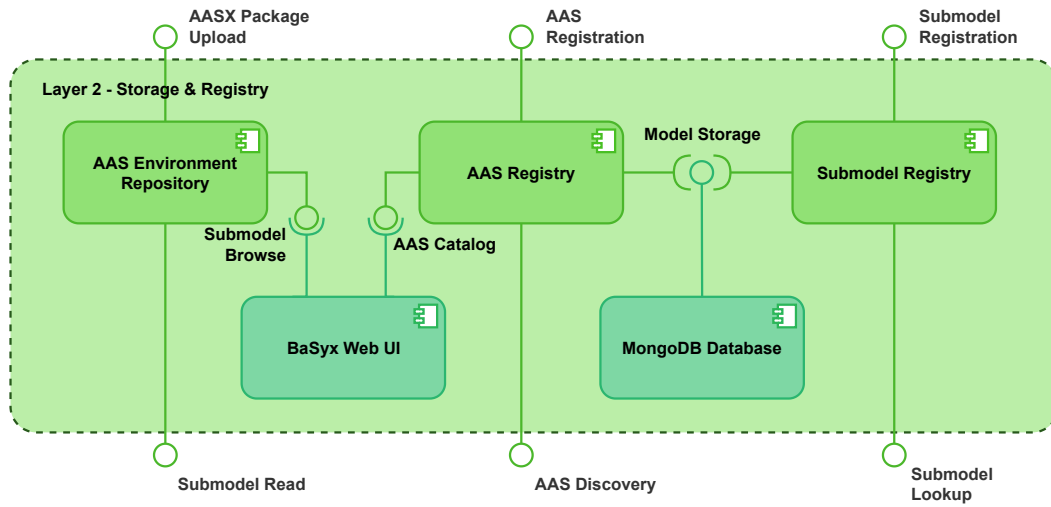


Figure 4.10: UML Component Diagram of Layer 2 (Storage and Registry).

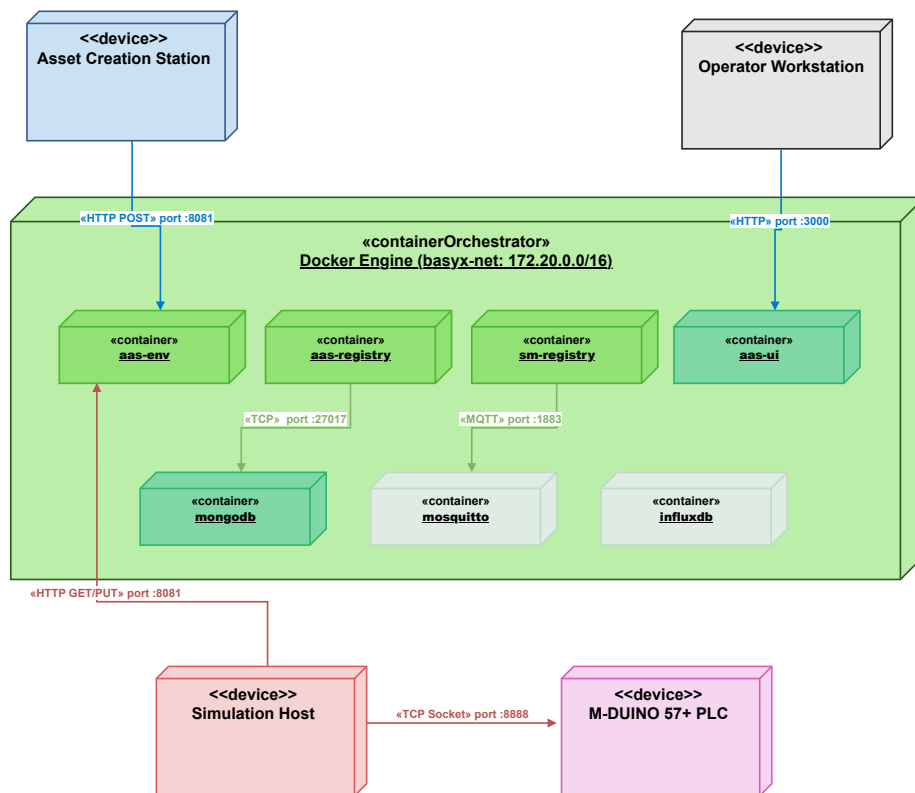


Figure 4.11: UML Deployment Diagram of the Eclipse BaSyx v2 Docker deployment and container topology.

4.4.1 Microservice Architecture and Port Allocation

The containerized services carry out the repository and registry tasks described in [Figure 3.4](#) (page 34). The AAS Environment Repository (port 8081) exposes standard REST endpoints to store and serve packages, submodels, and 3D mesh files. The AAS Registry (port 8082) and Submodel Registry (port 8083) maintain directories of active assets, mapping unique IDs to network URLs. Persistent storage is provided by MongoDB (port 27017). By separating these functions into microservices, Layer 2 scales easily as more machines and clients join the network, satisfying requirement **NFR2** (Scalability). Although the standard BaSyx stack includes Mosquitto MQTT (port 1883) and InfluxDB (port 8086), operational DT execution in this work uses direct TCP sockets with the PLC to minimize telemetry delay. Operators can inspect the running system through the BaSyx Web dashboard (port 3000).

4.4.2 Package Ingestion and Registry Resolution

When an AASX package is uploaded via HTTP POST to the repository endpoint (port 8081), the server decompresses the package archive and parses the internal manifests. Embedded 3D mesh files (.glb) are extracted and placed directly into the static web server directory, making them accessible to simulation clients via standard URLs. Concurrently, the AAS metadata and individual submodels (visual structure, behavior mapping, virtual pins, control logic, and network endpoints) are committed to MongoDB for persistent retrieval.

Following successful ingestion, registration calls are dispatched to both the AAS Registry (port 8082) and the Submodel Registry (port 8083). These records publish the asset identifier, human-readable name, and specific REST endpoint URLs for each submodel. By centralizing these discovery records, simulation clients in Layer 3 can locate all active machines, query their physical configurations, and retrieve their operational parameters with a single directory request.

4.5 Layer 3: Runtime Execution and Physical Synchronization

Layer 3 connects the Unity 3D simulation with the physical testbed through the components shown in the UML Component Diagram ([Figure 4.12](#)). The main orchestrator queries Layer 2 to download models and instantiate virtual assets. The asset importer parses submodel descriptors and attaches runtime scripts to the corresponding game objects. Virtual sensor inputs and actuator commands are managed by dedicated controllers, while the rule engine executes local conditional logic during VC. The dynamic product controller handles workpiece movement and applies position corrections. Runtime modes are managed by the operation mode manager, while the hardware telemetry link streams data over TCP and triggers physical actions via HTTP. On the physical bench, the Industrial Shields M-DUINO PLC reads sensor inputs through optocouplers and drives relay outputs

for conveyor motors. This modular design isolates failures between layers, fulfilling requirement **NFR3** (Layer Independence).

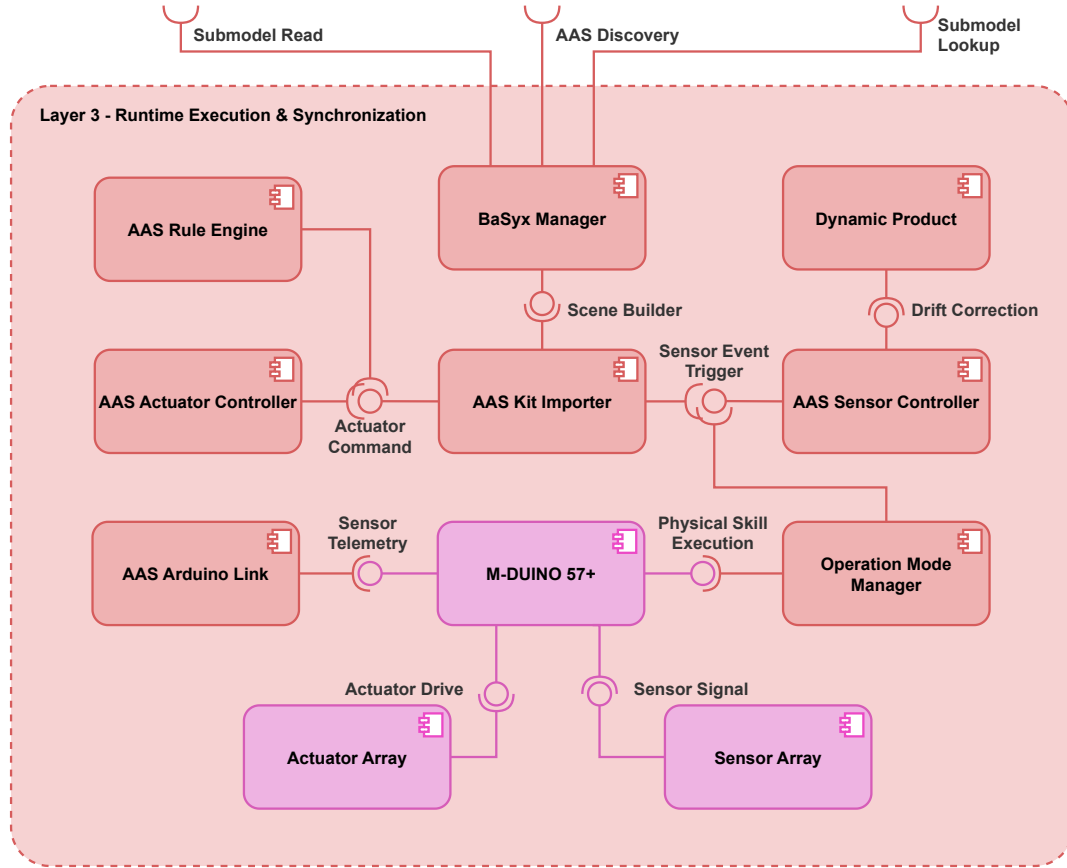


Figure 4.12: UML Component Diagram of Layer 3 (Runtime Execution and Physical Synchronization).

4.5.1 Dynamic Ingestion Flow and Procedural Scene Construction

To satisfy requirements **FR3** (Dynamic Scene Assembly) and **FR1** (Plug-and-Play Asset Onboarding), the Unity environment builds the scene procedurally at launch without using pre-saved scene files. The sequence follows three main phases, illustrated in the UML Sequence Diagram (Figure 4.13):

1. **Phase 1: Registry Discovery and Cache Clearing.** At launch, the orchestrator clears old cached files from disk. It then sends an HTTP GET request to the BaSyx AAS Registry on port 8082 to retrieve all registered assets and starts an asynchronous ingestion task for each asset found.
2. **Phase 2: Procedural Import, Mesh Setup, and Physics Configuration.** For each asset, the importer fetches the submodels (visual structure, virtual pins, behavior mapping, and control logic) from port 8081, along with the 3D GLB file. Using

glTFast, Unity builds the 3D object hierarchy. A 180° rotation around the Y-axis aligns Blender's right-handed coordinates with Unity's left-handed space. Non-uniform scales are applied directly to vertices to prevent distortion, and sensor/actuator scripts are attached to the corresponding nodes.

3. **Phase 3: Dual-Mode Execution and Telemetry Loop.** After a brief settling time for physics dynamics, the runtime enters one of two operating modes:

- **Mode A (VC):** The rule engine parses the IF-THEN rules from the control logic submodel. When a workpiece enters a sensor collider, the sensor controller notifies the rule engine, which evaluates the logic and commands virtual actuators in a closed software loop.
- **Mode B (DT):** Local rule evaluation is disabled. The telemetry link opens a multi-threaded TCP connection to the physical PLC on port 8888. Incoming telemetry packets are placed in a queue and processed on Unity's main thread to update sensor highlights and trigger kinematic position snapping. Meanwhile, asynchronous HTTP calls update live values in the BaSyx repository.

4.5.2 Conveyor Physics Mechanics, Dual-Mode Execution, and Drift Correction

To provide realistic behavior, conveyor belts are simulated using contact physics rather than artificial coordinate translation. Belt surfaces are configured with PhysX friction coefficients and surface speeds. When workpieces contact the belt, friction, acceleration, and contact forces are simulated naturally.

The runtime supports two execution modes (Figure 4.13):

- **Mode A (VC):** In VC mode (`DIGITAL_TWIN = false`), logic is tested without physical hardware. When a workpiece triggers a virtual sensor, the sensor controller informs the rule engine, which executes the IF-THEN rules from the *ControlLogic* submodel to command virtual actuators.
- **Mode B (DT):** In DT mode (`DIGITAL_TWIN = true`), local rule evaluation is disabled. The system connects directly to the PLC via a low-latency TCP socket (port 8888). Sensor state changes from the bench immediately update virtual markers and notify the dynamic product controller. At the same time, asynchronous HTTP calls update live properties in the BaSyx repository.

Because numerical physics calculations and network timing introduce small variations over time, virtual workpieces can drift slightly from physical parts. To meet requirement FR4 (Physical-Virtual Synchronization), the runtime uses a kinematic snapping mechanism. When a physical optical sensor triggers, the runtime finds the nearest virtual workpiece and snaps its position to the known sensor coordinates, eliminating accumulated drift. Figure 4.14 shows the runtime environment running in Unity.

4.5. LAYER 3: RUNTIME EXECUTION AND PHYSICAL SYNCHRONIZATION

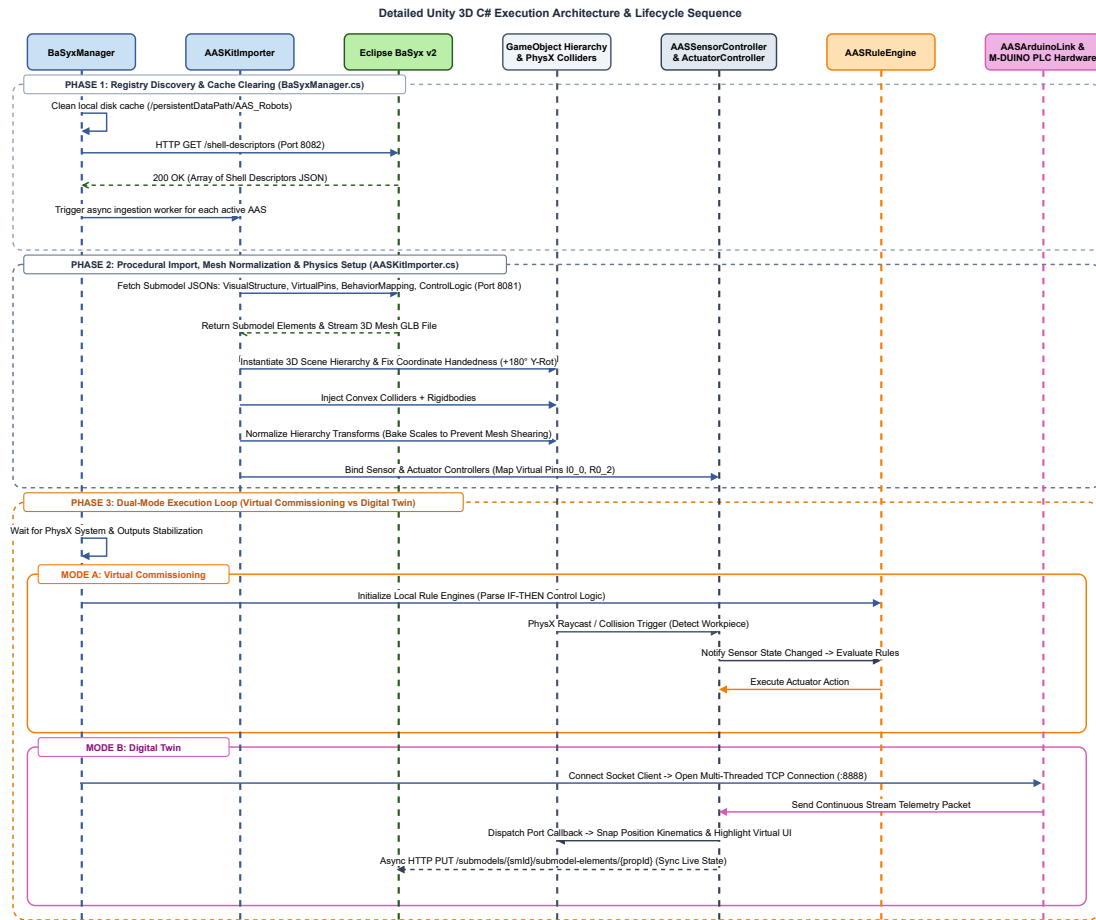


Figure 4.13: UML Sequence Diagram of the Unity 3D runtime execution environment.

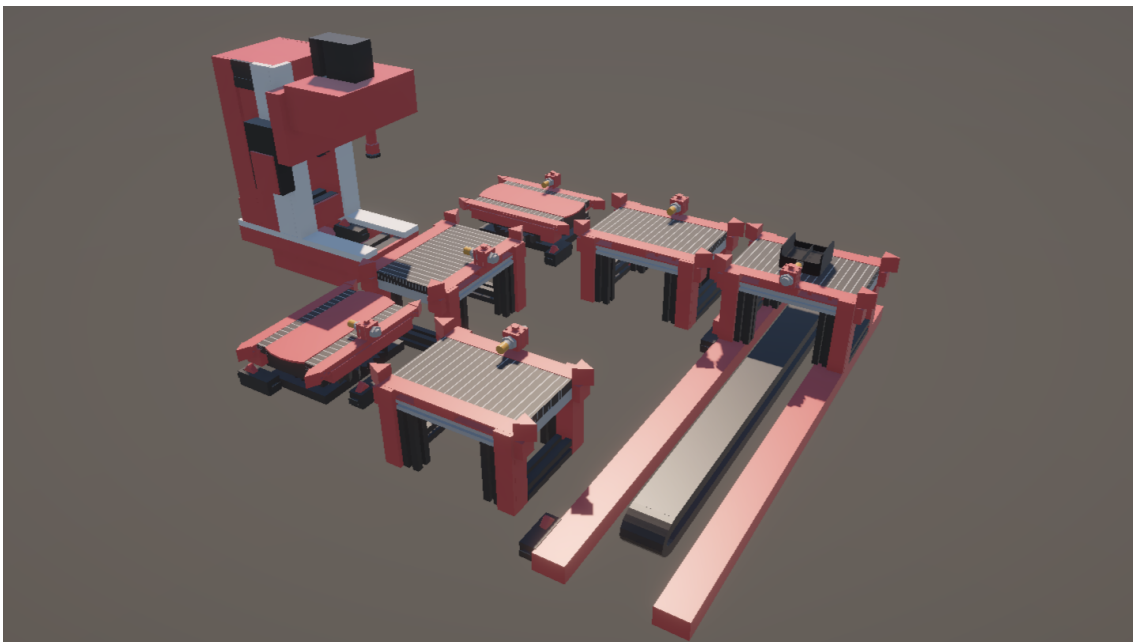


Figure 4.14: 3D visualization and DT runtime running in Unity 3D.

4.6 Physical Automation Setup and Embedded Firmware

The physical testbed is a modular closed-loop conveyor system controlled by an Industrial Shields M-DUINO 57R+ PLC, powered by an Arduino Mega 2560 microcontroller and a W5500 Ethernet controller. The UML Sequence Diagram in Figure 4.15 illustrates the embedded FreeRTOS firmware execution across hardware setup, sensor scanning, event queues, TCP telemetry streaming, and HTTP actuation channels.

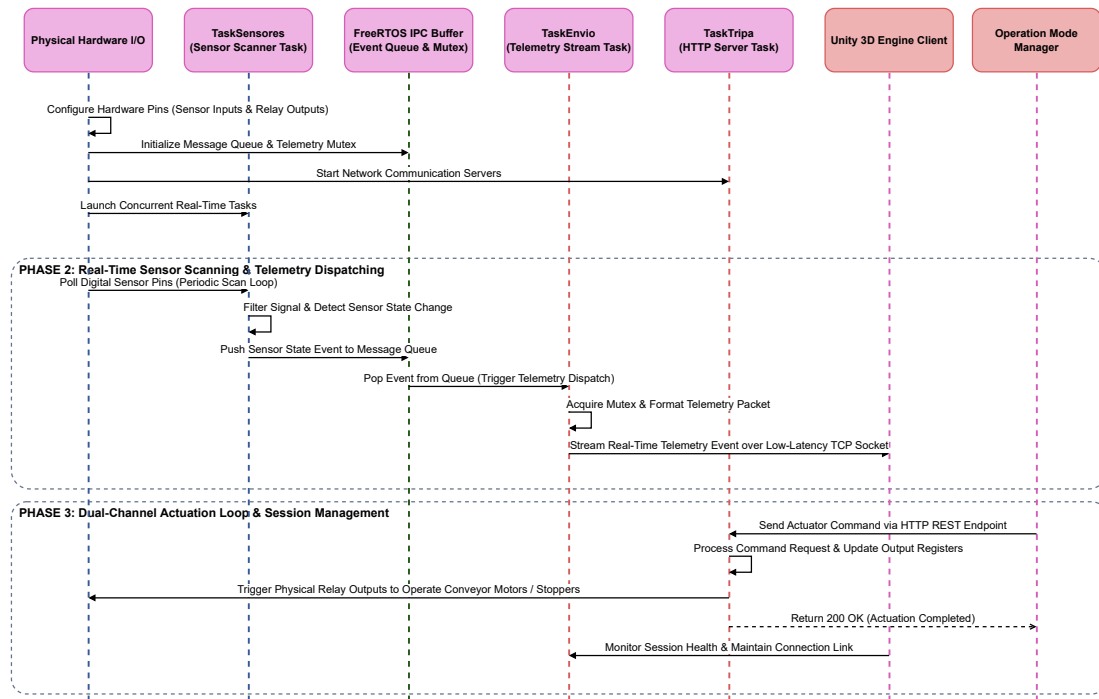


Figure 4.15: UML Sequence Diagram of the embedded FreeRTOS firmware execution.

The physical testbed is instrumented with 24 V DC optical and inductive proximity sensors. To protect the 5 V microcontroller logic from electrical noise and voltage transients caused by motor switching, all field inputs pass through optocoupled galvanic isolation stages before routing to controller General-Purpose Input/Output (GPIO) pins.

The embedded firmware runs on the FreeRTOS real-time operating system to guarantee deterministic task scheduling and non-blocking network communication across four concurrent tasks:

1. **Hardware Setup and Server Initialization:** On boot, the microcontroller configures input pins and motor relay output registers, initializes a 30-element First-In, First-Out (FIFO) sensor event queue, and creates a mutex semaphore protecting socket transmission. It then starts an embedded HTTP server on port 80 and a raw TCP socket server on port 8888, handing task execution to the FreeRTOS preemptive scheduler.

2. **Real-Time Scanning and Telemetry:** The sensor scanner task operates as the highest-priority routine (Priority 3). It performs a 5 ms scan loop across digital input pins with software debounce filtering. When a valid state change occurs, an event token is placed in the FIFO queue. Buffering up to 30 events in this queue prevents dropped edge transitions if temporary network jitter or socket congestion delays transmission. The queued token immediately wakes the telemetry task (Priority 2), which formats a compact JSON packet and streams it over TCP port 8888 to Unity with minimal latency, directly satisfying **NFR1** (Low-Latency Telemetry).
3. **Actuation and Connection Management:** External actuation requests are handled by the HTTP server task (Priority 1) on port 80, which parses incoming command parameters and drives physical relays. Concurrently, a session monitor task (Priority 1) performs connection watchdog checks, recovering automatically from network drops without interrupting automation routines.

4.7 Implementation Traceability and Requirement Coverage

[Table 4.1](#) maps each operational tier, core technology, and requirement to verify how the technical implementation fulfills the design criteria established in [Section 3.1](#).

[Section 4.2](#) and [Section 4.3](#) address asset preparation and onboarding. By utilizing Blender 5.2 LTS to enforce a 1:1 metric scale, aligned pivot points, and low-polygon geometries, Layer 0 satisfies **FR2** (Graphical and Kinematic Fidelity). Layer 1 builds upon this foundation via a PyQt6/PyVista desktop application, leveraging 3D surface raycasting, a visual rule editor, and automated AASX packaging to fulfill **FR1** (Modular Asset Specification), **FR2**, and **NFR4** (No-Code Configuration).

At the repository level ([Section 4.4](#)), Layer 2 relies on containerized Eclipse BaSyx v2 microservices deployed via Docker and backed by MongoDB. By maintaining decoupled AAS and submodel registries accessible through standard REST API endpoints, Layer 2 satisfies **FR1**, **FR3** (Dynamic Scene Assembly), **NFR2** (Interoperability), and **NFR3** (Extensibility and Scalability).

Finally, the execution loop is closed across Layer 3 and the physical hardware ([Section 4.5](#), [Section 4.6](#)). Unity 3D queries BaSyx registries at startup to perform procedural scene assembly, satisfying **FR1**, **FR3**, and **NFR3**. During runtime, PhysX contact mechanics and kinematic position snapping enforce **FR4** (Physical-Virtual Synchronization) by eliminating spatial drift between physical workpieces and virtual models. On the physical bench, the Industrial Shields PLC running FreeRTOS delivers 5 ms input scanning with software debouncing and dedicated TCP/HTTP socket streaming, directly fulfilling **NFR1** (Low-Latency Telemetry) and **FR4**.

Table 4.1: Traceability matrix mapping architectural layers and technical components to design requirements.

Layer / Component	Core Technologies	Requirements	Functional Role
Layer 0: Digitization	Blender 5.2 LTS, glTF/GLB	FR2	1:1 metric modeling, pivot alignment, low-poly geometry
Layer 1: Modeling App	Python, PyQt6, PyVista, basyx-python-sdk	FR1, FR2, NFR4	3D raycasting sensor mapping, rule editor, AASX packaging
Layer 2: Storage & Reg.	Eclipse BaSyx v2, Docker, MongoDB, REST	FR1, FR3, NFR2, NFR3	Decoupled AAS and submodel registries, dynamic asset discovery
Layer 3: Unity Runtime	Unity 3D, C#, glTFast, PhysX	FR1, FR3, FR4, NFR3	Procedural scene assembly, contact physics, dual-mode execution, drift correction
Hardware & Firmware	Industrial Shields PLC, FreeRTOS, TCP/IP	FR4, NFR1	Fast 5 ms input scanning, optocoupled isolation, dual-channel communication

4.8 Chapter Summary

This chapter presented the physical and software realization of the DT framework across four tiers. Layer 0 digitized existing conveyor hardware in Blender to produce low-poly GLB models with properly aligned motion pivots. Layer 1 implemented the Python desktop application, enabling engineers to visually instrument equipment using 3D raycasting and generate standardized AASX packages without programming. Layer 2 realized model storage and discovery registries using containerized Eclipse BaSyx v2 microservices on Docker backed by MongoDB. Finally, Layer 3 implemented the runtime in Unity 3D, providing procedural scene assembly, PhysX contact mechanics, dual-mode execution (VC and operational DT), and drift correction. The experimental evaluation of this implementation is presented in [Chapter 5](#).

RESULTS AND EVALUATION

This chapter evaluates the computational performance, scalability limits, and runtime feasibility of the proposed DT framework. The central premise of this work is that industrial simulation environments can be assembled and executed procedurally from standardized, vendor-neutral AAS packages without relying on hard-coded scenes or static configuration files.

The remainder of this chapter is organized as follows:

- [Section 5.1](#) presents an end-to-end qualitative case study demonstrating how an industrial mechatronic module is authored, packaged into AAS, uploaded to the registry, and procedurally instantiated in Unity.
- [Section 5.2](#) provides an empirical validation benchmarking the complete digital replica of the physical modular FMS bench across 100 cold-boot execution lifecycles.
- [section 5.3](#) conducts a parametric scalability stress test evaluating computational boundaries across eight scale levels ($N \in \{5, \dots, 1000\}$ conveyor modules).
- [Section 5.4](#) synthesizes practical findings, discusses computational limitations and geometric trade-offs, and systematically verifies the research hypotheses.

All benchmark tests were executed on a Lenovo laptop equipped with a 13th Generation Intel Core i7-13650HX processor (14 cores, 20 threads up to 4.9 GHz), 16 GB of DDR5 Random Access Memory (RAM), and an NVIDIA GeForce RTX 4060 Laptop Graphics Processing Unit (GPU) with 8 GB of Video Random Access Memory (VRAM), running Windows 11 Home (version 25H2) and Unity 6 (version 6000.3.10f1).

5.1 Demonstrative Case Study: End-to-End Asset Onboarding

To validate the practical usability and operational workflow of the framework (addressing the visual modeling objective and requirement **NFR4**), this section presents an end-to-end walkthrough demonstrating the onboarding of a new mechatronic asset: introducing an additional modular conveyor unit, designated as **Conveyor G** (Asset ID: Conveyor_G), into

the production line. The case study demonstrates how this new equipment is designed, semantically enriched, packaged into a standardized AAS, and procedurally executed as an active DT without modifying or recompiling the simulation scene.

The onboarding lifecycle spans five sequential stages, progressing from initial 3D mesh modeling to operational DT execution:

5.1.1 CAD Design and Kinematic Rigging in Blender

To guarantee full geometric control, lightweight rendering, and accurate dimensional conformity with the physical equipment, mechatronic modules are designed directly from scratch in Blender rather than relying on unoptimized third-party exports. As described in [Section 4.2](#) and illustrated in [Figure 4.4 \(page 51\)](#), the mechanical structure is modeled to scale based on physical tape measurements of the testbed.

The mesh reproduces the functional geometry while avoiding redundant internal features, ensuring high real-time rendering efficiency. The assembly is partitioned into distinct sub-meshes separating static frame elements from mobile components, such as conveyor belts. The local coordinate origin (*pivot point*) of each mobile part is positioned on its physical axis of motion (rotational center for rotating conveyors or linear axis for translation), preventing geometric distortion during procedural runtime animations. Finally, the finished assembly is exported as a single binary GLB container with embedded materials and standard Y-up coordinate orientation.

5.1.2 3D Model Ingestion and Hierarchy Verification

With the GLB file produced, the automation engineer opens the lightweight Asset Modeling Desktop Application ([Section 4.3](#)). As demonstrated in [Figure 4.6 \(page 52\)](#), the application provides an interactive 3D viewport powered by an embedded rendering engine.

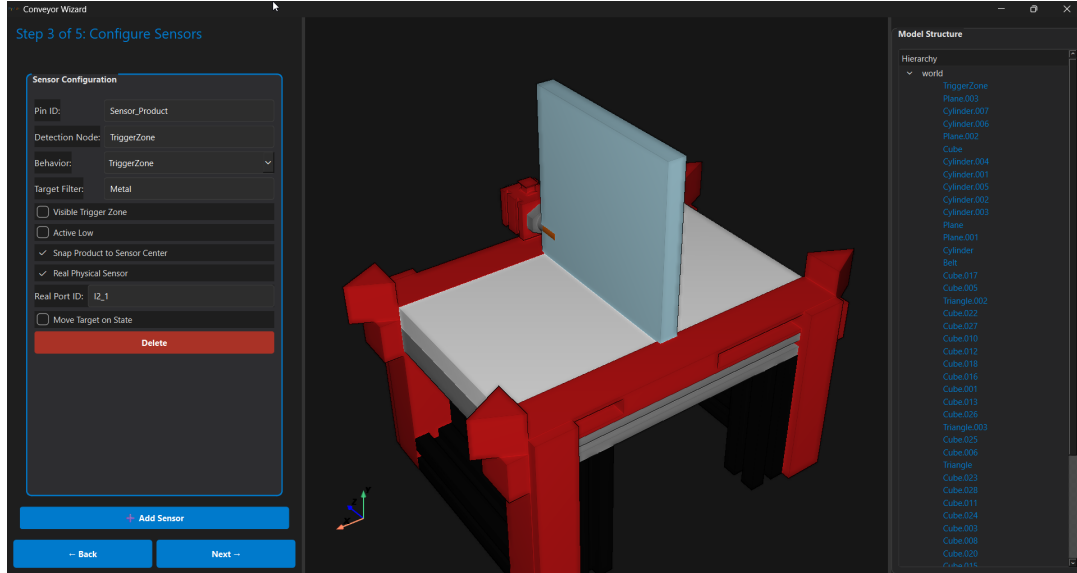
The engineer loads the binary GLB file into the workspace, prompting the ingestion pipeline to parse the internal scene graph, normalize bounding dimensions, and populate the interactive node hierarchy tree. The user inspects the component to verify that node names match the intended physical taxonomy and confirms that pivot orientations align with expected motion vectors before proceeding to semantic pin configuration.

5.1.3 Interactive Pin Mapping and Control Logic Definition

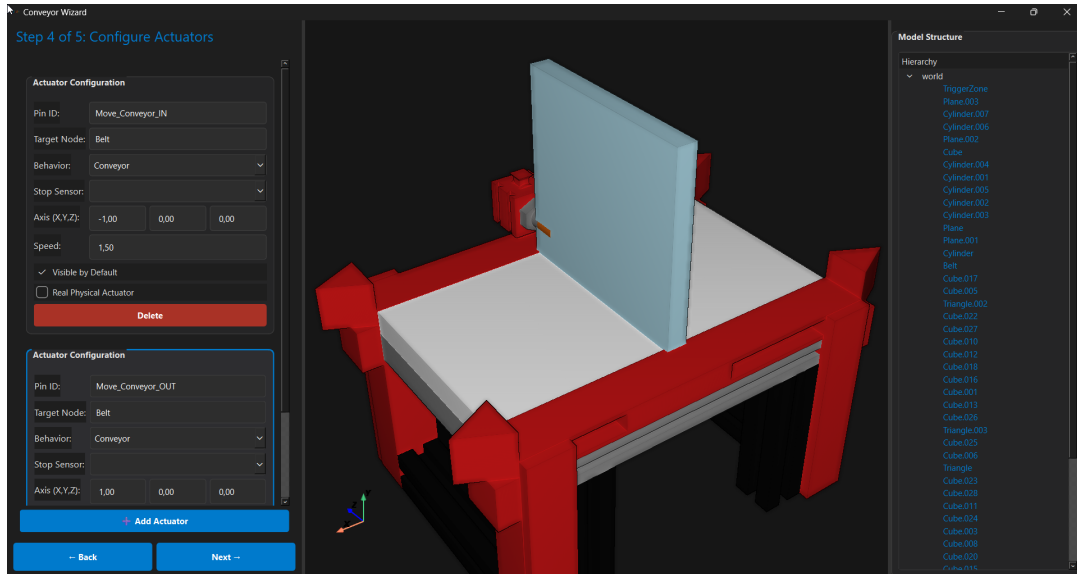
In the third stage, the engineer equips the geometric mesh with cyber-physical semantics without writing software scripts. Rather than manually measuring and typing three-dimensional coordinates, the engineer clicks directly onto the surface of the conveyor mesh in the viewport. The application casts a ray from the camera, calculates the exact surface intersection point, and computes the surface normal vector.

A virtual optical beam sensor is aligned along the surface normal and anchored to the clicked coordinate. In the configuration panel, the user specifies its operational properties, including register binding (`I0_0`), polarity (active-low), and ground-truth workpiece

snapping coordinates for position drift correction. Moving parts, such as the conveyor surface, are bound to actuator registers ($R0_2$). If local edge behavior is needed, such as stopping the belt automatically when a photoeye is blocked without waiting for master PLC polling, the user defines local event-condition-action rules in the *ControlLogic* editor.



(a) Interactive optical sensor placement via surface raycasting.



(b) Actuator kinematic configuration and output register binding.

Figure 5.1: Interactive virtual pin mapping in the desktop onboarding wizard, showing: (a) sensor placement using surface raycasting, and (b) kinematic actuator binding.

5.1.4 Control Logic Rule Definition

Following signal and kinematic configuration, the engineer specifies autonomous edge behavior for Conveyor G using the visual Rule Editor introduced in [Figure 4.8](#) (page 55).

Rather than writing low-level code or proprietary scripts, the engineer composes event-condition-action statements through visual blocks. For this conveyor module, a transfer sequence named $A \rightarrow G$ is defined: belt motion starts when the sensor on Conveyor A is triggered and stops once the sensor on Conveyor G is activated. These rules are serialized into the standard *ControlLogic* submodel schema, allowing the runtime to execute localized automation independently of external supervisory intervention.

5.1.5 AASX Compilation and Repository Publication

Once physical geometry, pin assignments, behavioral bindings, and operational rules are defined, the engineer triggers package export. The tool uses the Eclipse BaSyx Python SDK to generate the standardized submodel descriptors that describe the asset: *VisualStructure* (mesh references, node hierarchy, and spatial transforms), *VirtualPins* (input and output signal interfaces), *BehaviorMapping* (kinematic parameters, speeds, motion vectors, and sensing trigger envelopes), *AssetInterfacesDescription* (network communication endpoints and protocol bindings), and optionally *ControlLogic* (autonomous edge rules for virtual commissioning).

The serialized submodel definitions and the binary GLB mesh are packed into an OPC AASX container file. The application dispatches an HTTP POST request directly to the centralized Eclipse BaSyx v2 server. As shown in the BaSyx AAS Viewer in Figure 5.2, the server unpacks the container, exposes the submodels through standard REST endpoints, and registers the global Asset ID in the AAS Registry.

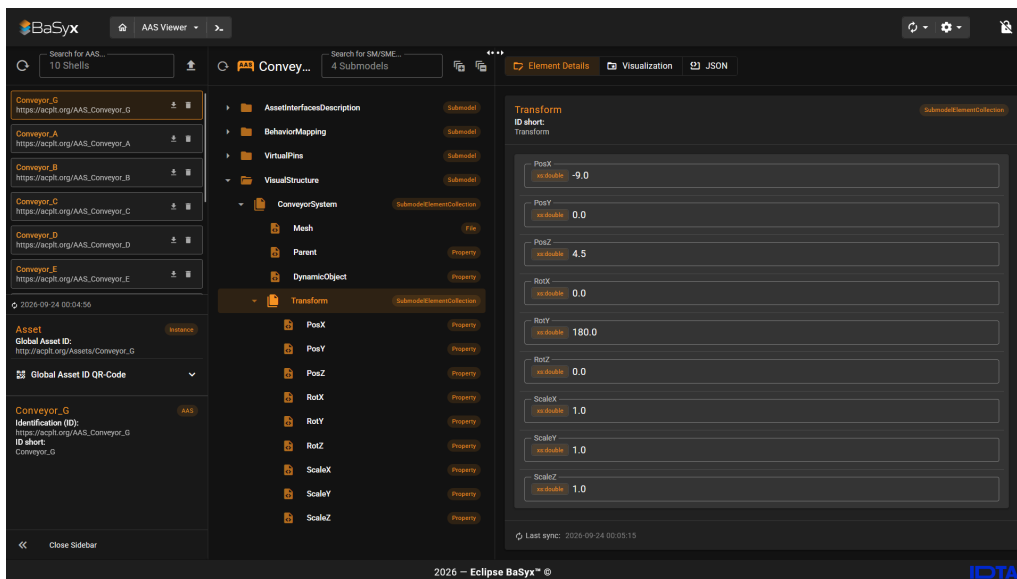


Figure 5.2: Standardized AASX container for Conveyor G uploaded and registered in the Eclipse BaSyx v2 repository, displaying its submodels in the AAS Viewer.

5.1.6 Zero-Touch Procedural Instantiation and Live Telemetry

In the final stage, the Unity 3D simulation environment is launched without pre-baked assets. Upon initialization, the Unity runtime queries the BaSyx registry over REST, retrieves active Asset IDs, and downloads the associated GLB binary files and submodel manifests into transient memory.

The runtime procedural builder parses the *VisualStructure* and instantiates the mesh hierarchy, inspecting component meshes dynamically to attach optimized physics colliders and configure physical rigidbodies. The builder then parses *VirtualPins* and *BehaviorMapping* to instantiate trigger zones at the exact coordinates recorded during raycasting, binding them to internal signal dispatchers, while configuring surface velocities, rotation axes, and constraints onto designated mobile parts. If present, *ControlLogic* rules are loaded by the local rule interpreter, completing the procedural assembly of the simulation scene (Figure 5.3).

Finally, the simulation engine connects to the physical controller via TCP telemetry. When a workpiece on the physical workbench crosses an optical sensor, the physical PLC sets register `I0_0` to HIGH. The telemetry listener receives the frame, trips the corresponding virtual sensor indicator, and immediately executes kinematic position snapping to reconcile any cumulative micro-slip, maintaining sub-millimeter positional fidelity throughout production cycles.

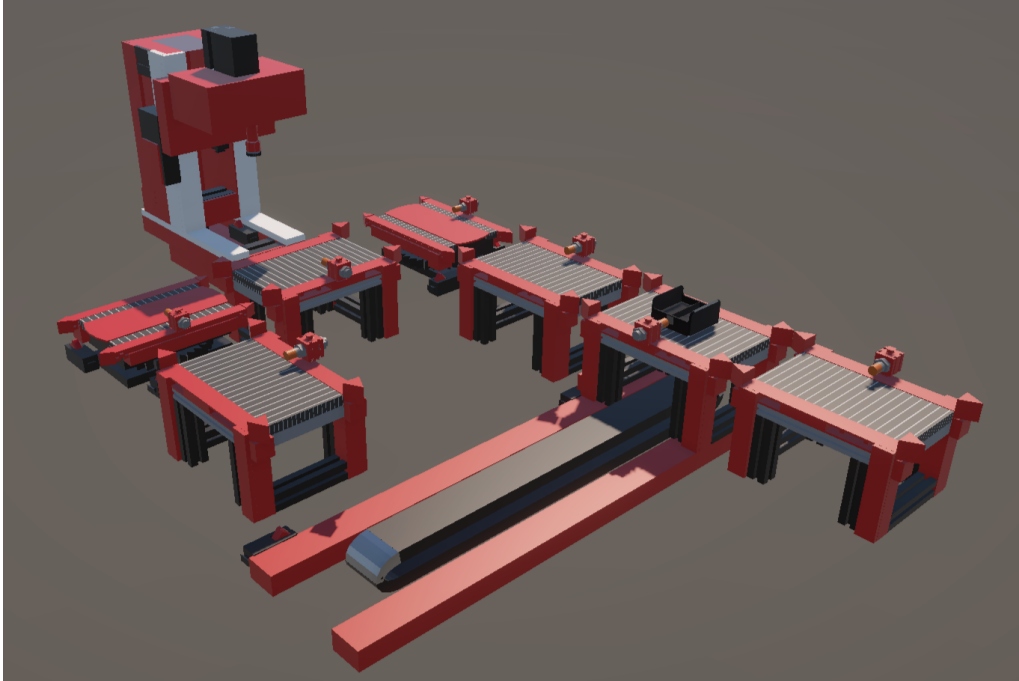


Figure 5.3: Procedural scene assembly and operational DT execution in Unity 3D, showing Conveyor G integrated into the reconstituted simulation scene.

5.2 Empirical Validation: Modular FMS Pilot Bench

To validate whether the DT operates with sufficient fidelity and responsiveness for physical automation systems, an individual benchmark was dedicated entirely to the modular FMS pilot system. Mechanically and functionally, the virtual replica instantiates the exact components of the physical workbench: three linear conveyors, two curved rotational conveyors, one translating conveyor, and the punch station, accompanied by all associated virtual sensing colliders and actuator endpoints.

To evaluate whether the runtime operates with sufficient responsiveness and determinism for industrial automation, several complementary metrics are monitored:

- **Rendering Throughput (FPS) and Target Thresholds:** In interactive 3D simulations, 60 FPS (16.6 ms per frame) is widely regarded as the target benchmark for fluid, real-time visualization and synchronized interaction. A threshold of 30 FPS (33.3 ms per frame) represents the acceptable operational minimum, below which visual stutter becomes noticeable and input latency can interfere with monitoring.
- **Mean Frame Time:** While FPS measures frequency, the frame time (in milliseconds) directly quantifies the computational cost spent by the engine in each update tick, measuring how much margin remains before missing the next frame deadline.
- **1% Low FPS:** The average frame rate of the slowest 1% of frames. Unlike the overall average FPS, which can hide momentary freezing, this metric reveals transient stutter caused by simultaneous sensor triggers or complex collision bursts.
- **P95 and P99 Tail Latency:** The 95th and 99th percentiles of frame render times. In real-time DT synchronization, these tail metrics ensure that even rare latency spikes do not breach controller communication windows or cause packet processing delays.

An automated test harness conducted 100 independent cold executions of the complete FMS cell. In each cycle, the Unity runtime ingested the AASX packages, rigged kinematic joints and surface friction physics, linked sensor callbacks, and executed the simulation. [Table 5.1](#) summarizes the performance metrics observed across the 100 test runs.

Table 5.1: Experimental results for the modular FMS pilot bench across 100 test runs.

Metric	Mean $\pm$ Std	Min	Max
Startup Initialization Time (s)	2.79 $\pm$ 0.04	2.72	2.96
Steady-State Frame Rate (FPS)	489.91 $\pm$ 49.21	352.66	694.09
1% Low Frame Rate (FPS)	272.43 $\pm$ 87.32	104.60	414.18
Mean Frame Time (ms)	2.36 $\pm$ 0.39	1.56	3.91
P95 Tail Latency (ms)	3.84 $\pm$ 1.47	2.38	8.53
P99 Tail Latency (ms)	4.35 $\pm$ 2.01	2.41	9.94

As shown in Table 5.1, procedural reconstitution of the pilot bench requires an average of only 2.79 s ($\sigma = 0.04$ s), demonstrating high determinism and repeatability across cold boots. Once initialized, the physics simulation runs at an average of 489.91 FPS, corresponding to a frame render latency of 2.36 ms. Even under transient workpiece transfer loads, the 1% low frame rate remains at 272.43 FPS, safely above standard interactive visualization rates.

Figure 5.4 illustrates the statistical dispersion of these metrics. The startup time exhibits a narrow interquartile range (2.76 s to 2.80 s), confirming that semantic archive decompression and mesh generation do not suffer from filesystem blocking. In addition, the rendering rate remains stable around 490 FPS, exceeding real-time requirements, while the 99th percentile (P99) tail latency stays below 9.94 ms across all test runs (Table 5.1).

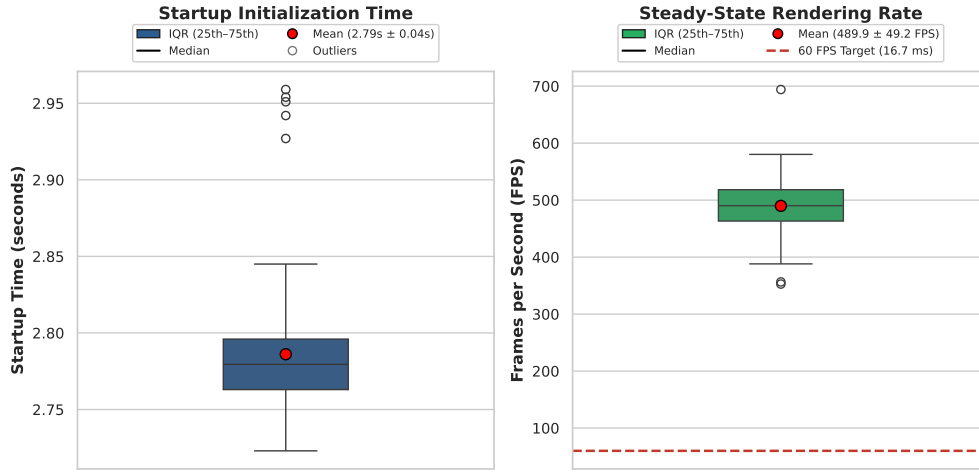


Figure 5.4: Distribution of startup initialization time and steady-state rendering frame rate across 100 experimental runs of the modular FMS pilot bench.

Figure 5.5 plots the temporal execution profile during runtime. During the startup window ($t < 2.79$ s), frame times show transient fluctuations while colliders and textures are bound. As soon as procedural initialization concludes, the system immediately transitions into a flat steady state, confirming that the DT operates without frame pacing stutter or cumulative jitter during pallet routing.

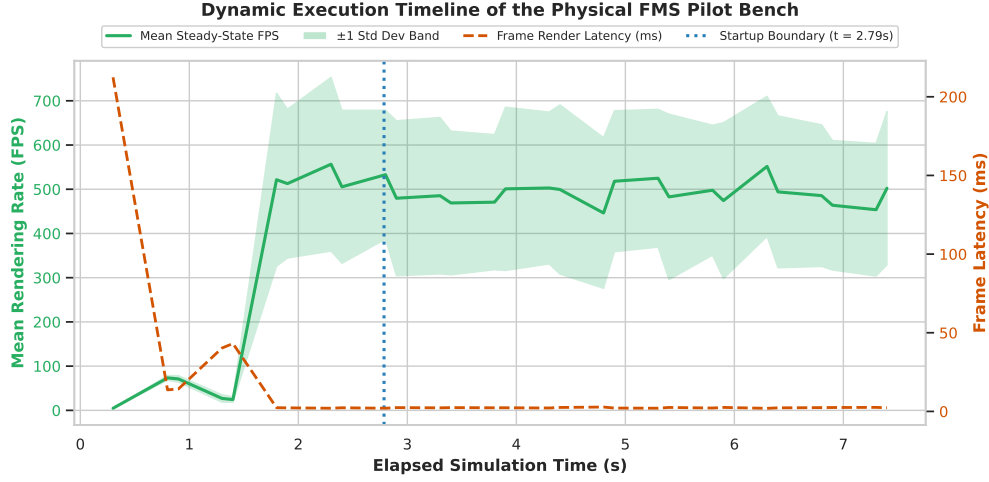


Figure 5.5: Dynamic execution timeline of the modular FMS bench, contrasting frame rate with frame latency over simulation time.

5.3 High-Density Scalability Stress Testing

While the individual FMS benchmark validates operational effectiveness for typical production cells, smart manufacturing facilities often integrate tens or hundreds of interconnected transport segments. To identify the computational ceilings of the pipeline, a parametric stress benchmark was conducted across eight discrete module density levels:

$$N \in \{5, 10, 25, 50, 100, 250, 500, 1000\} \text{ conveyor modules.}$$

5.3.1 Benchmark Methodology and Experimental Test Rig

In this setup, each module represents a complete modular asset encapsulated in an official AASX archive, containing GLB visual meshes, kinematic joints, virtual sensor pins, and behavioral logic. To replicate realistic distributed processing stations, one punch stamping station is procedurally instantiated for every five conveyor modules, placed at module index 3 and subsequently every 5 modules ($i \in \{3, 8, 13, 18, \dots\}$). At $N = 5$, 5 conveyors and 1 punch mechanism are instantiated (Figure 5.6), scaling up to 1000 conveyors and 200 punch stations at $N = 1000$.

For each scale level N , 100 independent automated runs were executed, recording metrics at 500 ms intervals over a steady-state execution window (totaling 800 simulation lifecycles). Table 5.2 summarizes the consolidated results. Note that at the highest density ($N = 1000$), the large standard deviation in frame rate reflects heavy distribution skewness caused by brief transient bursts interspersed between severe physics bottlenecks, rather than a symmetric Gaussian distribution.

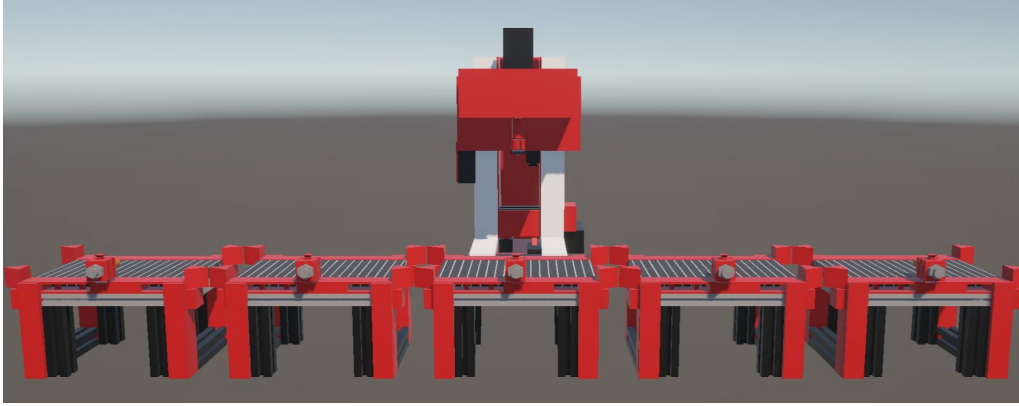


Figure 5.6: Procedural benchmark simulation layout showcasing the modular arrangement of conveyors and integrated punch stations (1 punch mechanism per 5 conveyors).

Table 5.2: Consolidated stress test results across 800 benchmark runs (Mean $\pm$ Standard Deviation).

Conveyors	Startup Time (s)	Mean FPS	Frame Time (ms)
5	2.61 $\pm$ 0.10	460.80 $\pm$ 63.73	2.49 $\pm$ 0.27
10	2.85 $\pm$ 0.04	418.03 $\pm$ 52.84	2.69 $\pm$ 0.28
25	3.66 $\pm$ 0.05	369.42 $\pm$ 52.10	3.10 $\pm$ 0.36
50	5.22 $\pm$ 0.09	344.68 $\pm$ 46.22	3.30 $\pm$ 0.37
100	8.55 $\pm$ 0.11	313.00 $\pm$ 51.02	3.73 $\pm$ 0.51
250	20.60 $\pm$ 0.40	254.24 $\pm$ 45.37	5.43 $\pm$ 1.42
500	57.45 $\pm$ 2.77	93.16 $\pm$ 20.98	14.96 $\pm$ 4.68
1000	416.70 $\pm$ 135.08	13.96 $\pm$ 43.28	342.21 $\pm$ 110.59

5.3.2 Startup Initialization and Procedural Instantiation Complexity

A critical measure of practical usability is the time required for the virtual runtime to discover, parse, and instantiate an arbitrary factory line. [Figure 5.7](#) displays the scaling trend of the total startup initialization time alongside the unit instantiation cost per module across all scale levels ($N \in \{5, \dots, 1000\}$).

The empirical results show that for realistic production line scales (between 5 and 50 conveyors), the procedural generation process is nearly instantaneous. Initializing a cell of 5 conveyors required an average of 2.61 s ($\sigma = 0.10$ s), while a 50-conveyor installation initialized in 5.22 s ($\sigma = 0.09$ s). Scaling to a facility of 100 conveyors took 8.55 s ($\sigma = 0.11$ s), and a large 250-conveyor layout was assembled in 20.60 s ($\sigma = 0.40$ s).

As shown in [Figure 5.7](#), the unit startup time per conveyor module exhibits an initial downward curve, dropping from 522.0 ms/module at $N = 5$ to 104.4 ms/module at $N = 50$, and reaching a minimum of 82.4 ms/module at $N = 250$. This higher unit time at smaller scales occurs because the Unity engine itself incurs an intrinsic baseline startup latency to bootstrap the core runtime, compile and link dynamic scripts, initialize graphics rendering pipelines, load shader variants, and allocate disk buffers. When only a small number of

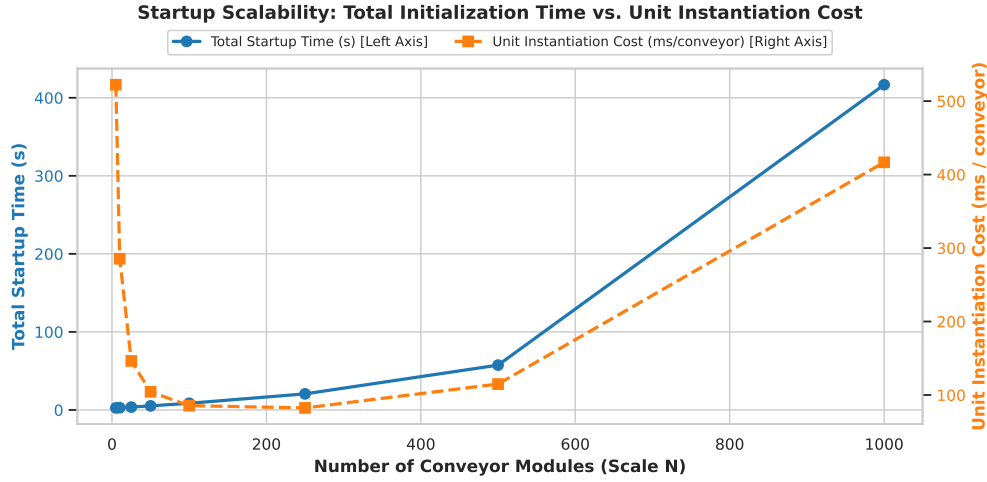


Figure 5.7: Procedural scaling complexity: total startup initialization time versus unit instantiation cost per conveyor module.

conveyors is instantiated (such as $N = 5$), this substantial fixed engine initialization cost is divided among very few assets, resulting in artificially high unit costs. As the facility scales up to 50 and 250 conveyors, this upfront static engine overhead becomes negligible, amortizing smoothly across the larger asset population until true per-module instantiation complexity dominates.

However, beyond $N = 500$, initialization time increases non-linearly, reaching 57.45 s at 500 modules and 416.70 s at 1000 modules. This escalation is driven by the main-thread bottleneck of Unity’s mesh collider generation and the sequential file decompression of multiple AASX zip archives.

5.3.3 Rendering Fluidity and Frame Rate Degradation

To evaluate visual responsiveness and stability, Figure 5.8 illustrates the degradation of mean frame rate alongside the 1% low frame rate metric (which highlights severe transient frame drops) relative to the industrial 60 FPS target (16.6 ms) and the 30 FPS critical threshold (33.3 ms).

For layouts up to 250 conveyor modules, the procedural DT operates with substantial performance headroom. The mean frame rate reaches 460.80 FPS for 5 conveyors, 344.68 FPS for 50 conveyors, and 254.24 FPS for 250 conveyors. The 1% low frame rate remains above 98.4 FPS across these scales, confirming that the simulation runs smoothly without stuttering or visual hitching during workpiece transitions.

At $N = 500$ modules, the mean frame rate drops to 93.16 FPS ($\sigma = 20.98$ FPS). While this average comfortably exceeds the 60 FPS threshold, the 1% low metric decreases to 38.4 FPS. This dip occurs when dozens of moving pallets simultaneously engage belt friction colliders, triggering bursts of narrow-phase collision calculations.

At $N = 1000$ modules, the system experiences severe bottlenecking, averaging 13.96 FPS ($\sigma = 43.28$ FPS) with 1% low values approaching zero. At this scale, the single-threaded

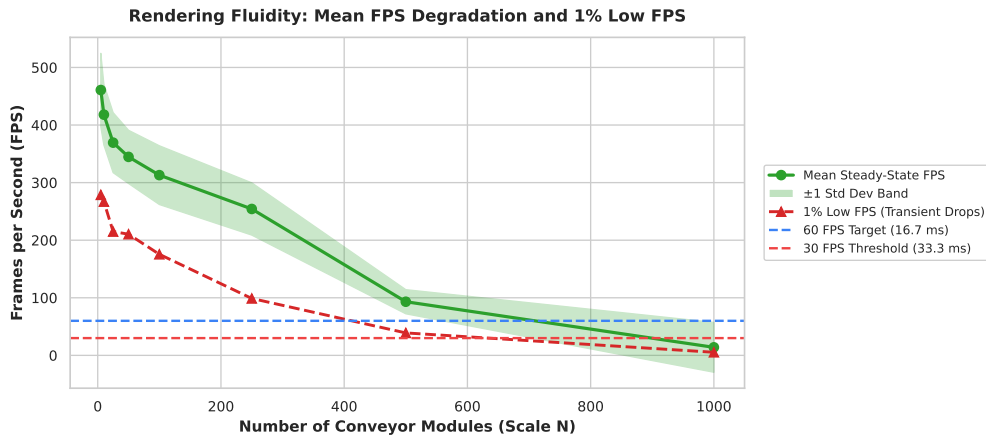


Figure 5.8: Rendering fluidity and frame rate degradation across conveyor module counts, showing mean steady-state FPS, standard deviation, and 1% low FPS.

PhysX physics update saturates the Central Processing Unit (CPU), exceeding the real-time threshold and proving that unconstrained 1000-module facilities require spatial culling or multi-threaded physics sub-stepping. The high sample standard deviation relative to the mean reflects this non-normal, heavily skewed behavior, where the engine alternates between long freeze intervals (single-digit frame rates) and brief, transient frame bursts when physics contacts temporarily settle.

5.3.4 Unit Computational Processing Cost

Real-time synchronization between physical automation hardware and a DT requires bounded computational execution times. Figure 5.9 presents the unit computational cost required to process each conveyor module per frame.

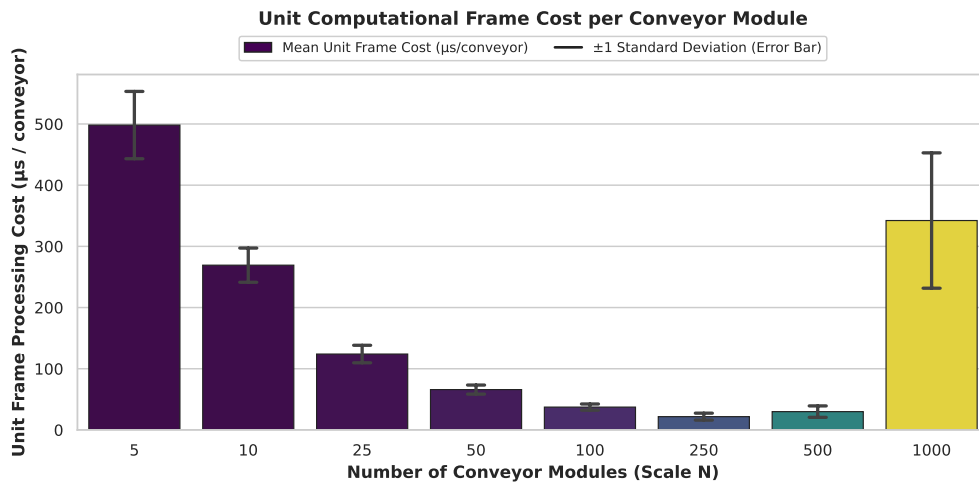


Figure 5.9: Unit computational processing cost per conveyor module in microseconds (μs /conveyor).

As shown in Figure 5.9, the unit computational cost per conveyor module reveals an

inverse scaling efficiency at moderate counts. Processing an individual conveyor requires approximately $498\ \mu\text{s}$ at $N = 5$, decreasing to $66\ \mu\text{s}$ at $N = 50$, and reaching an optimal minimum of $21.7\ \mu\text{s}$ at $N = 250$. Similar to the startup latency behavior, this inverse curve at low module counts is caused by Unity’s fixed per-frame baseline engine work (such as main-thread loop orchestration, input polling, and graphics draw call setup), which is initially distributed across only a few conveyors. Once the layout scales to 50 and 250 modules, the engine operates with maximal efficiency before inter-collider matrix checks and physics contacts cause unit costs to rise sharply to $342.2\ \mu\text{s}$ per module at $N = 1000$.

5.3.5 Dynamic Temporal Stability and Statistical Correlations

To verify that the simulation maintains stable behavior over extended operation without progressive frame degradation, [Figure 5.10](#) illustrates the dynamic FPS time-series recorded across the benchmark execution window. [Figure 5.11](#) presents the Pearson linear correlation matrix computed across all recorded performance variables.

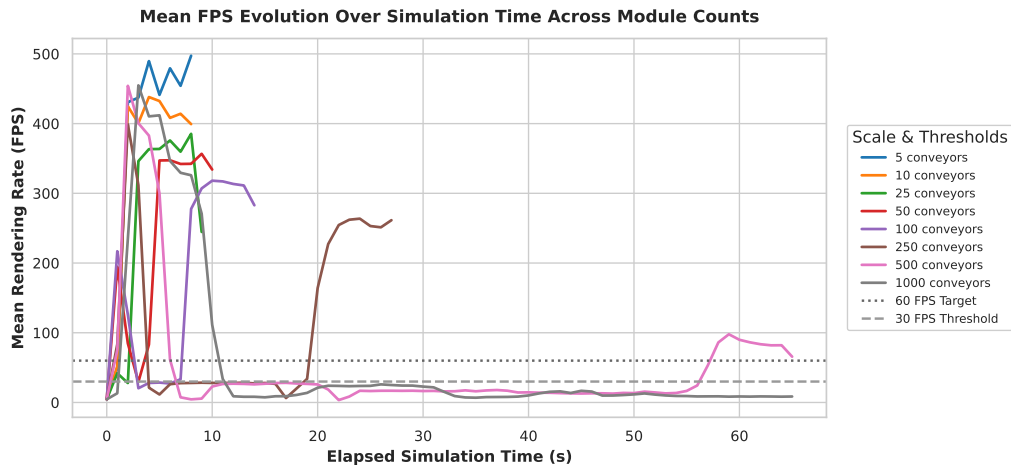


Figure 5.10: Dynamic simulation fluidity over the benchmark execution window across all evaluated module counts.

The temporal profiles in [Figure 5.10](#) demonstrate clear phase separation. During the initial seconds of each run (corresponding to the startup times identified in [Section 5.3.2](#)), frame rates exhibit transient dips as assets are instantiated into the scene. Once initialization completes, all configurations from 5 to 250 conveyors enter steady-state execution, maintaining flat, stable frame rates across the entire measurement period.

The Pearson correlation matrix in [Figure 5.11](#) provides quantitative confirmation of key architectural behaviors:

- **Conveyor Count vs. Frame Time ($r = 0.84$):** Computational frame latency correlates strongly with module count, demonstrating direct dependency on physics scene complexity.

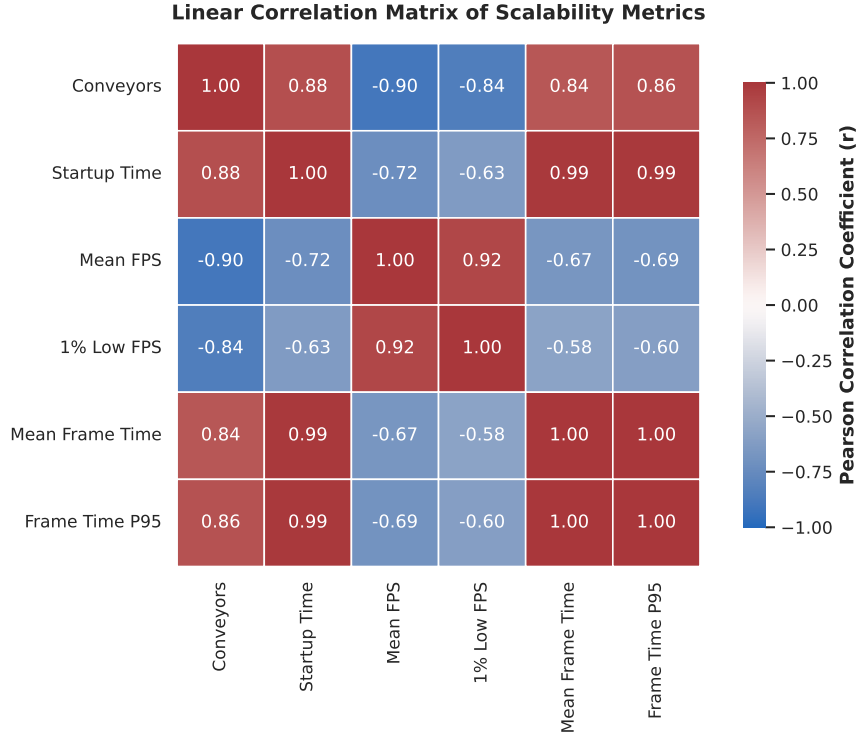


Figure 5.11: Pearson linear correlation matrix computed across procedural scalability metrics.

- **Frame Time vs. Startup Time ($r = 0.99$):** Demonstrates that the processing overhead of mesh collider creation during startup directly mirrors the ongoing cost of narrow-phase collision checks during simulation.
- **Mean FPS vs. 1% Low FPS ($r = 0.92$):** Confirms that frame drops remain tightly coupled to the underlying average performance rather than being triggered by anomalous background spikes.

5.4 Discussion and Limitations

The experimental results validate that the proposed framework can procedurally generate and execute industrial DT simulations from standardized AAS packages without pre-compiled scenes. Both the physical FMS pilot bench validation and the high-density stress testing demonstrate key practical conclusions:

1. **High-Fidelity Real-Time Feasibility:** For realistic modular production setups such as the FMS bench, the DT operates at nearly 490 FPS with latencies of 2.36 ms, providing ample performance headroom to process bidirectional PLC communication and physical synchronization without frame pacing issues. Across all realistic factory scales ($N \leq 100$ conveyors), the runtime comfortably delivers over 300 FPS and

sub-4 ms frame times, proving that real-world manufacturing cells operate with vast computational safety margins.

2. **Single-Threaded Physics Bottleneck ($N > 500$):** Unity’s default PhysX engine updates rigid-body dynamics sequentially on the main CPU thread. While the framework scales comfortably up to 250 modules (averaging over 250 FPS) and maintains real-time execution at 500 modules (93 FPS), performance drops to 14 FPS at 1000 modules. For facilities exceeding 500 active conveyors, future work should implement spatial culling or migrate rigid-body dynamics to compute shaders using Unity’s Data-Oriented Technology Stack (DOTS).
3. **Package Ingestion Bottleneck:** At large scales, opening and extracting hundreds of separate AASX zip archives introduces significant disk I/O overhead during startup (416 s at 1000 modules). Implementing an in-memory package cache or pre-indexing submodel manifests could substantially reduce initialization times for large-scale production facilities.
4. **Geometric Optimization Trade-Offs:** The fast loading times and low resource overhead observed in the benchmarks rely on the low-polygon geometric models generated in Layer 0. While these simplified meshes represent kinematic envelopes and sensor triggers accurately, they omit internal manufacturing details.

5.4.1 Systematic Verification of Research Objectives

The empirical findings from both evaluation tracks directly validate the core research objectives formulated in [Chapter 1](#):

- **Semantic Asset Integration:** Confirmed. The modular AAS submodel schema successfully united 3D visual geometry, kinematic bounds, virtual I/O pins, and declarative control rules. The exact same AASX packages drove both standalone VC in Unity and live physical hardware telemetry without modifying asset manifests.
- **Procedural Scene Assembly:** Confirmed. The runtime engine dynamically assembled all visual meshes, physical colliders, and telemetry bindings entirely in memory at launch (cold-boot initialization in 2.79 ± 0.04 s for the pilot bench and 5.22 ± 0.09 s for a 50-conveyor layout), fully eliminating pre-compiled scenes.
- **Real-Time Synchronization and Scalability:** Confirmed. Decoupling the asynchronous TCP telemetry stream from the fixed physics loop alongside sensor-triggered kinematic snapping maintained continuous physical-digital alignment without drift. Parametric stress testing mapped the deterministic operational ceiling up to 500 modules (93 FPS) and isolated the single-threaded CPU physics saturation at $N = 1000$.

Overall, the evaluation demonstrates that the framework provides responsive, bounded real-time performance for modular production lines containing up to 250 modules, while maintaining valid operational fidelity up to 500 modules, confirming the feasibility of procedural DT generation in smart manufacturing.

CONCLUSION AND FUTURE WORK

This dissertation addressed the engineering bottleneck of creating, configuring, and synchronizing industrial DTs. Traditional approaches to virtual commissioning and digital replication rely on fragmented CAD files, vendor-locked simulation software, and manual matching of kinematics, sensors, and I/O addresses. This reliance makes the deployment of DTs prohibitively labor-intensive and error-prone. To overcome these limitations, this research proposed, implemented, and empirically validated an open, modular framework that procedurally generates interactive 3D simulation environments directly from standardized AAS packages without pre-compiled scenes.

This final chapter synthesizes the primary conclusions and technical contributions of the dissertation, discusses the operational limitations identified during testing, and details concrete directions for future research.

6.1 Conclusions and Summary of Contributions

Guided by the DSR methodology, this research delivered a multi-tier pipeline organized across four distinct layers: physical measurements and low-polygon asset digitization (Layer 0), AAS authoring and semantic packaging (Layer 1), containerized registry and submodel storage via Eclipse BaSyx v2 (Layer 2), and dynamic procedural runtime instantiation and drift correction in Unity 3D (Layer 3).

The experimental validation conducted in [Chapter 5](#) substantiated the feasibility, determinism, and performance boundaries of this approach:

1. **Fidelity on Mechatronic Scale Hardware:** Cold-boot instantiation of the complete modular FMS bench (comprising linear, rotational, and transfer conveyors, punch stamping, optical sensors, and physical PLC integration) averaged 2.79 ± 0.04 s. In steady state, the physics runtime sustained an average rendering rate of 489.91 ± 49.21 FPS with a mean frame latency of 2.36 ± 0.39 ms. The 1% low frame rate remained above 272 FPS, and the P99 tail latency stayed below 9.94 ms. These metrics confirm that the procedural DT provides ample real-time headroom for continuous bidirectional telemetry and visual synchronization.

2. **Algorithmic Scalability and Real-Time Limits:** High-density parametric stress testing from $N = 5$ to $N = 1000$ conveyor modules revealed that the framework scales with near-linear complexity up to 250 modules (254 FPS at 20.6 s startup time) and retains real-time execution up to 500 modules (93 FPS). The empirical evaluation identified single-threaded CPU physics resolution as the primary bottleneck beyond 500 modules, establishing clear operational sizing guidelines for procedural DTs.
3. **Decoupled Control Logic and Virtual Commissioning:** By packaging operational rules within semantic *ControlLogic* submodels, virtual cells operate in pure software VC mode without requiring physical hardware. Transitioning to operational DT mode requires only redirecting sensor state streams to live TCP telemetry from the shop floor, eliminating configuration fragmentation.

6.1.1 Operational Scope and Process Boundaries

While these results prove the viability of procedural digital twins, it is essential to delineate the operational boundaries of this research. The proposed framework was specifically designed and validated for discrete manufacturing systems, where production steps are defined by solid, identifiable workpieces moving across mechanical handling units, triggering binary sensors, and receiving discrete actuation commands. Continuous process industries (such as chemical refining, fluid mixing, or thermodynamic processing) operate on fluid dynamics, thermal gradients, and differential equations that cannot be represented solely by rigid-body contact physics or discrete state snapping. As detailed in the future research directions, extending the architecture to such hybrid and continuous environments requires coupling the semantic AAS descriptions to numerical co-simulation solvers via standards such as FMI.

6.2 Future Work

The experimental findings presented in this dissertation validate the feasibility of procedural digital twins for discrete modular transport systems. By demonstrating automated scene generation, sub-3-second startup times, and deterministic multi-hundred FPS synchronization on mechatronic hardware, the framework proves that semantic standards can eliminate manual simulation rebuilds.

At the same time, the quantitative stress tests and architectural boundaries identified in [Chapter 5](#) highlight key challenges that arise when transitioning from controlled pilot benches to large-scale, heterogeneous manufacturing plants. Modern industrial environments operate across multiple physical domains, incorporate articulated robotic manipulators, execute diverse programming logic beyond simple boolean conditions, and require immersive visual interfaces for human operators on the shop floor.

To bridge these gaps and expand the applicability of the proposed open architecture, six key research trajectories are identified:

6.2.1 Physics Acceleration via Unity DOTS and Massively Parallel Compute Shaders

As demonstrated in [Section 5.3.4](#), scaling beyond 500 active modules exposes the single-threaded CPU update boundary of Unity’s conventional PhysX implementation. To scale procedural DTs to gigafactory density ($N \geq 1000$ modules and tens of thousands of dynamic rigidbodies):

- **Adoption of Unity DOTS:** Transforming asset instances from object-oriented GameObjects into memory-contiguous entities via the Entity Component System (ECS), the C# Job System, and the Burst Compiler. This transition allows kinematics, belt velocities, and bounding-box overlap tests to be distributed across all available CPU cores.
- **Compute Shader Physics:** Offloading contact-friction calculations and pallet surface kinematics to the GPU using compute shaders, enabling real-time execution of massive discrete manufacturing lines without CPU saturation.

6.2.2 Robotic Manipulation Integration via Robot Operating System (ROS) and ROS 2

The current framework was benchmarked primarily on material handling and discrete stamping mechanisms. Expanding the pipeline to articulate industrial manipulators and mobile logistics requires standardizing robotic kinematics:

- **Unified Robot Description Format (URDF) and Kinematic Tree Ingestion:** Standardizing the encapsulation of URDF joint definitions, link mass properties, and transmission limits inside AAS submodels.
- **Bidirectional ROS / ROS 2 Interfaces:** Connecting Unity’s DT runtime to ROS 2 nodes via standardized bridges (e.g., *ROS-TCP-Connector*). This allows motion planners (such as MoveIt 2) to command virtual robotic arms in VC mode, with joint state trajectories subsequently verified against physical robot controllers over micro-ROS or industrial Ethernet.

6.2.3 Expansion to Continuous and Hybrid Non-Discrete Manufacturing Systems

The present implementation focuses on discrete manufacturing, where events are triggered by item arrivals and binary sensor states. Many industrial processes operate under continuous or hybrid dynamics (such as chemical batching, fluid flow, thermodynamic heating, and continuous extrusion):

- **FMI Co-Simulation:** Integrating FMI/FMU standards within the AAS ecosystem. Differential equations representing thermal dissipation, pressure dynamics,

or material degradation can be solved inside specialized numerical solvers (e.g., OpenModelica, MATLAB/Simulink) while exchanging boundary values cyclically with the 3D physics runtime.

- **Continuous State Submodels:** Specifying volumetric flow rates, temperature fields, and viscosity curves within semantic property hierarchies, bridging discrete kinematics with physical transport equations.

6.2.4 Extensible Control Logic Interpreters: C, Python, Scratch, and Node-RED

The rule engine currently parses declarative IF-THEN conditions stored in JSON submodels. While effective for simple sequencing, complex manufacturing machinery requires expressive, widely adopted programming paradigms:

- **Native Scripting and Micro-Runtimes (C / Python):** Embedding sandboxed execution engines (such as WebAssembly runtimes or embedded Python/Lua interpreters) directly inside the 3D engine. This allows control engineers to upload procedural control routines written in native C or Python into the AAS, which are executed dynamically without modifying the simulation host.
- **Visual and Block-Based Programming (Scratch / Blockly):** Providing educational and rapid-prototyping environments where operators can construct manufacturing sequences using drag-and-drop functional blocks that compile directly into *ControlLogic* submodel schemas.
- **Flow-Based Orchestration via Node-RED:** Supporting external execution engines such as Node-RED. Virtual sensors push MQTT telemetry to visual dataflow pipelines, which execute supervisory coordination and dispatch control signals back to virtual actuators in real time.

6.2.5 Multi-Agent System (MAS) Coordination and Autonomous Negotiation

While the current architecture validates procedural digital twin generation and control rule synchronization from static or central AAS manifests, modern reconfigurable manufacturing relies on decentralized decision-making. Future iterations can couple each procedural DT asset with an autonomous software agent to form a collaborative MAS:

- **Autonomous Asset Agents for Plug-and-Produce:** As proposed by Rocha et al. [3], wrapping modular mechatronic assets into agent-driven AAS units enables dynamic plug-and-produce. Instead of relying on a centralized layout topology, individual conveyor modules and processing stations can autonomously announce their capabilities, negotiate workpiece routing via contract net protocols, and discover neighboring transfer points.

- **Agent-to-Twin Communication Protocols:** Implementing standard agent communication languages (such as FIPA-ACL over MQTT or TCP) directly into the simulation runtime. Software agents representing physical orders or material pallets can query the virtual asset capabilities, request alternative routing paths when a bottleneck or fault occurs, and validate proposed schedule adaptations inside the procedural twin before executing them on physical PLCs.

6.2.6 Immersive Interfaces: Virtual Reality and Augmented Reality (Virtual Reality (VR)/AR)

Because the runtime engine is developed natively in Unity 3D, extending the procedural pipeline to immersive extended reality headsets (such as Meta Quest, Apple Vision Pro, or Microsoft HoloLens) represents a natural and highly valuable progression:

- **Immersive VR Operator Training and Commissioning:** Allowing maintenance personnel and plant operators to navigate procedurally generated factories at 1:1 scale before physical installation, testing manual interventions, safety zones, and ergonomic clearance without hazards.
- **On-Floor AR Digital Overlays:** Superimposing live procedural DT data directly onto physical machinery in real time using spatial tracking. Operators equipped with AR headsets can visualize operational parameters (such as internal sensor states, pneumatic pressures, kinematic speeds, and active control logic transitions) positioned directly over physical conveyor tracks and actuator enclosures.
- **Remote Collaborative Teleoperation:** Using WebRTC streaming and multi-user networked sessions to allow geographically dispersed automation engineers to inspect anomalous shop-floor events, interact with virtual control panels, and trigger coordinated maintenance routines simultaneously.

This dissertation confirms that open semantic architectures can successfully automate the creation and synchronization of industrial DTs. By combining open AAS standards, modular 3D meshes, and procedural runtimes, the proposed framework eliminates manual scene re-engineering and provides a scalable foundation for next-generation Industry 4.0 smart manufacturing. All artifacts, models, and source code supporting this work are publicly hosted at <https://github.com/TSGouveia/AAS-DT-Framework>.

BIBLIOGRAPHY

- [1] M. Q. Khan, M. A. H. Alvi, H. H. Nawaz, and M. Umar. “Impact of Digital Twins on Real Practices in Manufacturing Industries”. In: *Inventions* 10 (6 2025-11), p. 106. ISSN: 24115134. DOI: [10.3390/INVENTIONS10060106](https://doi.org/10.3390/INVENTIONS10060106) (cit. on pp. 2, 7, 9–11, 19).
- [2] T. A. Abdel-Aty, E. Negri, and S. Galparoli. “Asset Administration Shell in Manufacturing: Applications and Relationship with Digital Twin”. In: *IFAC-PapersOnLine* 55 (10 2022-01), pp. 2533–2538. ISSN: 2405-8963. DOI: [10.1016/J.IFACOL.2022.10.090](https://doi.org/10.1016/J.IFACOL.2022.10.090) (cit. on pp. 2, 7, 9, 13, 14).
- [3] A. D. Rocha, M. Arvana, N. Freitas, R. M. Dinis, T. Gouveia, D. Machado, and J. Barata. “Human-Centric Digital Twin-Driven Approach for Plug-and-Produce in Modular Cyber-Physical Production Systems”. In: *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. 2024, pp. 1–7. DOI: [10.1109/ETFA61755.2024.10710730](https://doi.org/10.1109/ETFA61755.2024.10710730) (cit. on pp. 2, 7, 25, 26, 47, 84).
- [4] F. Tao, H. Zhang, A. Liu, and A. Y. Nee. “Digital Twin in Industry: State-of-the-Art”. In: *IEEE Transactions on Industrial Informatics* 15 (4 2019-04), pp. 2405–2415. ISSN: 15513203. DOI: [10.1109/TII.2018.2873186](https://doi.org/10.1109/TII.2018.2873186) (cit. on pp. 2, 8, 9, 11, 12, 19, 27).
- [5] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn. “Digital Twin in manufacturing: A categorical literature review and classification”. In: *IFAC-PapersOnLine* 51 (11 2018-01), pp. 1016–1022. ISSN: 2405-8963. DOI: [10.1016/J.IFACOL.2018.08.474](https://doi.org/10.1016/J.IFACOL.2018.08.474) (cit. on pp. 2, 9–11, 19, 27, 28).
- [6] M. Pajpach, R. Pribiš, P. Drahoš, E. Kučera, and O. Haffner. “Design of an Educational-development Platform for Digital Twins using the Interoperability of the OPC UA Standard and Industry 4.0 Components”. In: *International Conference on Electrical, Computer, Communications and Mechatronics Engineering, ICECCME 2023* (2023). DOI: [10.1109/ICECCME57830.2023.10252941](https://doi.org/10.1109/ICECCME57830.2023.10252941) (cit. on pp. 2, 12, 16, 18, 19, 24, 28).
- [7] D. Lu, T. Lu, Y. Lei, F. Zitian, W. Yubo, and F. Jiajun. “Design for the Industry device function model with the AutomationML based on AAS”. In: *2025 IEEE 23rd International Conference on Industrial Informatics (INDIN)* (2025-07), pp. 1–3. DOI: [10.1109/INDIN64977.2025.11279009](https://doi.org/10.1109/INDIN64977.2025.11279009) (cit. on pp. 2, 8, 26).

-
- [8] S. Schaper, F. G. Listl, T. Schenk, S. Grimm, A. Ast, and A. Verl. "Automatic Behavior Simulation Model Generation for Virtual Commissioning of Machine Tools through Engineering Data Integration". In: *IEEE International Conference on Emerging Technologies and Factory Automation, ETFA* (2025). ISSN: 19460759. DOI: [10.1109/ETFA65518.2025.11205741](https://doi.org/10.1109/ETFA65518.2025.11205741) (cit. on pp. 2, 19–21, 23–25, 28).
 - [9] J. Chowdhury, K. Stockman, and J. D. M. D. Kooning. "Automating Asset Generation of Motion Systems from CAD Data in a Game Engine Using FMI". In: (2025-07), pp. 1–7. DOI: [10.1109/ICPS65515.2025.11087902](https://doi.org/10.1109/ICPS65515.2025.11087902) (cit. on pp. 2, 16, 17, 21, 24).
 - [10] A. Alexopoulos, A. Bompotas, N. R. Kalogeropoulos, P. Kechagias, A. P. Kalogeras, and C. Alexakos. "A Model-Based Approach to Automated Digital Twin Generation in Manufacturing". In: (2025-10). URL: <https://arxiv.org/abs/2511.03742v1> (cit. on pp. 2, 16, 19–26).
 - [11] N. Striffler and T. Voigt. "Concepts and trends of virtual commissioning – A comprehensive review". In: *Journal of Manufacturing Systems* 71 (2023-12), pp. 664–680. ISSN: 0278-6125. DOI: [10.1016/j.jmsy.2023.10.013](https://doi.org/10.1016/j.jmsy.2023.10.013) (cit. on pp. 2, 11, 16, 17, 24, 27, 28).
 - [12] A. Lüder, A. K. Behnert, F. Rinker, and S. Biffl. "Generating Industry 4.0 Asset Administration Shells with Data from Engineering Data Logistics". In: *IEEE International Conference on Emerging Technologies and Factory Automation, ETFA 2020-September* (2020-09), pp. 867–874. ISSN: 19460759. DOI: [10.1109/ETFA46521.2020.9212149](https://doi.org/10.1109/ETFA46521.2020.9212149) (cit. on pp. 3, 19, 20, 28).
 - [13] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee. "A Design Science Research Methodology for Information Systems Research". In: *Journal of Management Information Systems* 24.3 (2007), pp. 45–77. DOI: [10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302) (cit. on p. 5).
 - [14] M. Hermann, T. Pentek, and B. Otto. "Design principles for industrie 4.0 scenarios". In: *Proceedings of the Annual Hawaii International Conference on System Sciences* 2016-March (2016-03), pp. 3928–3937. ISSN: 15301605. DOI: [10.1109/HICSS.2016.488](https://doi.org/10.1109/HICSS.2016.488) (cit. on p. 6).
 - [15] H. Kagermann and W. Wahlster. "Ten Years of Industrie 4.0". In: *Informatik Spektrum* 45.4 (2022), pp. 237–242. DOI: [10.1007/s00287-022-01473-8](https://doi.org/10.1007/s00287-022-01473-8) (cit. on p. 6).
 - [16] H. R. Ahvanouee, J. Hoos, B. Sautter, M. Kiele-Dunsche, D. Arnold, K. Heidrich, J. Bleibdrey, J. Schmelter, H. Haag, and A. Fay. "Asset Administration Shells for Integrated Toolchains and Collaborative Automation Engineering: Insights from the Factory-X Project". In: (2025-07), pp. 1–6. DOI: [10.1109/ICPS65515.2025.11087834](https://doi.org/10.1109/ICPS65515.2025.11087834) (cit. on pp. 7, 12, 14, 19–21, 28).
 - [17] L. D. Xu, E. L. Xu, and L. Li. "Industry 4.0: state of the art and future trends". In: *International Journal of Production Research* 56 (8 2018), pp. 2941–2962. ISSN: 1366588X. DOI: [10.1080/00207543.2018.1444806](https://doi.org/10.1080/00207543.2018.1444806) (cit. on pp. 7, 8).

- [18] B. A. Talkhestani, T. Jung, B. Lindemann, N. Sahlab, N. Jazdi, W. Schloegl, and M. Weyrich. "An architecture of an Intelligent Digital Twin in a Cyber-Physical Production System". In: *At-Automatisierungstechnik* 67 (9 2019-09), pp. 762–782. ISSN: 2196677X. DOI: [10.1515/AUTO-2019-0039](https://doi.org/10.1515/AUTO-2019-0039) (cit. on pp. 7, 12, 26).
- [19] H. Amthiou, M. Arioua, and T. Benbarrad. "Digital Twins in Industry 4.0: A Literature Review". In: *ITM Web of Conferences* 52 (2023), p. 01002. DOI: [10.1051/ITMCONF/20235201002](https://doi.org/10.1051/ITMCONF/20235201002) (cit. on pp. 7, 8).
- [20] D. Jones, C. Snider, A. Nassehi, J. Yon, and B. Hicks. "Characterising the Digital Twin: A systematic literature review". In: *CIRP Journal of Manufacturing Science and Technology* 29 (2020-05), pp. 36–52. ISSN: 1755-5817. DOI: [10.1016/J.CIRPJ.2020.02.002](https://doi.org/10.1016/J.CIRPJ.2020.02.002) (cit. on pp. 7–9, 11, 19).
- [21] R. Drath, A. Lüder, J. Peschke, and L. Hundt. "AutomationML - The glue for seamless Automation engineering". In: *IEEE International Conference on Emerging Technologies and Factory Automation, ETFA* (2008), pp. 616–623. DOI: [10.1109/ETFA.2008.4638461](https://doi.org/10.1109/ETFA.2008.4638461) (cit. on pp. 8, 12).
- [22] M. Pajpach, M. Šlauka, R. Pribiš, P. Drahoš, E. Kučera, and O. Haffner. "Asset Administration Shell - Key-enabling technology of interoperability in Industry 4.0". In: *Proceedings of the International Conference on Cybernetics and Informatics, K and I* (2025 2025). ISSN: 2767875X. DOI: [10.1109/KI64036.2025.10916454](https://doi.org/10.1109/KI64036.2025.10916454) (cit. on pp. 8, 9, 13–15).
- [23] C. Semeraro, M. Lezoche, H. Panetto, and M. Dassisti. "Digital twin paradigm: A systematic literature review". In: *Computers in Industry* 130 (2021-09). ISSN: 01663615. DOI: [10.1016/J.COMPIND.2021.103469](https://doi.org/10.1016/J.COMPIND.2021.103469) (cit. on pp. 9, 12, 19).
- [24] M. Grieves. "Digital Twin: Manufacturing Excellence through Virtual Factory Replication". In: *White Paper*. Dassault Systèmes, 2014, pp. 1–7 (cit. on p. 9).
- [25] M. Shafto, M. Conroy, R. Doyle, E. Glaessgen, C. Kemp, J. LeMoigne, and L. Wang. *Draft Modeling, Simulation, Information Technology & Processing Roadmap: Technology Area 11*. Tech. rep. TA 11. National Aeronautics and Space Administration (NASA), 2012 (cit. on p. 10).
- [26] S. Gil, P. H. Mikkelsen, C. Gomes, and P. G. Larsen. "Survey on open-source digital twin frameworks—A case study approach". In: *Software: Practice and Experience* 54 (6 2024-06), pp. 929–960. ISSN: 1097-024X. DOI: [10.1002/SPE.3305](https://doi.org/10.1002/SPE.3305) (cit. on pp. 10, 28).
- [27] A. Barnard. "Digital Twin Frameworks: A Comparative Study of AAS, OPC UA, and AML". In: *International Conference on Electrical, Computer, and Energy Technologies, ICECET 2024* (2024). DOI: [10.1109/ICECET61485.2024.10698583](https://doi.org/10.1109/ICECET61485.2024.10698583) (cit. on pp. 12, 13).
- [28] M. Reichardt, M. Gundall, D. Buch, P. Stricker, and H. D. Schotten. "Artificial Intelligence Meets Digital Twin: The AI Asset Administration Shell". In: *2024 IEEE 3rd Industrial Electronics Society Annual On-Line Conference (ONCON)*. IEEE, 2024-12, pp. 1–6. DOI: [10.1109/oncon62778.2024.10931277](https://doi.org/10.1109/oncon62778.2024.10931277) (cit. on pp. 13, 22, 23).

-
- [29] A. P. Vaz, E. C. Tobío, D. F. Martínez, and L. A. Ferreira. “Automated Configuration of BaSyx DataBridge using Standardized Asset Administration Shell Interface Modelling”. In: *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2025-09, pp. 1–7. doi: [10.1109/etfa65518.2025.11205726](https://doi.org/10.1109/etfa65518.2025.11205726) (cit. on pp. 14, 15, 21, 24).
 - [30] V. M. Caldana, L. Fumagalli, and E. P. Godoy. “A Proposal for Production Control of Legacy Flexible Manufacturing Systems using Asset Administration Shell and Digital Twin”. In: *Proceedings of the International Conference on Industrial Engineering and Operations Management*. IEOM Society International, 2025-05. doi: [10.46254/sa6.20250091](https://doi.org/10.46254/sa6.20250091) (cit. on p. 15).
 - [31] M. Pajpach, P. Drahos, R. Pribis, and E. Kucera. “Educational-development Workplace for Digital Twins using the OPC UA and Unity 3D”. In: *Proceedings of the 31st International Conference on Cybernetics and Informatics, K and I 2022* (2022). doi: [10.1109/KI55792.2022.9925933](https://doi.org/10.1109/KI55792.2022.9925933) (cit. on pp. 16, 18, 24).
 - [32] R. Zheng, B. Lu, Y. Chang, and Y. Guo. “Design of a Digital Twin System for Substations Based on Unity”. In: *2024 4th International Symposium on Artificial Intelligence and Intelligent Manufacturing, AIIM 2024* (2024), pp. 111–115. doi: [10.1109/AIIM64537.2024.10934638](https://doi.org/10.1109/AIIM64537.2024.10934638) (cit. on p. 16).
 - [33] A. Garcia, X. Oregui, M. Ojer, S. Valdes, J. R. Sanchez, and I. Mendizabal. “AAS to support the Operator as a Service vision”. In: *IEEE International Conference on Emerging Technologies and Factory Automation, ETFA* (2025). issn: 19460759. doi: [10.1109/ETFA65518.2025.11205634](https://doi.org/10.1109/ETFA65518.2025.11205634) (cit. on pp. 16, 18, 24).
 - [34] C. G. Lee and S. C. Park. “Survey on the virtual commissioning of manufacturing systems”. In: *Journal of Computational Design and Engineering* 1 (3 2014-07), pp. 213–222. issn: 2288-4300. doi: [10.7315/JCDE.2014.021](https://doi.org/10.7315/JCDE.2014.021) (cit. on pp. 17, 28).
 - [35] P. Juhlin, A. Karaagac, J. C. Schlake, S. Gruner, and J. Ruckert. “Cloud-enabled Drive-Motor-Load Simulation Platform using Asset Administration Shell and Functional Mockup Units”. In: *IEEE International Conference on Emerging Technologies and Factory Automation, ETFA 2022-September* (2022). issn: 19460759. doi: [10.1109/ETFA52439.2022.9921678](https://doi.org/10.1109/ETFA52439.2022.9921678) (cit. on p. 17).
 - [36] J. Moyne, Y. Qamsane, E. C. Balta, I. Kovalenko, J. Faris, K. Barton, and D. M. Tilbury. “A Requirements Driven Digital Twin Framework: Specification and Opportunities”. In: *IEEE Access* 8 (2020), pp. 107781–107801. issn: 21693536. doi: [10.1109/ACCESS.2020.3000437](https://doi.org/10.1109/ACCESS.2020.3000437) (cit. on p. 19).
 - [37] F. A. Garcia, P. Ereemeev, H. Devriendt, F. Naets, H. Metin, and M. Ozer. “Automatic generation of digital-twins in advanced manufacturing: A feasibility study”. In: *2023 International Conference on Control, Automation and Diagnosis, ICCAD 2023* (2023). doi: [10.1109/ICCAD57653.2023.10152364](https://doi.org/10.1109/ICCAD57653.2023.10152364) (cit. on pp. 19, 20, 22, 23).

- [38] T. Koch, J. Bonari, M. von Danwitz, and A. Popp. "Automatic Generation of Simulation Models for Digital Twins from State-of-the-Art Simulation Frameworks". In: *Proceedings - IEEE International Symposium on Distributed Simulation and Real-Time Applications, DS-RT* (2024), pp. 132–135. ISSN: 15506525. DOI: [10.1109/DS-RT62209.2024.00031](https://doi.org/10.1109/DS-RT62209.2024.00031) (cit. on pp. 20, 21, 24, 27).
- [39] M. Herzog, S. Dalecke, J. Koch, and C. Grimm. "Integration of Generatable Digital Twins into Model-Driven Engineering Processes". In: *International Conference on System of Systems Engineering, SoSE* (2025 2025). ISSN: 28353161. DOI: [10.1109/SOSE66311.2025.11083827](https://doi.org/10.1109/SOSE66311.2025.11083827) (cit. on p. 20).
- [40] P. Richard, A. T. Bernhard, A. Luxenburger, H. Gössling, and A. Wagner. "Agent-based Initialization of Autonomous Mobile Robots: Leveraging the Asset Administration Shell for automated Commissioning". In: *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2025-09, pp. 1–8. DOI: [10.1109/etfa65518.2025.11205779](https://doi.org/10.1109/etfa65518.2025.11205779) (cit. on p. 21).
- [41] H. Heermann, M. Herzog, J. Koch, and C. Grimm. "Generating Digital Twins from SysMLv2 Models". In: *IEEE International Conference on Industrial Informatics (INDIN)* (2024). ISSN: 19354576. DOI: [10.1109/INDIN58382.2024.10774476](https://doi.org/10.1109/INDIN58382.2024.10774476) (cit. on p. 21).
- [42] D. Gollner, T. Klausmann, R. Rasor, and R. Dumitrescu. "Use Case Driven Digital Twin Generation". In: *Proceedings - 2022 IEEE 5th International Conference on Industrial Cyber-Physical Systems, ICPS 2022* (2022). DOI: [10.1109/ICPS51978.2022.9816907](https://doi.org/10.1109/ICPS51978.2022.9816907) (cit. on pp. 21, 24, 26, 28).
- [43] G. Lugaresi and A. Matta. "Automated Digital Twins Generation for Manufacturing Systems: a Case Study". In: *IFAC-PapersOnLine* 54 (1 2021-01), pp. 749–754. ISSN: 2405-8963. DOI: [10.1016/j.ifacol.2021.08.087](https://doi.org/10.1016/j.ifacol.2021.08.087) (cit. on pp. 22, 23).
- [44] J. Dong and L. Ren. "A Digital Twin Modeling Code Generation Framework based on Large Language Model". In: *IECON Proceedings (Industrial Electronics Conference)* (2024). ISSN: 25771647. DOI: [10.1109/IECON55916.2024.10905976](https://doi.org/10.1109/IECON55916.2024.10905976) (cit. on pp. 22–24).
- [45] Y. Xia, Z. Xiao, N. Jazdi, and M. Weyrich. "Generation of Asset Administration Shell with Large Language Model Agents: Toward Semantic Interoperability in Digital Twins in the Context of Industry 4.0". In: *IEEE Access* 12 (2024), pp. 84863–84877. DOI: [10.1109/ACCESS.2024.3415470](https://doi.org/10.1109/ACCESS.2024.3415470) (cit. on pp. 22, 23).
- [46] D. Alemão, A. D. Rocha, S. Nikghadam-Hojjati, and J. Barata. "How to Design Scheduling Solutions for Smart Manufacturing Environments Using RAMI 4.0?" In: *IEEE Access* 10 (2022), pp. 71284–71298. DOI: [10.1109/ACCESS.2022.3187974](https://doi.org/10.1109/ACCESS.2022.3187974) (cit. on pp. 26, 45).

PHYSICAL FMS CONTROLLER SPECIFICATIONS AND I/O PIN MAPPING

This appendix presents the hardware specifications and I/O pin mapping for the Industrial Shields M-DUINO 57R+ PLC used in the FMS testbed.

A.1 M-DUINO 57R+ Controller Technical Specifications

Table A.1 summarizes the main hardware specifications and operating parameters of the M-DUINO 57R+ controller.

Table A.1: Hardware specifications of the Industrial Shields M-DUINO 57R+ PLC controller.

Specification Parameter	Value / Feature
Base Microcontroller Board	Arduino Mega 2560 (ATmega2560 @ 16 MHz)
Memory (Flash / SRAM / EEPROM)	256 KB (8 KB bootloader) / 8 KB / 4 KB
Operating Input Voltage	12 to 24 V DC (Rated: 24 V DC, Fuse protection: 2.5 A)
Max Rated Power / Max Current	10.20 W / 1.5 A
Integrated Network Interfaces	W5500 Ethernet, RS-485, RS-232, SPI, I2C, Serial TTL, USB 2.0
Storage Expansion	MicroSD card slot (μ SD), Real-Time Clock (RTC)
Total I/O Capacity	18 Inputs / 31 Outputs (49 channels total)
Digital Optoisolated Inputs	6 channels (supporting hardware interrupts)
Analog / Digital Inputs	12 channels (0 to 10 V DC / 0 to 24 V DC, 10-bit ADC)
Optoisolated Relay Outputs	23 channels (24 V DC @ 2 A / 230 V AC @ 4 A max)
Analog / Digital / PWM Outputs	8 channels (0 to 10 V DC / 0 to 24 V DC, 8-bit PWM)
Operating Temperature / Humidity	0 °C to 60 °C / 10% to 90% (non-condensing)
Enclosure Dimensions	119.5 mm $\times$ 119.3 mm $\times$ 101 mm (Weight: 598 g)

A.2 Digital Sensor Inputs (I-Registers)

Table A.2 details the digital input pin mapping (I-Registers) for the optical and proximity sensors across the three controller modules.

Table A.2: Digital sensor input pin mapping on the Industrial Shields M-DUINO 57R+ controller.

Module Number	Register Index	Function	Active Logic
0	0	Conveyor B	High
0	1	Conveyor A	High
0	2	Punch Down	Low
0	3	Punch Up	Low
0	4	Punch Forward	High
0	5	Punch Backward	High
1	0	Conveyor C	High
1	1	Conveyor D	High
1	2	Rotate E Give	Low
1	3	Rotate E Receive	Low
1	4	Rotate C Give	Low
1	5	Rotate C Receive	Low
2	0	Conveyor E	High
2	1	Conveyor F	High
2	2	Conveyor A Left	Low
2	3	Conveyor A Right	Low

A.3 Relay Actuator Outputs (R-Registers)

Table A.3 details the relay output pin mapping (R-Registers) used to control conveyor motors and pneumatic actuators across the three controller modules.

Table A.3: Relay actuator output pin mapping on the Industrial Shields M-DUINO 57R+ controller.

Module Number	Register Index	Function
0	2	Punch Up
0	3	Punch Backward
0	4	Conveyor E
0	5	Conveyor E Receive
0	6	Conveyor C Give
0	7	Conveyor A Backwards
0	8	Conveyor A Right
1	3	Punch Down
1	4	Conveyor F
1	5	Rotate E Give
1	6	Conveyor C
1	7	Conveyor B
1	8	Conveyor A Left
2	3	Punch Spin
2	4	Punch Forward
2	6	Conveyor D
2	7	Conveyor C Receive
2	8	Conveyor A Forward

